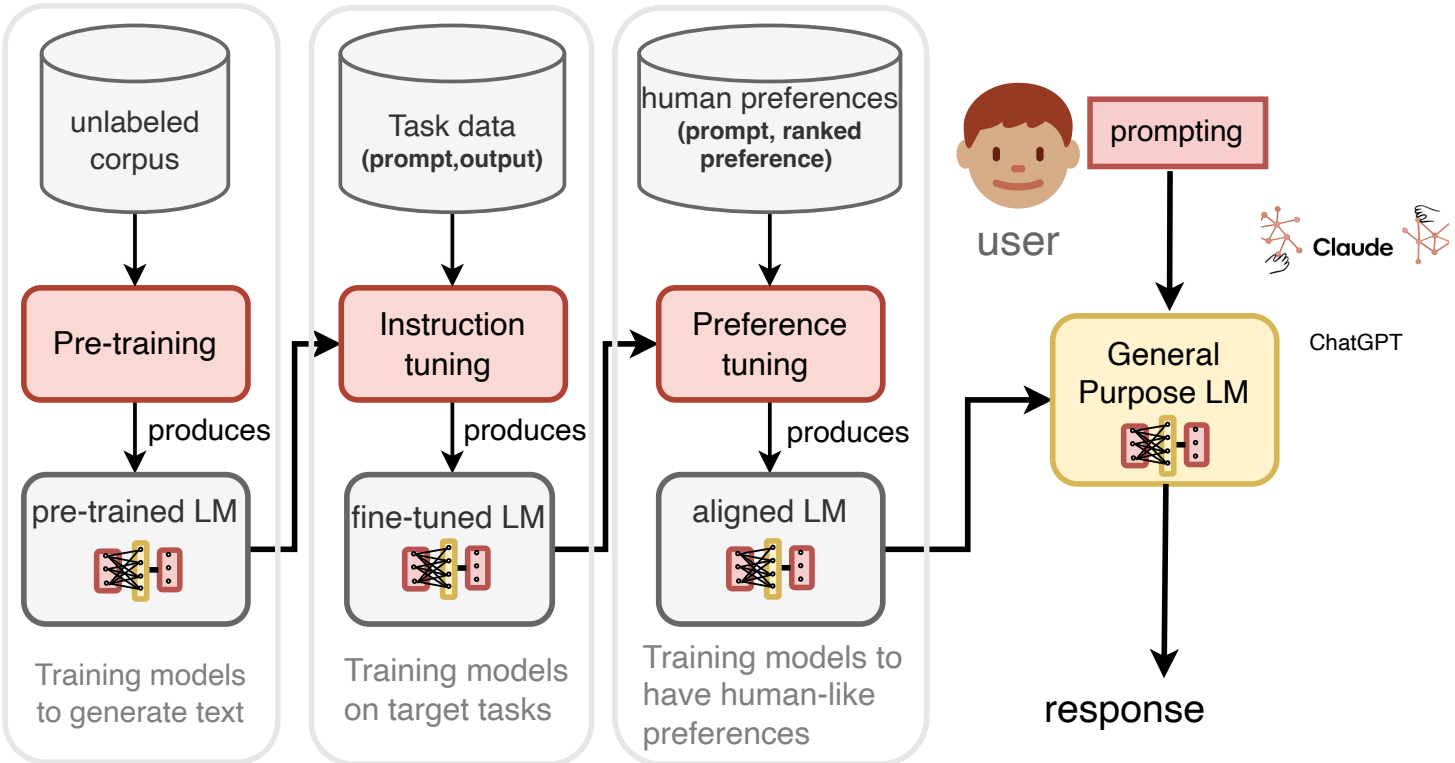
Discrete Methods for Modern AI: lecture 2

Kyle Richardson

Nov 9 2026

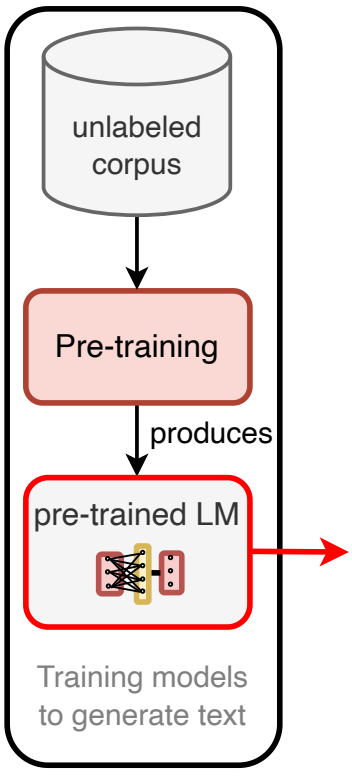
Reminder of what the course is about

How do we get to general-purpose LLMs? **recipe**



- This is the **modern AI** part of the course: **large language models** (LLMs), how they are built, trained and used.

The landscape of generative AI research: **Foundations**



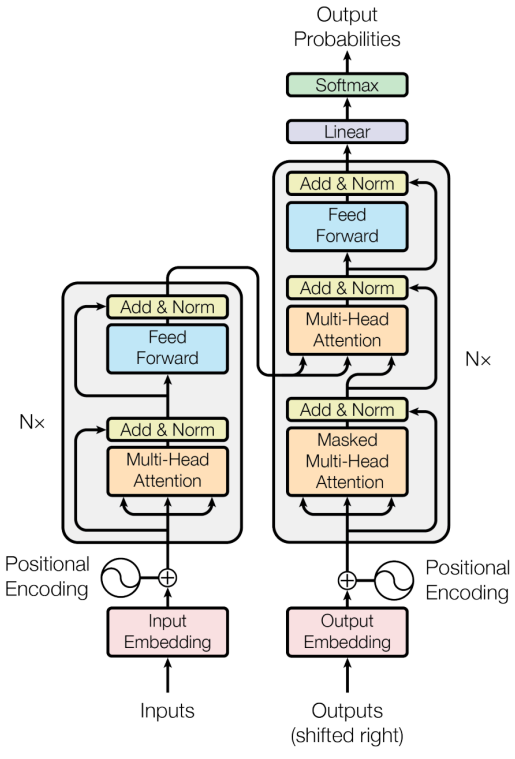
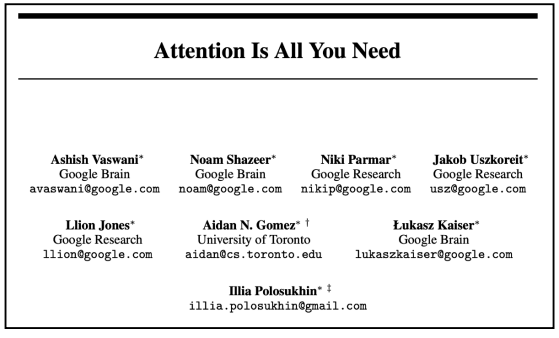


Figure 1: The Transformer - model architecture.

Model architectures



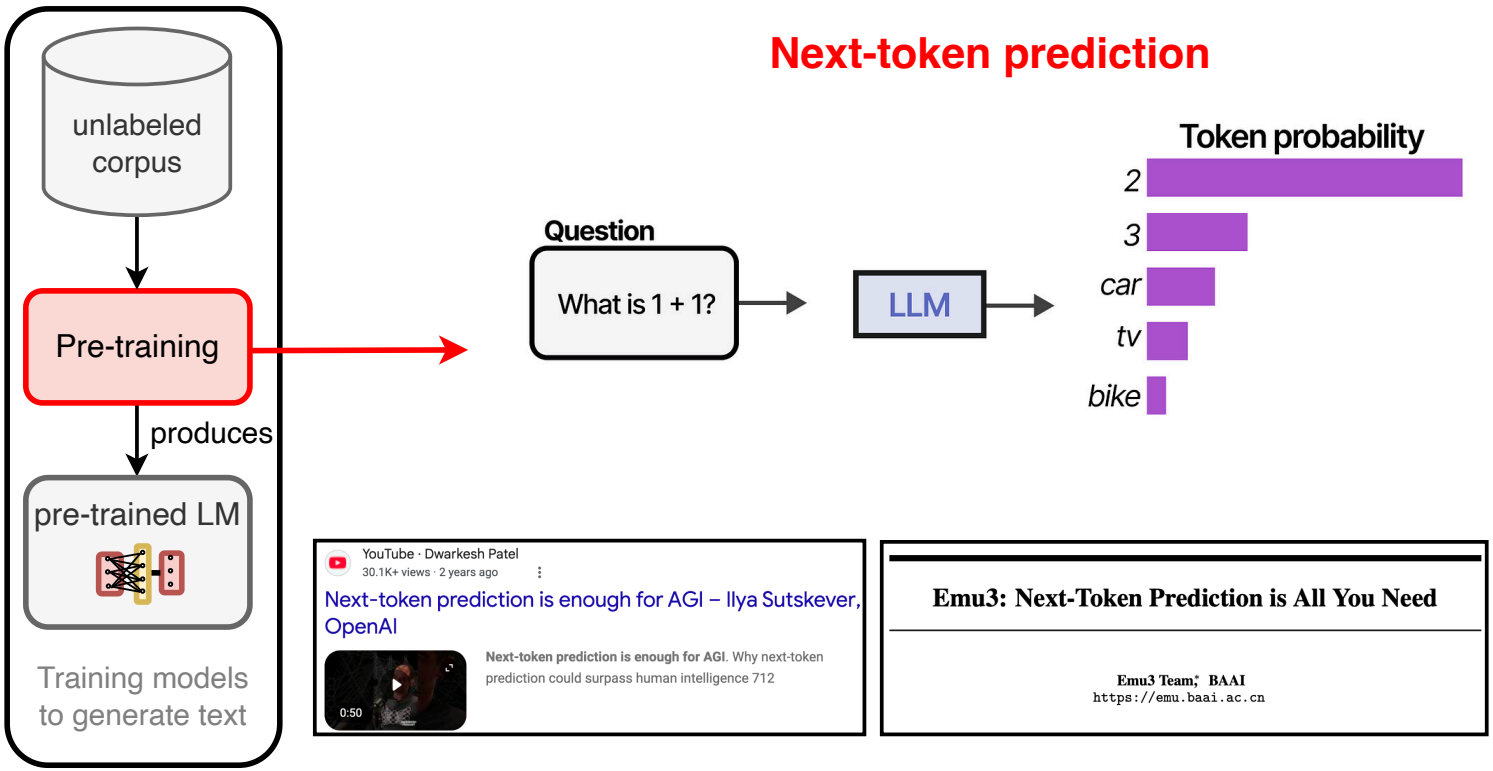
Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

- **Model architecture design**: what kinds of problems can specific architectures (e.g., transformers) express? Are there clear limitations?

This is what we will talk about today

The landscape of generative AI research: **Foundations**



- **Pre-training and evaluation**: Why is next-token prediction such an effective signal and how do we effectively measure model knowledge?

The landscape of generative AI research: **Alignment**

Loss function design

$$-\log\sigma\left(\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)}-\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

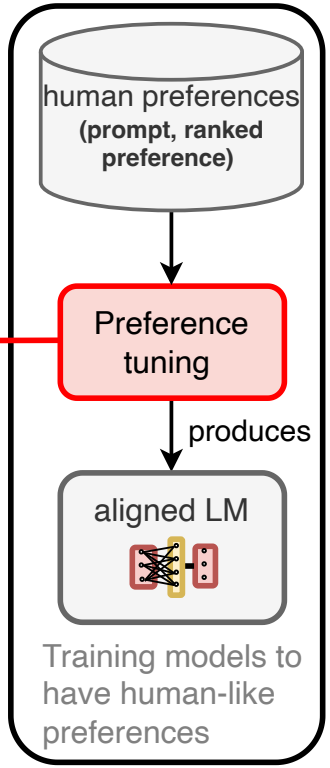
~

Training language models to follow instructions with human feedback

Long Ouyang* Jeff Wu* Xu Jiang* Diogo Almeida* Carroll L. Wainwright*
Pamela Mishkin* Chong Zhang Sandhini Agarwal Katarina Slama Alex Ray

Direct Preference Optimization: Your Language Model is Secretly a Reward Model

Rafael Rafailov*† Archit Sharma*† Eric Mitchell*†
Stefano Ermon†‡ Christopher D. Manning† Chelsea Finn†

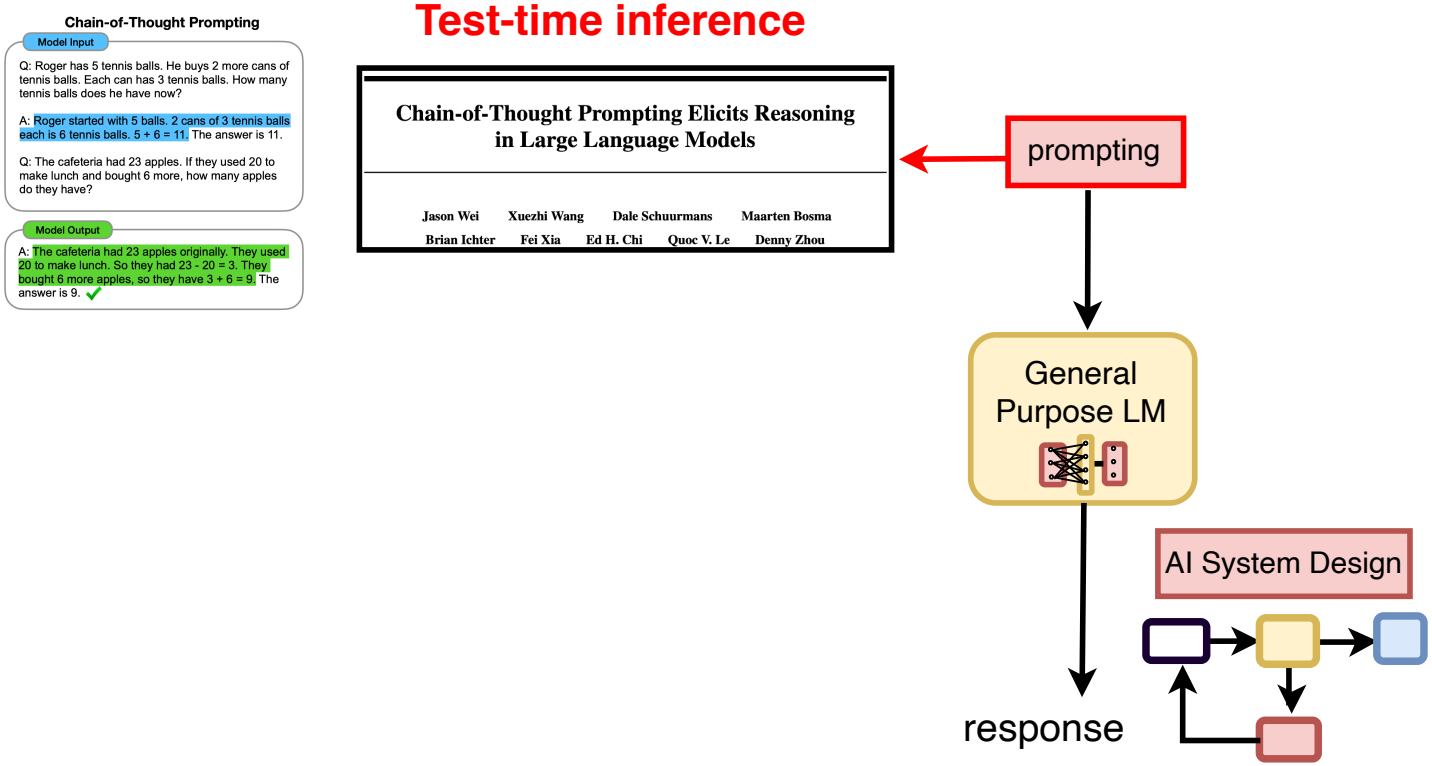


- **Loss design**: What do we do when we train models to preferences? Does the optimization have an internal logic?

The landscape of generative AI research: **Alignment**

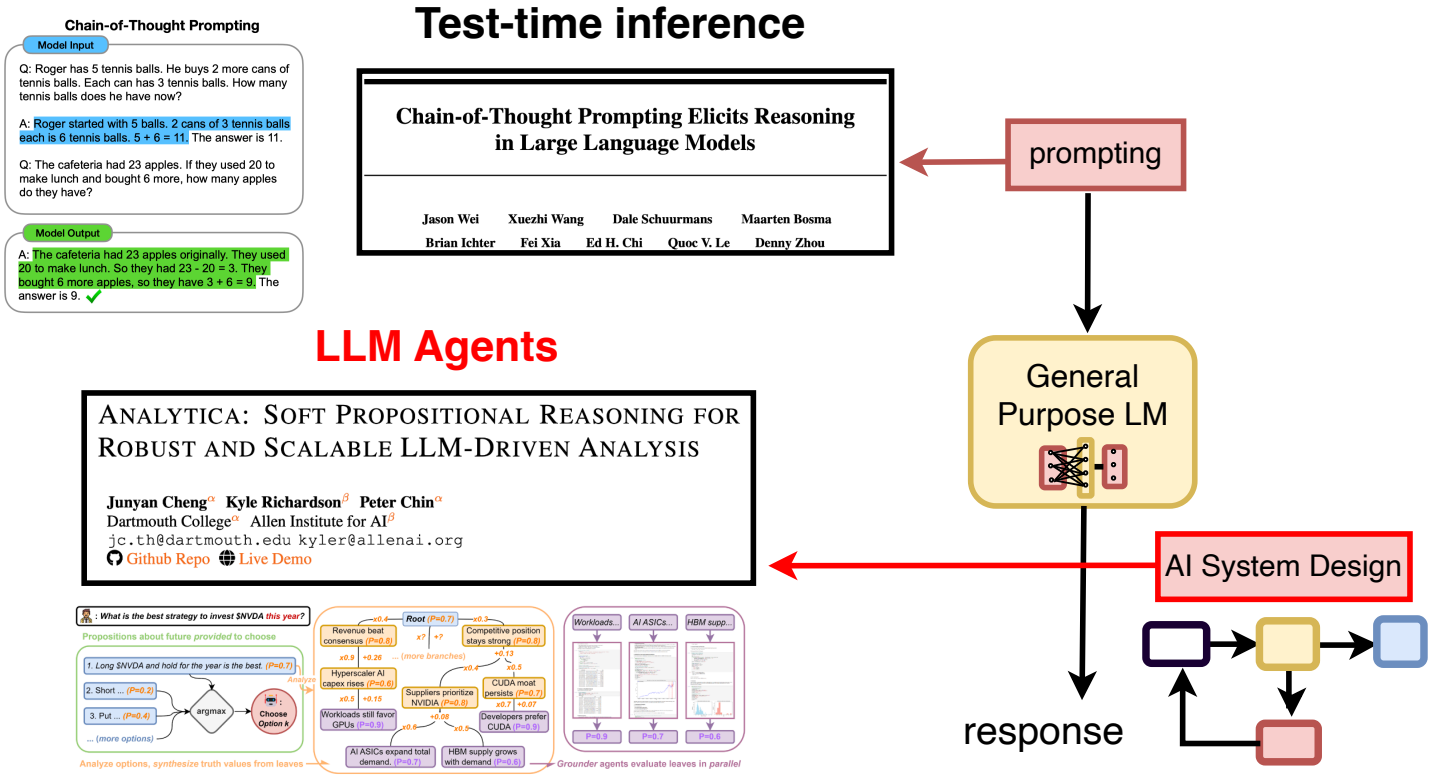
What do these computations mean?

The landscape of generative AI research: **Inference**



- **Test-time inference**: What kinds of reasoning structures do LLMs build? How can we know if the reasoning is correct and sound?

The landscape of generative AI research: **Inference**



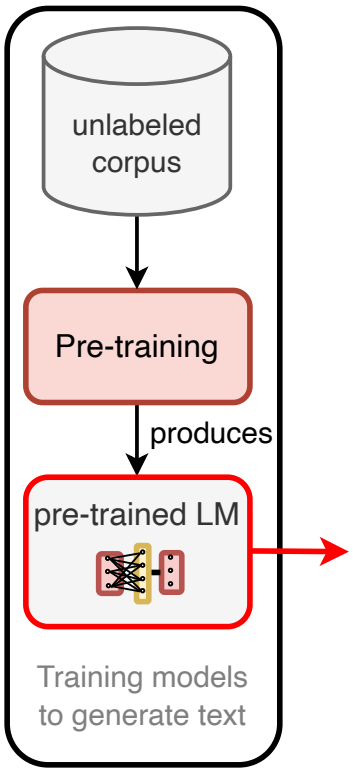
- **Test-time inference**: What kinds of reasoning structures do LLMs build?
  How can we know if the reasoning is correct and sound?

LLMs build structures, what are these structures?

The landscape of generative AI research: **Inference**

How are all these areas connected to one another?

Programming languages for LLM research



Restricted Access Sequence Processing Language (Rasp)

Thinking Like Transformers

Gail Weiss [1] Yoav Goldberg [2 3] Eran Yahav [1]

```
1  def num_prevs(bools) {
2      prevs = select(indices,indices,<=);
3      return (indices+1) *
4             aggregate(prevs,
5                       indicator(bools));
6  }
7  n_opens = num_prevs(tokens=="(");
8  n_closes = num_prevs(tokens==")");
9  balance = n_opens - n_closes;
10 prev_imbalances = num_prevs(balance<0);
11 dyck1PTF = "F" if prev_imbalances > 0
12                   else
13            ("T" if balance==0 else "P");
```

Figure 15: RASP code for computing Dyck-1-PTF with the parentheses (and).

Masked Hard-Attention Transformers Recognize Exactly the Star-Free Languages

Andy Yang, University of Notre Dame; David Chiang, University of Notre Dame; Dana Angluin, Yale University

$$P_\ell(i) := \blacktriangleright_j\,[j < i, 1]\;\; Q_\ell(j) : 0$$
$$S_r(i) := \blacktriangleleft_i\,[j > i, 1]\;\; Q_r(j) : 0.$$
$$I(i) := (Q_\ell(i) \wedge S_r(i)) \vee (Q_r(i) \wedge P_\ell(i)).$$
$$B_\ell(i) := \blacktriangleright_j\,[j < i, \neg I(j)]\;\; Q_\ell(j) : 0$$
$$A_r(i) := \blacktriangleleft_j\,[j > i, \neg I(j)]\;\; Q_r(j) : 0$$
$$C(i) := I(i) \vee (Q_\ell(i) \wedge A_r(i)) \vee (Q_r(i) \wedge B_\ell(i)).$$
$$Y(i) := \blacktriangleright_j\,[1, \neg C(j)]\;\; 0 : 1.$$

Figure: B-rasp for computing Dyck-1.

- **Rasp**: Languages that capture the underlying computational model of transformers (Weiss et al.2021; Yang and Chiang2024; Yang et al.2024; Huang et al.2026).

Thinking Like Transformers

Gail Weiss [1] **Yoav Goldberg** [2 3] **Eran Yahav** [1]

Abstract

What is the computational model behind a Transformer? Where recurrent neural networks have direct parallels in finite state machines, allowing clear discussion and thought around architecture variants or trained models, Transformers have no such familiar parallel. In this paper we aim to change that, proposing a computational model for the transformer-encoder in the form of a programming language. We map the basic components of a transformer-encoder—attention and feed-forward computation—into simple primitives, around which we form a programming language: the Restricted Access Sequence Processing Language (RASP). We show how RASP can be used to program solutions to tasks that could conceivably be learned by a Transformer, and how a Transformer can be trained to mimic a RASP solution. In particular, we provide RASP programs for histograms, sorting, and Dyck-languages. We further use our model to relate their difficulty in terms of the number of required layers and attention heads: analyzing a RASP program implies a maximum number of heads and layers necessary to encode a task in a transformer. Finally, we see how insights gained from our abstraction might be used to explain phenomena seen in recent works.

a transformer operates at a higher-level of abstraction, reasoning in terms of a composition of *sequence operations* rather than neural network primitives.

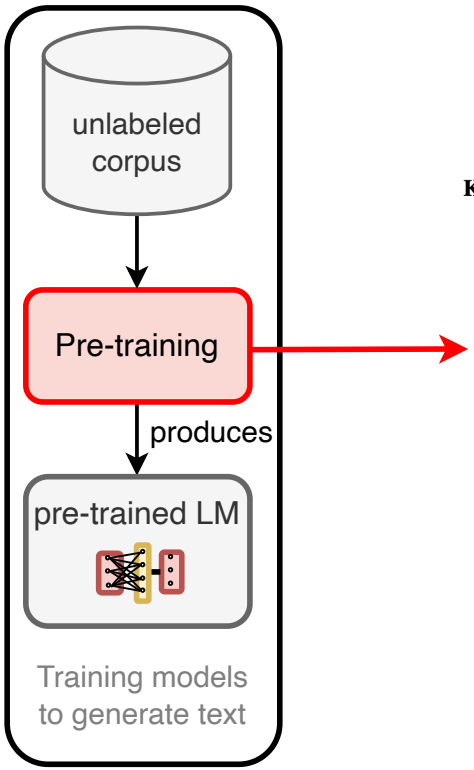
We are inspired by the use of automata as an abstract computational model for recurrent neural networks (RNNs). Using automata as an abstraction for RNNs has enabled a long line of work, including extraction of automata from RNNs (Omlin & Giles, 1996; Weiss et al., 2018b; Ayache et al., 2018), analysis of RNNs' practical expressive power in terms of automata (Weiss et al., 2018a; Rabusseau et al., 2019; Merrill, 2019; Merrill et al., 2020b), and even augmentations based on automata variants (Joulin & Mikolov, 2015). Previous work on transformers explores their computational power, but does not provide a computational model (Yun et al., 2020; Hahn, 2020; Pérez et al., 2021).

Thinking in terms of the RASP model can help derive computational results. Bhattamishra et al. (2020) and Ebrahimi et al. (2020) explore the ability of transformers to recognize Dyck-k languages, with Bhattamishra et al. providing a construction by which Transformer-encoders can recognize a simplified variant of Dyck-k. Using RASP, we succinctly express the construction of (Bhattamishra et al., 2020) as a short program, and further improve it to show, for the first time, that transformers can fully recognize Dyck-k for all k.

Scaling up the complexity, Clark et al. (2020) showed empirically that transformer networks can learn to perform multi-step logical reasoning over first order logical formulas

We will cover this paper today (Weiss et al.2021)

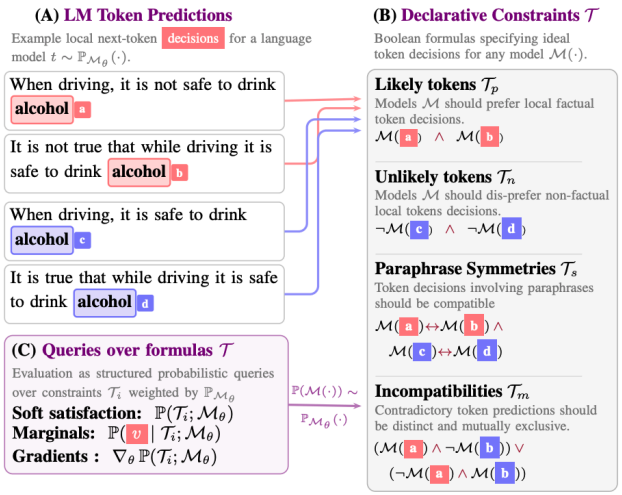
Programming languages for LLM research



ModelLog

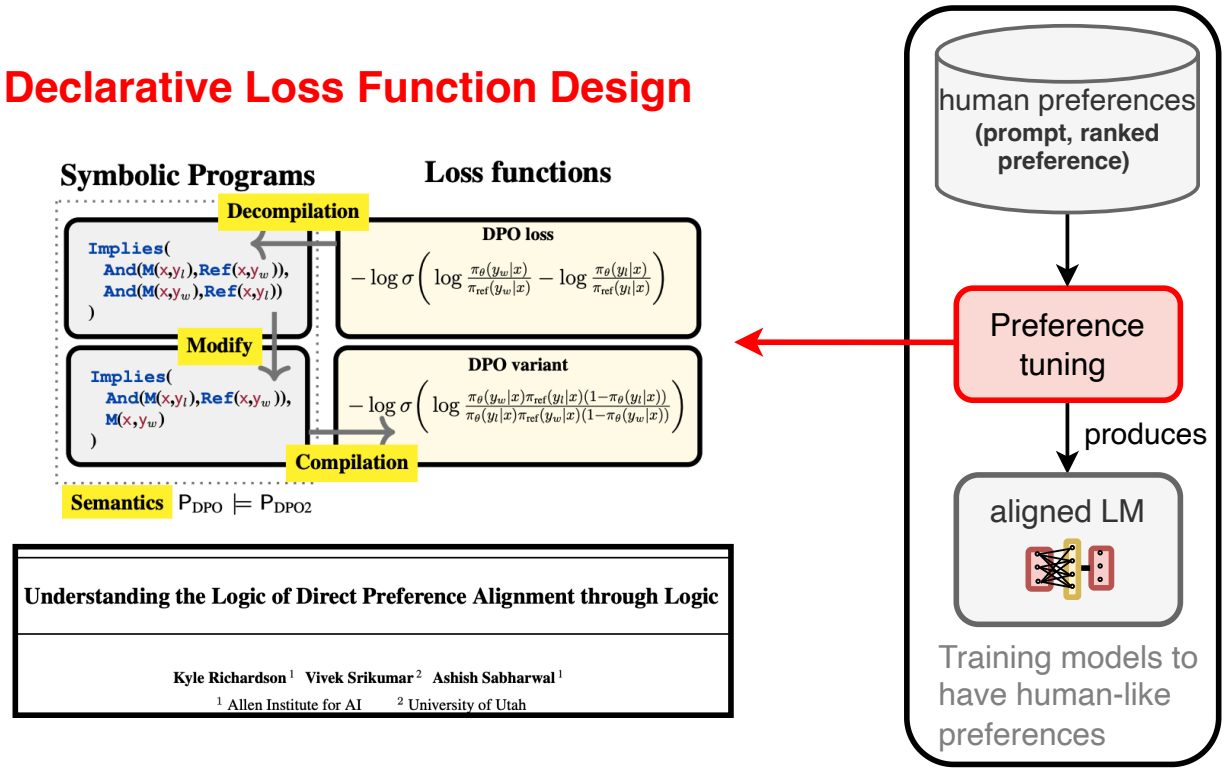
From Token Probabilities to Semantic Constraints: Towards Declarative Probabilistic Evaluation of Language Models

Kyle Richardson[1] Cullen Anderson[2] Pranav Balakrishnan[2] Takuto Ban[2] Daksha Ladia[2]
Ankita Gupta[2] Marisa Hudspeth[2]



- Declarative-style probabilistic programs, probabilistic logic, takes much inspiration from PLP (De Raedt and Kimmig2015; Fierens et al.2015; Manhaeve et al.2018).

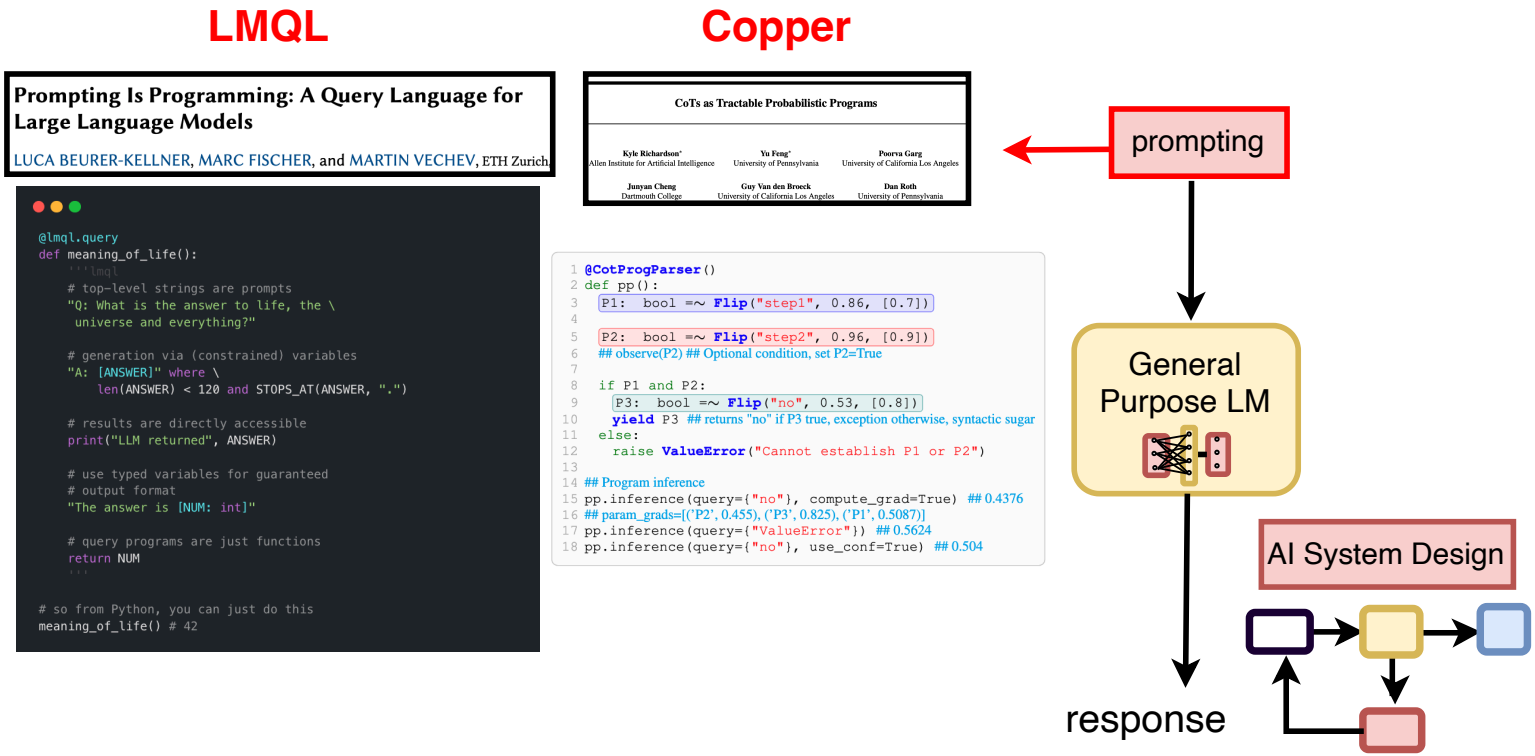
Programming languages for LLM research



- Declarative-style probabilistic programs, probabilistic logic, takes much inspiration from PLP (De Raedt and Kimmig2015; Fierens et al.2015; Manhaeve et al.2018).

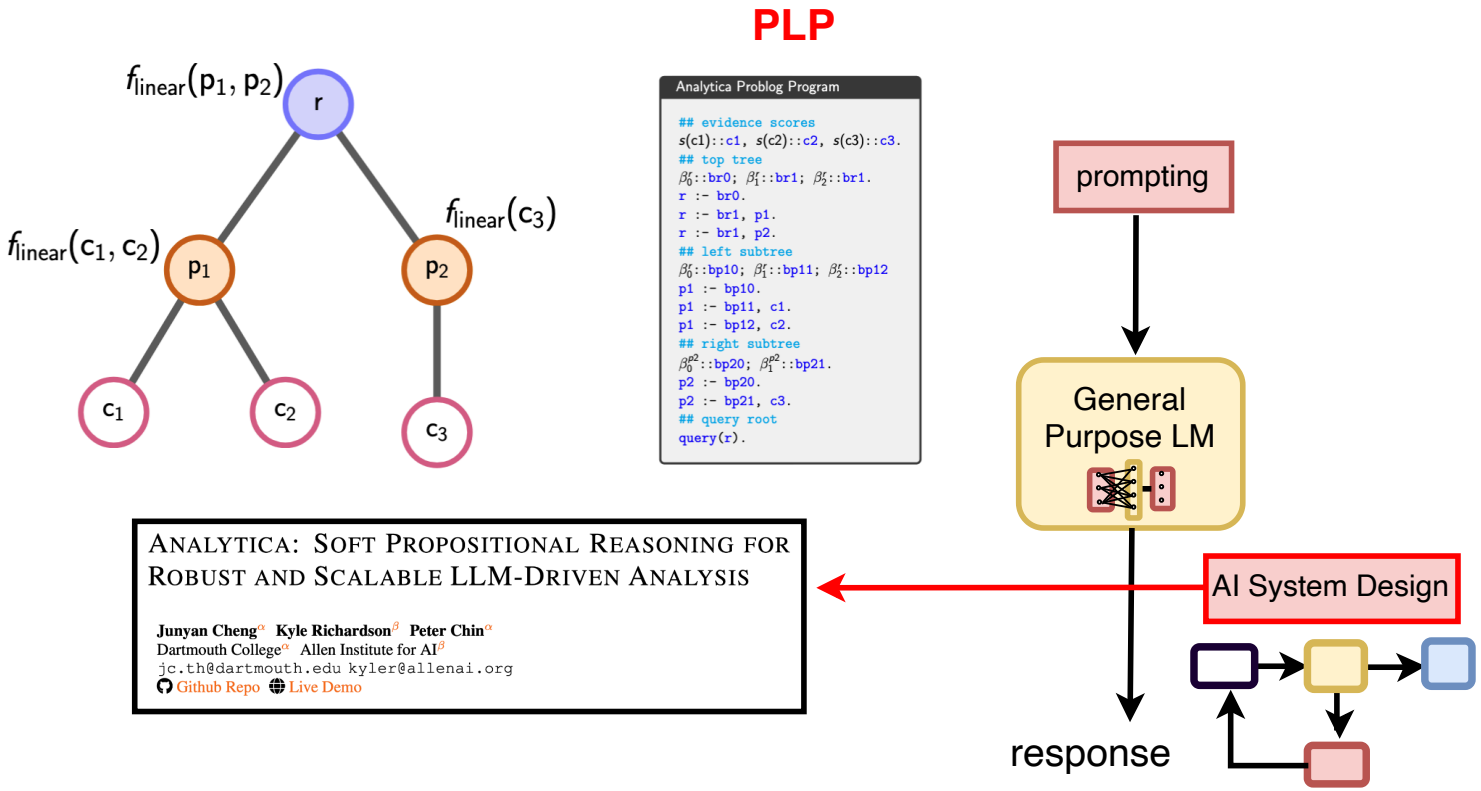
Languages provide a rich, declarative semantics

Programming languages for LLM research



- Imperative-style programming, add structure to LLM workflows, mix with conventional programming (Beurer-Kellner et al.2023; Mandel et al.2026; Wu et al.2026).

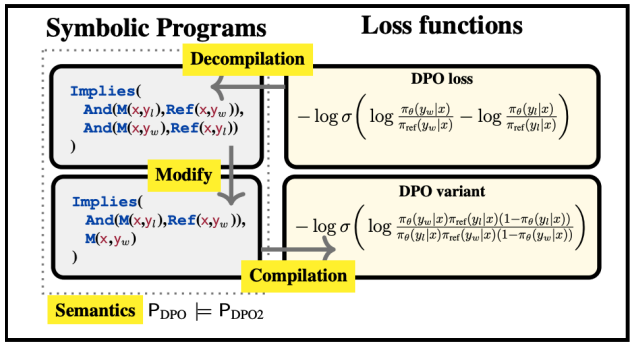
Programming languages for LLM research



- Modeling message passing, score aggregation in forecasting agents, LLM soft scoring; problog (De Raedt et al.2007; Dries et al.2015).

Language Model Programming

Language Model Programming




```python
@CotProgParser()
def pp():
  P1:  bool =~ Flip("step1", 0.86, [0.7])

  P2:  bool =~ Flip("step2", 0.96, [0.9])
  ## observe(P2) ## Optional condition, set P2=True

  if P1 and P2:
    P3:  bool =~ Flip("no", 0.53, [0.8])
    yield P3 ## returns "no" if P3 true, exception otherwise, syntactic sugar
  else:
    raise ValueError("Cannot establish P1 or P2")

## Program inference
pp.inference(query={"no"}, compute_grad=True)  ## 0.4376
## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
pp.inference(query={"ValueError"})  ## 0.5624
pp.inference(query={"no"}, use_conf=True)  ## 0.504
```

$$
\begin{aligned}
P_\ell(i) &:= \blacktriangleright_j\ [j < i, 1]\ \ Q_\ell(j) : 0 \\
S_r(i) &:= \blacktriangleleft_i\ [j > i, 1]\ \ Q_r(j) : 0. \\
I(i) &:= (Q_\ell(i) \wedge S_r(i)) \vee (Q_r(i) \wedge P_\ell(i)). \\
B_\ell(i) &:= \blacktriangleright_j\ [j < i, \neg I(j)]\ \ Q_\ell(j) : 0 \\
A_r(i) &:= \blacktriangleleft_j\ [j > i, \neg I(j)]\ \ Q_r(j) : 0 \\
C(i) &:= I(i) \vee (Q_\ell(i) \wedge A_r(i)) \vee (Q_r(i) \wedge B_\ell(i)). \\
Y(i) &:= \blacktriangleright_j\ [1, \neg C(j)]\ \ 0 : 1.
\end{aligned}
$$

```python
@lmql.query
def meaning_of_life():
    '''lmql
    # top-level strings are prompts
    "Q: What is the answer to life, the \
     universe and everything?"

    # generation via (constrained) variables
    "A: [ANSWER]" where \
        len(ANSWER) < 120 and STOPS_AT(ANSWER, ".")

    # results are directly accessible
    print("LLM returned", ANSWER)

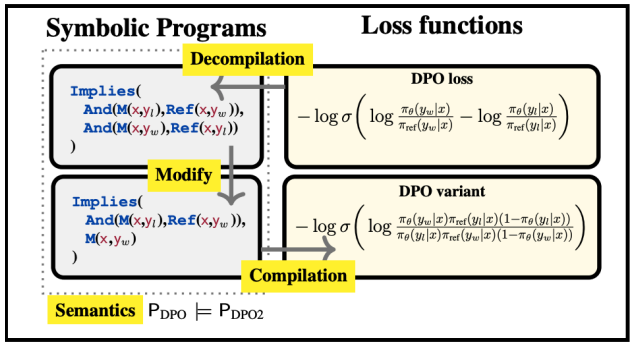
    # use typed variables for guaranteed
    # output format
    "The answer is [NUM: int]"

    # query programs are just functions
    return NUM
    '''

# so from Python, you can just do this
meaning_of_life() # 42
```

- **LM programming**: The study and design of languages for specifying, composing and reasoning about LLM computations.

Discrete Modeling Techniques




```
@CotProgParser()
def pp():
  P1:  bool =~ Flip("step1", 0.86, [0.7])

  P2:  bool =~ Flip("step2", 0.96, [0.9])
  ## observe(P2) ## Optional condition, set P2=True

  if P1 and P2:
    P3:  bool =~ Flip("no", 0.53, [0.8])
    yield P3 ## returns "no" if P3 true, exception otherwise, syntactic sugar
  else:
    raise ValueError("Cannot establish P1 or P2")

## Program inference
pp.inference(query={"no"}, compute_grad=True)  ## 0.4376
## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
pp.inference(query={"ValueError"})  ## 0.5624
pp.inference(query={"no"}, use_conf=True)  ## 0.504
```

$$
\begin{aligned}
P_\ell(i) &:= \blacktriangleright_j\ [j < i, 1]\ \ Q_\ell(j) : 0 \\
S_r(i) &:= \blacktriangleleft_i\ [j > i, 1]\ \ Q_r(j) : 0. \\
I(i) &:= (Q_\ell(i) \wedge S_r(i)) \vee (Q_r(i) \wedge P_\ell(i)). \\
B_\ell(i) &:= \blacktriangleright_j\ [j < i, \neg I(j)]\ \ Q_\ell(j) : 0 \\
A_r(i) &:= \blacktriangleleft_j\ [j > i, \neg I(j)]\ \ Q_r(j) : 0 \\
C(i) &:= I(i) \vee (Q_\ell(i) \wedge A_r(i)) \vee (Q_r(i) \wedge B_\ell(i)). \\
Y(i) &:= \blacktriangleright_j\ [1, \neg C(j)]\ \ 0 : 1.
\end{aligned}
$$

```
@lmql.query
def meaning_of_life():
    '''lmql
    # top-level strings are prompts
    "Q: What is the answer to life, the \
     universe and everything?"

    # generation via (constrained) variables
    "A: [ANSWER]" where \
        len(ANSWER) < 120 and STOPS_AT(ANSWER, ".")

    # results are directly accessible
    print("LLM returned", ANSWER)

    # use typed variables for guaranteed
    # output format
    "The answer is [NUM: int]"

    # query programs are just functions
    return NUM
    '''

# so from Python, you can just do this
meaning_of_life() # 42
```

- **What this course is about**: The formal techniques needed for this kind of modeling, using and implementing symbolic programming languages.

Discrete Methods for Modern AI: Foundations

Kyle Richardson
University of Tübingen
Draft: work in progress (November 1, 2026)

1 Introduction

These notes are a companion to a course on *Discrete Methods for Modern AI* [1], which examines applications of discrete modeling techniques – specifically, techniques drawn from logic, automated reasoning and classical probabilistic inference – to problems in modern language modeling and generative AI. Throughout the course, we study how these techniques can support deeper semantic analysis of current systems, provide a means of incorporating structured knowledge and constraints into learning and inference, and offer theoretical tools for characterizing the capabilities and limitations of these systems. In these notes, we focus on the logical foundations shared by each of these directions and introduce the basic propositional calculus. In particular, our review aims to be *algorithmic* in nature by focusing on the technical tools needed for *building* the kinds of automated reasoning systems used throughout this area of the machine learning literature (see review in Marra et al. (2021); Feldstein et al. (2024)).

The structure of this note is as follows: starting from a review of basic concepts in propositional logic (§ 2), we then state some fundamental algorithmic questions involved with *using* propositional logic (§ 3)[2], including the *Boolean Satisfiability* (SAT) problem and some variants of this problem, which are at the heart of many problems in automated reasoning and probabilistic inference. To help build intuitions for how build automated reasoning systems, we derive a classical search-based algorithm for SAT, called the DPLL Algorithm (Davis et al., 1962), and relate its generalized algebraic form to solutions to these different variants of SAT (e.g., *model counting* and its weighted variants). Building on these intuitions and algorithmic details, we next (§ 5) give an overview of representations of logic based on tractable classes of Boolean and arithmetic circuits that are at the core of many systems that combine logic and learning.

While working in this literature often requires considerable technical sophistication and knowledge of logic, researchers often rely on a broad set of tools from the automated reasoning community (e.g., SAT solving and other theorem proving technology). We therefore provide code examples using such tools and encourage readers to work through these examples. We also use examples from a library built by the author, `symprog` [3], which aims to make it easy to quickly try out the ideas covered in these notes. For advanced readers who want to explore specific topics in more detail, we also provide pointers to related work and various *technical asides* that address topics not directly covered in our main survey.

Last lecture: the beginning of the course notes

Recap of last lecture: propositional logic, SAT, counting

Syntax

- ✓ formulas are trees
- ✓ NNF, CNF, DNF
- ✓ CNF as a set of clauses

Recap of last lecture: propositional logic, SAT, counting

Syntax

- ✓ formulas are trees
- ✓ NNF, CNF, DNF
- ✓ CNF as a set of clauses

Semantics

- ✓ worlds: each atom **1** or **0**
- ✓ truth tables, bottom-up
- ✓ equivalence and the laws

Recap of last lecture: propositional logic, SAT, counting

Syntax

- ✓ formulas are trees
- ✓ NNF, CNF, DNF
- ✓ CNF as a set of clauses

Semantics

- ✓ worlds: each atom **1** or **0**
- ✓ truth tables, bottom-up
- ✓ equivalence and the laws

Inference

- ✓ satisfiable, valid, $\Gamma \models \alpha$
- ✓ $\Gamma \models \alpha$ *iff* $\Gamma \cup \{\neg\alpha\}$ is unsat
- ✓ truth tables need 2^n worlds

Recap of last lecture: propositional logic, SAT, counting

Syntax

- ✓ formulas are trees
- ✓ NNF, CNF, DNF
- ✓ CNF as a set of clauses

Inference

- ✓ satisfiable, valid, $\Gamma \models \alpha$
- ✓ $\Gamma \models \alpha$ *iff* $\Gamma \cup \{\neg\alpha\}$ is unsat
- ✓ truth tables need 2^n worlds

Semantics

- ✓ worlds: each atom **1** or **0**
- ✓ truth tables, bottom-up
- ✓ equivalence and the laws

Computation

- ✓ conditioning $\Delta|l$
- ✓ DPLL: splitting, unit rule
- ✓ #SAT, WMC: $\bigoplus$ of $\bigotimes$

Recap of last lecture: propositional logic, SAT, counting

Syntax

- ✓ formulas are trees
- ✓ NNF, CNF, DNF
- ✓ CNF as a set of clauses

Inference

- ✓ satisfiable, valid, $\Gamma \models \alpha$
- ✓ $\Gamma \models \alpha$ *iff* $\Gamma \cup \{\neg\alpha\}$ is unsat
- ✓ truth tables need 2^n worlds

Semantics

- ✓ worlds: each atom **1** or **0**
- ✓ truth tables, bottom-up
- ✓ equivalence and the laws

Computation

- ✓ conditioning $\Delta|l$
- ✓ DPLL: splitting, unit rule
- ✓ #SAT, WMC: $\bigoplus$ of $\bigotimes$

Today: language modeling background, symbolic transformer computation.

Quick check on lecture 1: true or false?

1. $P_1 \rightarrow P_2 \equiv \neg P_2 \rightarrow \neg P_1$
2. $P_1 \vee \neg P_1$ is satisfiable but not valid
3. $\{P_1 \rightarrow P_2,\ P_1\} \models P_2$
4. a formula over 4 atoms has 8 worlds
5. DPLL has to split on every atom
6. $P_1 \vee P_2$ has 3 models
7. $\mathrm{WMC}(P_1 \vee P_2) = 0.75$ (all $w = 0.5$)

Quick check on lecture 1: true or false?

1. $P_1 \rightarrow P_2 \equiv \neg P_2 \rightarrow \neg P_1$ **true** contrapositive
2. $P_1 \vee \neg P_1$ is satisfiable but not valid
3. $\{P_1 \rightarrow P_2,\ P_1\} \models P_2$
4. a formula over 4 atoms has 8 worlds
5. DPLL has to split on every atom
6. $P_1 \vee P_2$ has 3 models
7. $\mathrm{WMC}(P_1 \vee P_2) = 0.75$ (all $w = 0.5$)

Quick check on lecture 1: true or false?

1. $P_1 \rightarrow P_2 \equiv \neg P_2 \rightarrow \neg P_1$ — **true** — contrapositive
2. $P_1 \vee \neg P_1$ is satisfiable but not valid — **false** — true in every world: valid
3. $\{P_1 \rightarrow P_2,\ P_1\} \models P_2$
4. a formula over 4 atoms has 8 worlds
5. DPLL has to split on every atom
6. $P_1 \vee P_2$ has 3 models
7. $\mathrm{WMC}(P_1 \vee P_2) = 0.75$ (all $w = 0.5$)

Quick check on lecture 1: true or false?

1. $P_1 \rightarrow P_2 \equiv \neg P_2 \rightarrow \neg P_1$ — **true** — contrapositive
2. $P_1 \vee \neg P_1$ is satisfiable but not valid — **false** — true in every world: valid
3. $\{P_1 \rightarrow P_2,\ P_1\} \models P_2$ — **true** — with $\neg P_2$: unsat
4. a formula over 4 atoms has 8 worlds
5. DPLL has to split on every atom
6. $P_1 \vee P_2$ has 3 models
7. $\mathrm{WMC}(P_1 \vee P_2) = 0.75$ (all $w = 0.5$)

Quick check on lecture 1: true or false?

1.	$P_1 \rightarrow P_2 \equiv \neg P_2 \rightarrow \neg P_1$	**true**	contrapositive
2.	$P_1 \vee \neg P_1$ is satisfiable but not valid	**false**	true in every world: valid
3.	$\{P_1 \rightarrow P_2,\ P_1\} \models P_2$	**true**	with $\neg P_2$: unsat
4.	a formula over 4 atoms has 8 worlds	**false**	$2^4 = 16$ worlds
5.	DPLL has to split on every atom		
6.	$P_1 \vee P_2$ has 3 models		
7.	$\mathrm{WMC}(P_1 \vee P_2) = 0.75$		(all $w = 0.5$)

Quick check on lecture 1: true or false?

1.	$P_1 \rightarrow P_2 \equiv \neg P_2 \rightarrow \neg P_1$	**true**	contrapositive
2.	$P_1 \vee \neg P_1$ is satisfiable but not valid	**false**	true in every world: valid
3.	$\{P_1 \rightarrow P_2,\ P_1\} \models P_2$	**true**	with $\neg P_2$: unsat
4.	a formula over 4 atoms has 8 worlds	**false**	$2^4 = 16$ worlds
5.	DPLL has to split on every atom	**false**	unit clauses need no split
6.	$P_1 \vee P_2$ has 3 models		
7.	$\mathrm{WMC}(P_1 \vee P_2) = 0.75$		(all $w = 0.5$)

Quick check on lecture 1: true or false?

1.	$P_1 \rightarrow P_2 \equiv \neg P_2 \rightarrow \neg P_1$	**true**	contrapositive
2.	$P_1 \vee \neg P_1$ is satisfiable but not valid	**false**	true in every world: valid
3.	$\{P_1 \rightarrow P_2,\ P_1\} \models P_2$	**true**	with $\neg P_2$: unsat
4.	a formula over 4 atoms has 8 worlds	**false**	$2^4 = 16$ worlds
5.	DPLL has to split on every atom	**false**	unit clauses need no split
6.	$P_1 \vee P_2$ has 3 models	**true**	all worlds but $0, 0$
7.	$\mathrm{WMC}(P_1 \vee P_2) = 0.75$		(all $w = 0.5$)

Quick check on lecture 1: true or false?

1.	$P_1 \rightarrow P_2 \equiv \neg P_2 \rightarrow \neg P_1$	**true**	contrapositive
2.	$P_1 \vee \neg P_1$ is satisfiable but not valid	**false**	true in every world: valid
3.	$\{P_1 \rightarrow P_2,\ P_1\} \models P_2$	**true**	with $\neg P_2$: unsat
4.	a formula over 4 atoms has 8 worlds	**false**	$2^4 = 16$ worlds
5.	DPLL has to split on every atom	**false**	unit clauses need no split
6.	$P_1 \vee P_2$ has 3 models	**true**	all worlds but $0, 0$
7.	$\mathrm{WMC}(P_1 \vee P_2) = 0.75$ (all $w = 0.5$)	**true**	3 models, 0.25 each

Quick check on lecture 1: true or false? (cont.)

8. $(P_1 \wedge P_2) \rightarrow P_1$ is a tautology
9. $P_1 \rightarrow (P_1 \wedge P_2)$ is a tautology
10. α is valid *iff* $\neg\alpha$ is unsatisfiable
11. $(P_1 \vee P_2) \wedge \neg P_1 \wedge \neg P_2$ is unsat.
12. $\#(P_1 \wedge \neg P_1) = 1$
13. over atoms P_1, P_2, $\#(P_1) = 1$
14. $\#(F) + \#(\neg F) = 2^n$ (n atoms)

Quick check on lecture 1: true or false? (cont.)

8. $(P_1 \wedge P_2) \rightarrow P_1$ is a tautology — **true** — no world makes it false
9. $P_1 \rightarrow (P_1 \wedge P_2)$ is a tautology
10. α is valid *iff* $\neg\alpha$ is unsatisfiable
11. $(P_1 \vee P_2) \wedge \neg P_1 \wedge \neg P_2$ is unsat.
12. $\#(P_1 \wedge \neg P_1) = 1$
13. over atoms P_1, P_2, $\#(P_1) = 1$
14. $\#(F) + \#(\neg F) = 2^n$ (n atoms)

Quick check on lecture 1: true or false? (cont.)

8. $(P_1 \wedge P_2) \rightarrow P_1$ is a tautology — **true** — no world makes it false
9. $P_1 \rightarrow (P_1 \wedge P_2)$ is a tautology — **false** — fails at $P_1 = 1, P_2 = 0$
10. α is valid *iff* $\neg\alpha$ is unsatisfiable
11. $(P_1 \vee P_2) \wedge \neg P_1 \wedge \neg P_2$ is unsat.
12. $\#(P_1 \wedge \neg P_1) = 1$
13. over atoms P_1, P_2, $\#(P_1) = 1$
14. $\#(F) + \#(\neg F) = 2^n$ (n atoms)

Quick check on lecture 1: true or false? (cont.)

8. $(P_1 \wedge P_2) \rightarrow P_1$ is a tautology — **true** — no world makes it false
9. $P_1 \rightarrow (P_1 \wedge P_2)$ is a tautology — **false** — fails at $P_1 = 1, P_2 = 0$
10. α is valid *iff* $\neg\alpha$ is unsatisfiable — **true** — no world makes α false
11. $(P_1 \vee P_2) \wedge \neg P_1 \wedge \neg P_2$ is unsat.
12. $\#(P_1 \wedge \neg P_1) = 1$
13. over atoms P_1, P_2, $\#(P_1) = 1$
14. $\#(F) + \#(\neg F) = 2^n$ (n atoms)

Quick check on lecture 1: true or false? (cont.)

8.	$(P_1 \wedge P_2) \rightarrow P_1$ is a tautology	**true**	no world makes it false
9.	$P_1 \rightarrow (P_1 \wedge P_2)$ is a tautology	**false**	fails at $P_1 = 1, P_2 = 0$
10.	α is valid *iff* $\neg\alpha$ is unsatisfiable	**true**	no world makes α false
11.	$(P_1 \vee P_2) \wedge \neg P_1 \wedge \neg P_2$ is unsat.	**true**	$F \wedge \neg F$
12.	$\#(P_1 \wedge \neg P_1) = 1$		
13.	over atoms P_1, P_2, $\#(P_1) = 1$		
14.	$\#(F) + \#(\neg F) = 2^n$ (n atoms)		

Quick check on lecture 1: true or false? (cont.)

8.	$(P_1 \wedge P_2) \rightarrow P_1$ is a tautology	**true**	no world makes it false
9.	$P_1 \rightarrow (P_1 \wedge P_2)$ is a tautology	**false**	fails at $P_1 = 1, P_2 = 0$
10.	α is valid *iff* $\neg\alpha$ is unsatisfiable	**true**	no world makes α false
11.	$(P_1 \vee P_2) \wedge \neg P_1 \wedge \neg P_2$ is unsat.	**true**	$F \wedge \neg F$
12.	$\#(P_1 \wedge \neg P_1) = 1$	**false**	unsat: 0 models
13.	over atoms P_1, P_2, $\#(P_1) = 1$		
14.	$\#(F) + \#(\neg F) = 2^n$ (n atoms)		

Quick check on lecture 1: true or false? (cont.)

8.	$(P_1 \wedge P_2) \rightarrow P_1$ is a tautology	**true**	no world makes it false
9.	$P_1 \rightarrow (P_1 \wedge P_2)$ is a tautology	**false**	fails at $P_1 = 1, P_2 = 0$
10.	α is valid *iff* $\neg\alpha$ is unsatisfiable	**true**	no world makes α false
11.	$(P_1 \vee P_2) \wedge \neg P_1 \wedge \neg P_2$ is unsat.	**true**	$F \wedge \neg F$
12.	$\#(P_1 \wedge \neg P_1) = 1$	**false**	unsat: 0 models
13.	over atoms P_1, P_2, $\#(P_1) = 1$	**false**	2: P_2 is free
14.	$\#(F) + \#(\neg F) = 2^n$ (n atoms)		

Quick check on lecture 1: true or false? (cont.)

8.	$(P_1 \wedge P_2) \rightarrow P_1$ is a tautology	**true**	no world makes it false
9.	$P_1 \rightarrow (P_1 \wedge P_2)$ is a tautology	**false**	fails at $P_1 = 1, P_2 = 0$
10.	α is valid *iff* $\neg\alpha$ is unsatisfiable	**true**	no world makes α false
11.	$(P_1 \vee P_2) \wedge \neg P_1 \wedge \neg P_2$ is unsat.	**true**	$F \wedge \neg F$
12.	$\#(P_1 \wedge \neg P_1) = 1$	**false**	unsat: 0 models
13.	over atoms P_1, P_2, $\#(P_1) = 1$	**false**	2: P_2 is free
14.	$\#(F) + \#(\neg F) = 2^n$ (n atoms)	**true**	each world: one of them

More questions in the course notes

Quick check on lecture 1: true or false? (cont.)

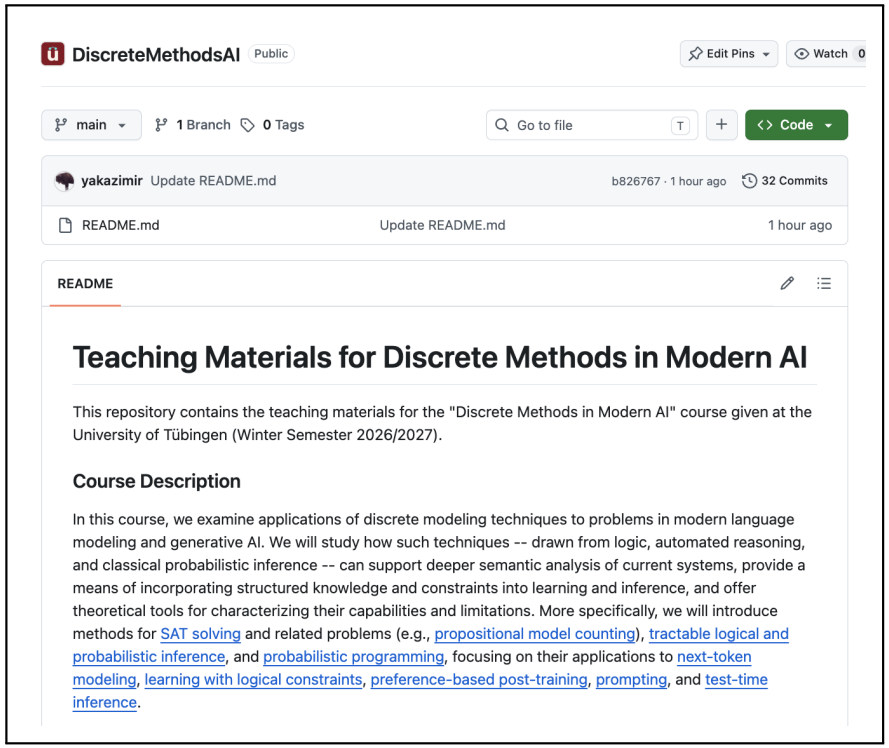
8. $(P_1 \wedge P_2) \rightarrow P_1$ is a tautology — **true** — no world makes it false
9. $P_1 \rightarrow (P_1 \wedge P_2)$ is a tautology — **false** — fails at $P_1 = 1, P_2 = 0$
10. α is valid *iff* $\neg\alpha$ is unsatisfiable — **true** — no world makes α false
11. $(P_1 \vee P_2) \wedge \neg P_1 \wedge \neg P_2$ is $F \wedge \neg F$
12. $\#(P_1 \wedge \neg P_1) = 1$ — **false** — unsat: 0 models
13. over atoms P_1, P_2, $\#(P_1) = 1$ — **false** — 2: P_2 is free
14. $\#(F) + \#(\neg F) = 2^n$ (n atoms) — **true** — each world: one of them

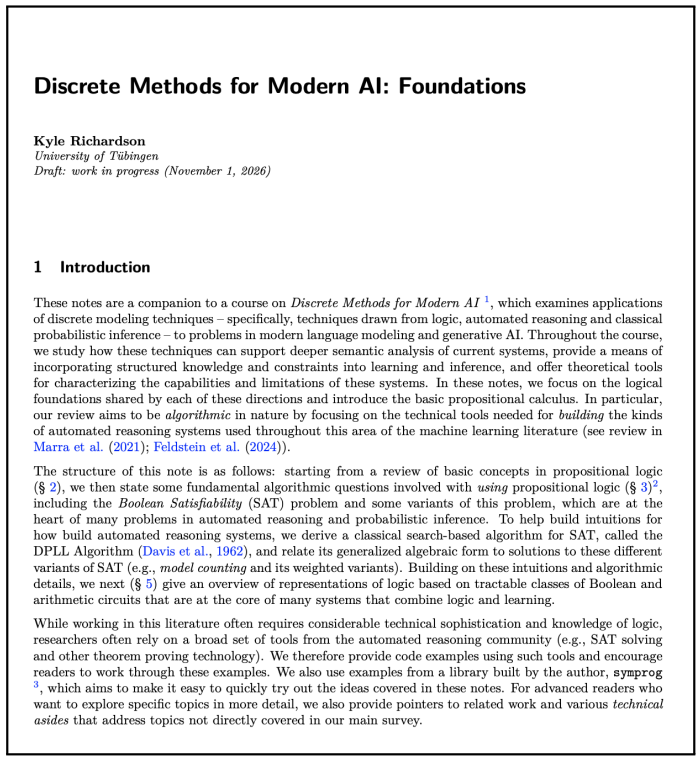
Questions?

Course structure

Course materials and website

https://github.com/TuLLI-AI/DiscreteMethodsAI

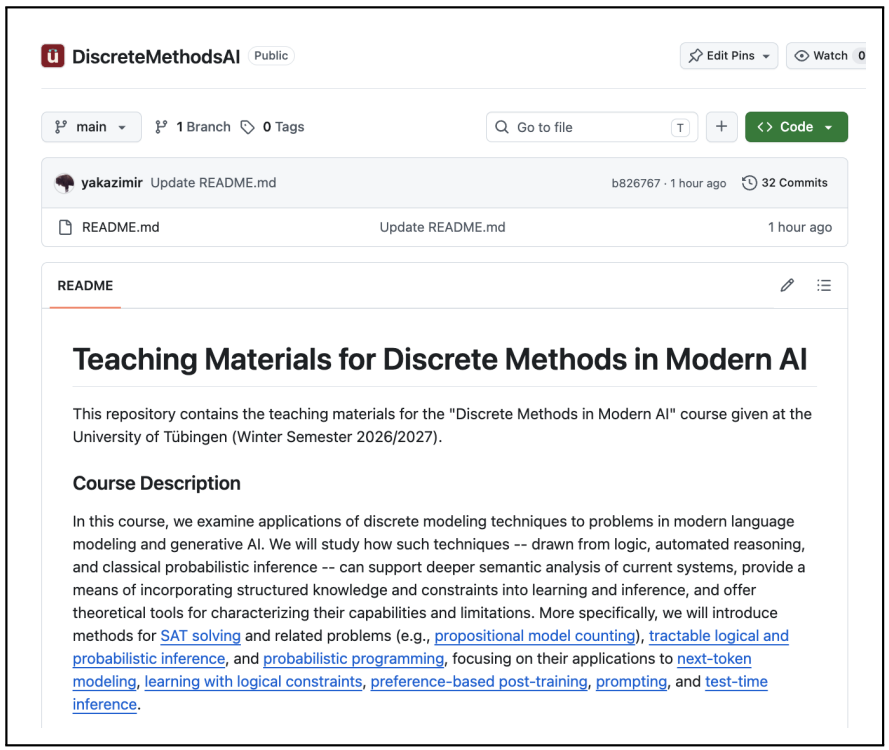


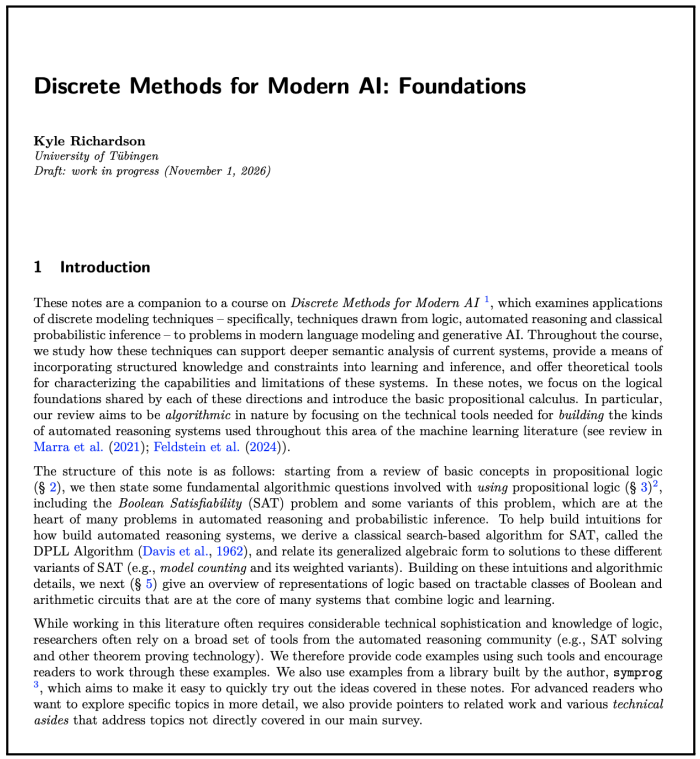


- **Expectations**: Attend weekly lectures, follow the readings and Python notebooks (on GitHub), quickly learn new topics.

Course evaluation

https://github.com/TuLLI-AI/DiscreteMethodsAI





- **Evaluation**: In-class comprehension exam (30%, Dec. 7), coding project + short presentation (60%), attendance (10%), **6 credit** course.

Did everyone manage to install `symprog`?

```
$ pip install symprog          github.com/TuLLI-AI/symprog
```

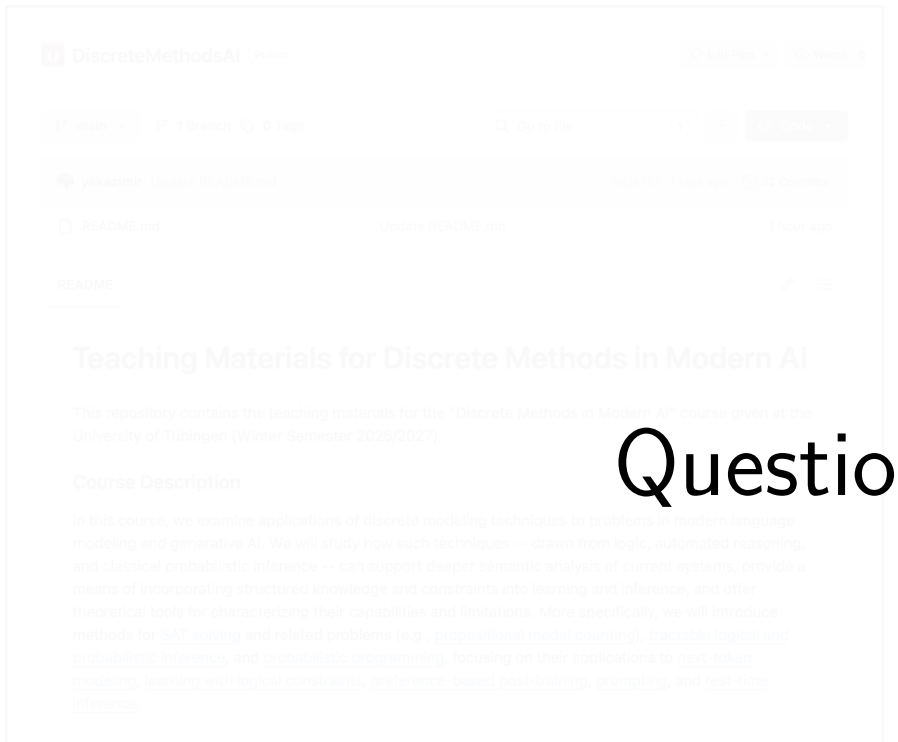
```
from symprog import Solver

solver = Solver()
solver.add(["Or(Not(p1), p2)", "p3"])          ## F
solver.add_weights({"p1": 0.8, "p2": 0.75, "p3": 0.4})

solver.query(mode="SAT").inference        ## => yes
solver.query(mode="Count").inference      ## => 3.0
solver.query(mode="Prob").inference       ## => 0.32  WMC
solver.query(mode="ViterbiAssignment").variables   ## MPE
## => {'p1': True, 'p2': True, 'p3': True}
```

Course evaluation

https://github.com/TuLLI-AI/DiscreteMethodsAI





Questions?

- **Evaluation**: In-class comprehension exam (30%, Dec. 7), coding project + short presentation (60%), attendance (10%), **6 credit** course.

Language modeling basics

What are language models?

- Machine learning models that **assign probabilities to sequences**.

What are language models?

- Machine learning models that **assign probabilities to sequences**.

  e.g., t =Germany is between Czechia and France

What are language models?

- Machine learning models that **assign probabilities to sequences**.

  e.g., t =Germany is between Czechia and France

$$\underbrace{p(w_1, w_2, \ldots, w_m)}_{\text{joint distribution}} = \prod_{j=1}^{m} p(\underbrace{w_j}_{\text{next word}} \mid \underbrace{w_1, \ldots, w_{j-1}}_{\text{previous words}})$$

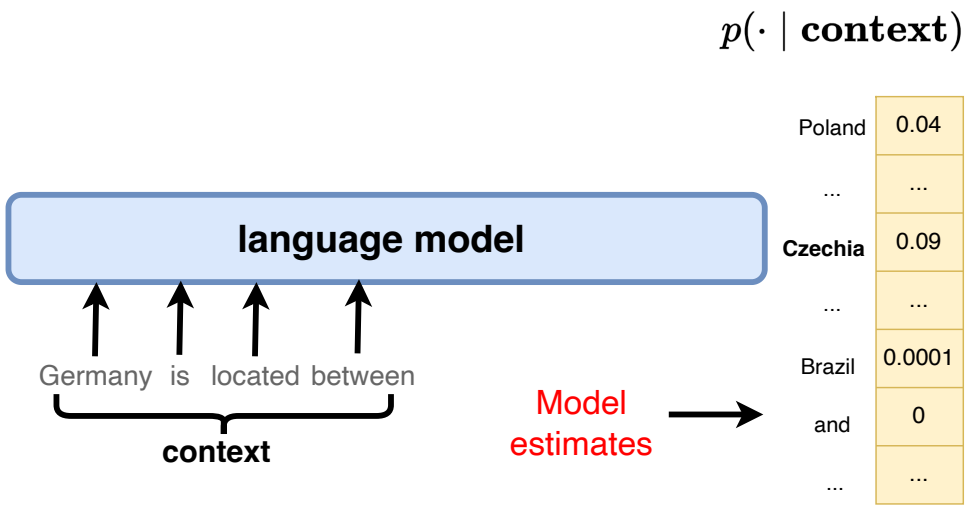
What are language models?

- Machine learning models that **assign probabilities to sequences**.

e.g., t =Germany is between Czechia and France

$$\underbrace{p(w_1, w_2, \ldots, w_m)}_{\text{joint distribution}} = \prod_{j=1}^{m} p(\underbrace{w_j}_{\text{next word}} \mid \underbrace{w_1, \ldots, w_{j-1}}_{\text{previous words}})$$

Language models **generate**



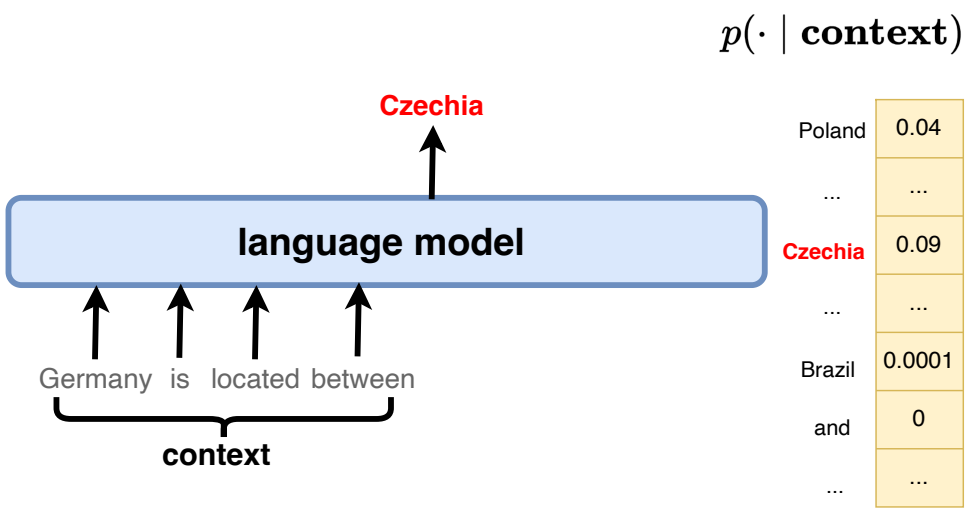
What are language models?

- Machine learning models that **assign probabilities to sequences**.

e.g., t =Germany is between Czechia and France

$$\underbrace{p(w_1, w_2, \ldots, w_m)}_{\text{joint distribution}} = \prod_{j=1}^{m} p(\underbrace{w_j}_{\text{next word}} \mid \underbrace{w_1, \ldots, w_{j-1}}_{\text{previous words}})$$

Language models **generate**



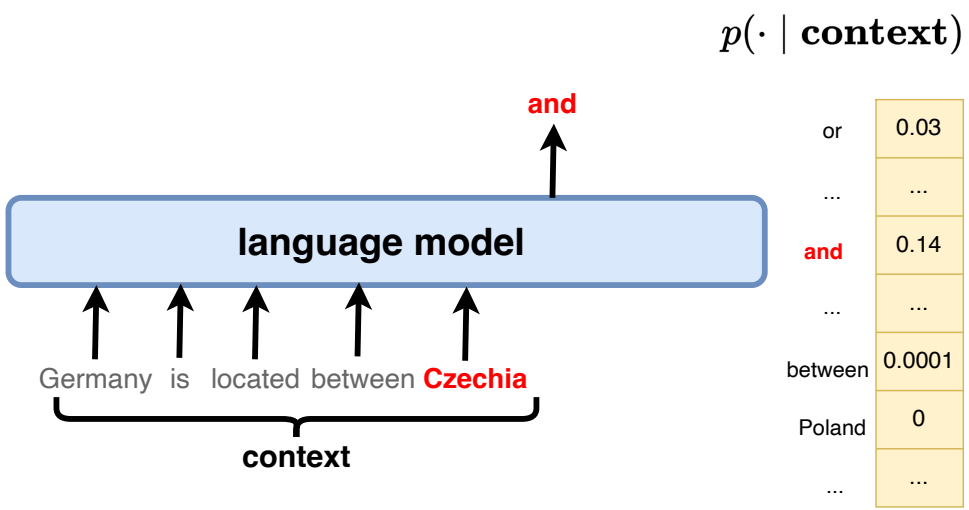
What are language models?

- Machine learning models that **assign probabilities to sequences**.

e.g., t =Germany is between Czechia and France

$$\underbrace{p(w_1, w_2, \ldots, w_m)}_{\text{joint distribution}} = \prod_{j=1}^{m} p(\underbrace{w_j}_{\text{next word}} \mid \underbrace{w_1, \ldots, w_{j-1}}_{\text{previous words}})$$

Language models **generate**



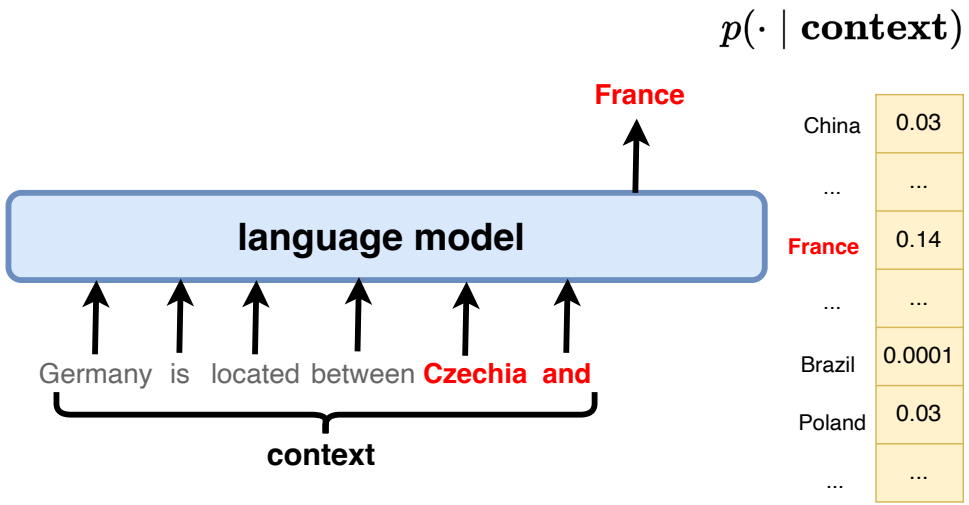
What are language models?

- Machine learning models that **assign probabilities to sequences**.

e.g., t =Germany is between Czechia and France

$$\underbrace{p(w_1, w_2, \ldots, w_m)}_{\text{joint distribution}} = \prod_{j=1}^{m} p(\underbrace{w_j}_{\text{next word}} \mid \underbrace{w_1, \ldots, w_{j-1}}_{\text{previous words}})$$

Language models **generate**



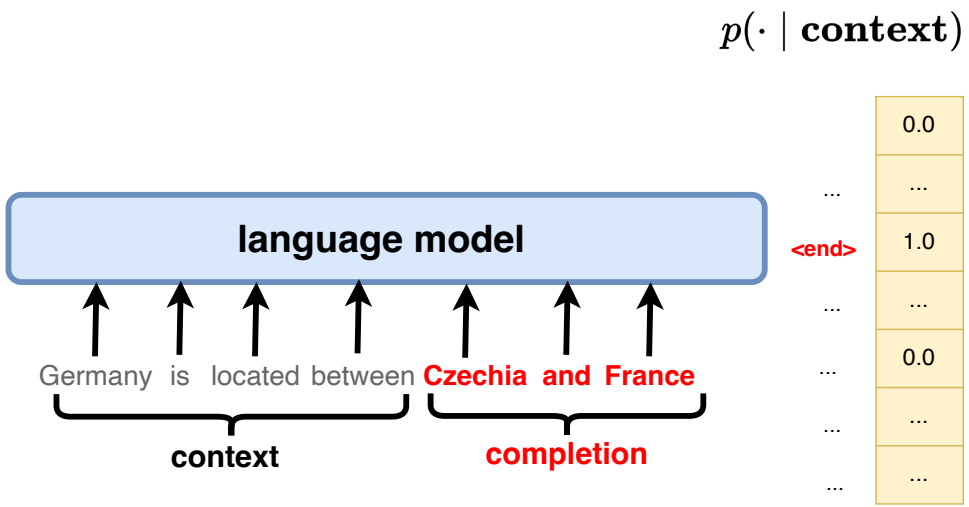
What are language models?

- Machine learning models that **assign probabilities to sequences**.

e.g., t =Germany is between Czechia and France

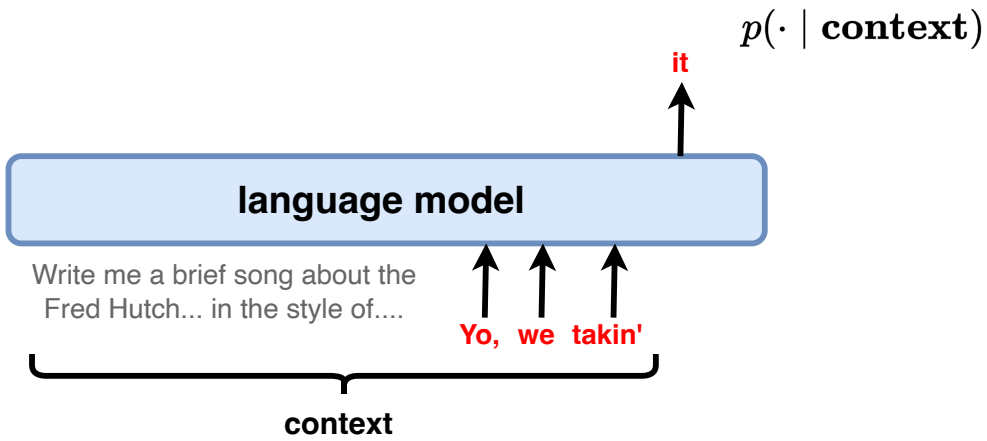
$$\underbrace{p(w_1, w_2, \ldots, w_m)}_{\text{joint distribution}} = \prod_{j=1}^{m} p(\underbrace{w_j}_{\text{next word}} \mid \underbrace{w_1, \ldots, w_{j-1}}_{\text{previous words}})$$

Language models **generate**



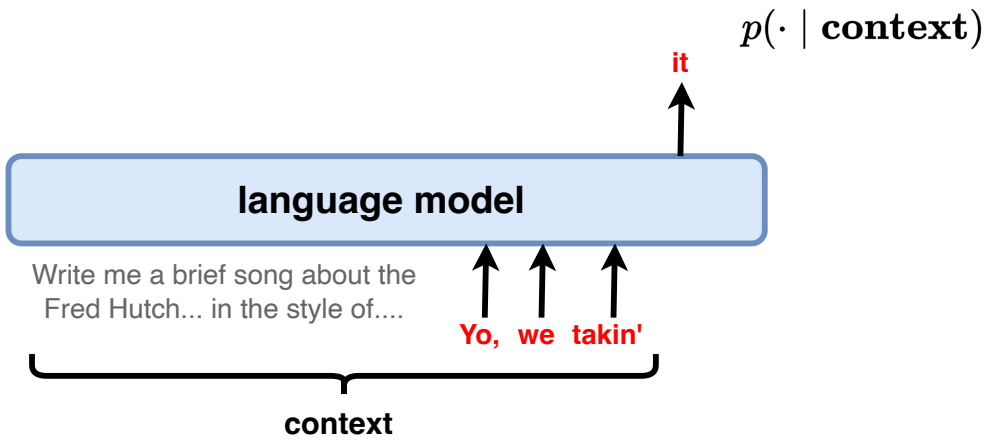
What are language models?

Sequence modeling is not a new problem.



What are language models?

Sequence modeling is not a new problem.



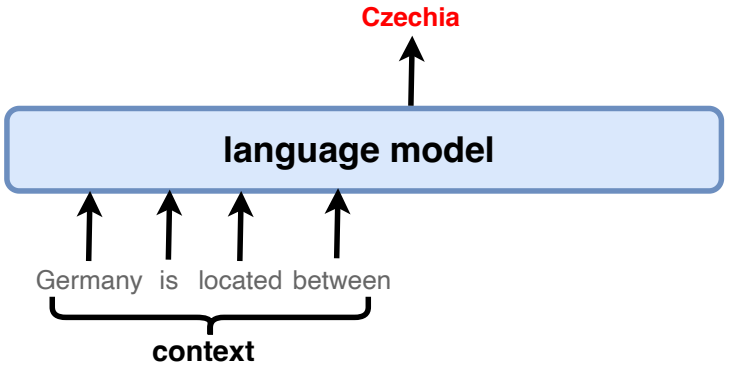
Text documents

```
TaskFirst pose the question:Here are four tr
iangles. What do all of these triangles have
 in common? What makes them different from t
he figures that are not triangles? What is t
rue for some but not all of these triangles?
###IMAGE0###If students come up with a state
ment that is true about all of the triangles
 that they see but not true of all triangles
 in general, the teacher should ask students
 if they can imagine a triangle without that
 attribute. For example, if a student says,
"All of the triangles are white on the insid
e," the teacher can ask, "Would it be possib
le for a triangle to have a different color
on the inside?" When the class comes up with
 an attribute that is truly shared by all tr
iangles, then the class can complete the sen
tence frame:  All triangles __________, but
 only some triangles _________________.  Whe
n the students have written (or composed) th
eir sentences based on the sentence frames,
the class can write the definition of a tria
ngle together:A triangle is a closed shape w
```

Biological sequences

```
ATG GAT TTT GGT GTT GTT TGT CTT CTG CCT GTG TGG GTT GAT GGT TTG GAG GGT
GCG GAG TAG CCG GCA GCG GAT GAG CGG GCG CCG TCC GTC GCA GCG GGA GGA
GGA CCC ATG
GAA GCG GAA GGC GCT GCC CGC GGC CGG GGT CCA CGC CGT GAG GCC ATC GAG
GGA GGC GGT GGC GAC CCA CCA ACA CGC GAT GCC AGT TTG ACC GGC AGC CGC GGG
CGA GGA GGG
CTT GCC AAA GGC CGG GGC TGC CGG GCG GGC CCG GCT CTG GTG GAG GAG GGC
CGA GAG GAA CTT GGC TTC CGA CGC GGC CAC CTG GCG GGC TGC GGA GCG GCC CTG
CGT GGA CGC
ACT CCG CGT GGC CGC GAG GCG GAG ATG TGC GCG GCG GGC CGT GAG GAC GGC ATC
GCC AGC GAA CTG GAG GTG CGC AGA TGG TGC CGG GAT GAG CGG TGC GCG GGC CTG
GCC CCT
```

The problem with traditional solutions

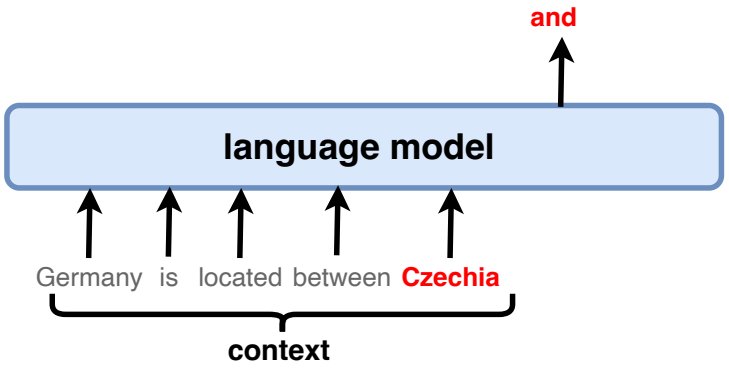


$p(\cdot \mid \textbf{context})$

Discrete representations

Poland	0.04	Germany is located between
...	...	Germany is located between
Czechia	0.09	Germany is located between
...	...	Germany is located between
Brazil	0.0001	Germany is located between
and	0	Germany is located between
...	...	Germany is located between

The problem with traditional solutions

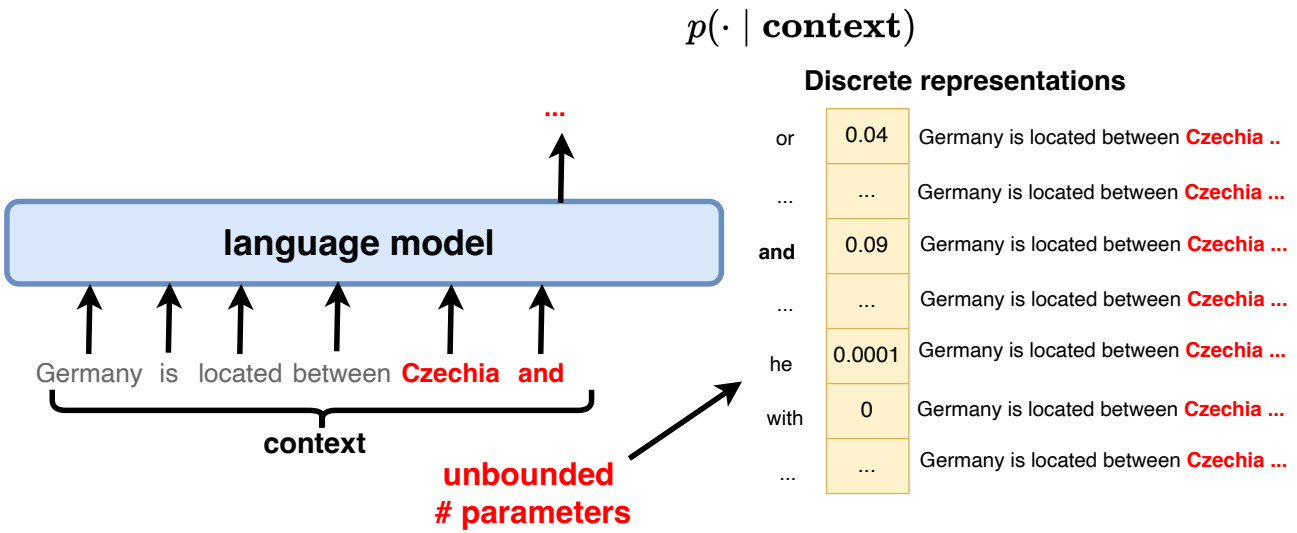


$p(\cdot \mid \textbf{context})$

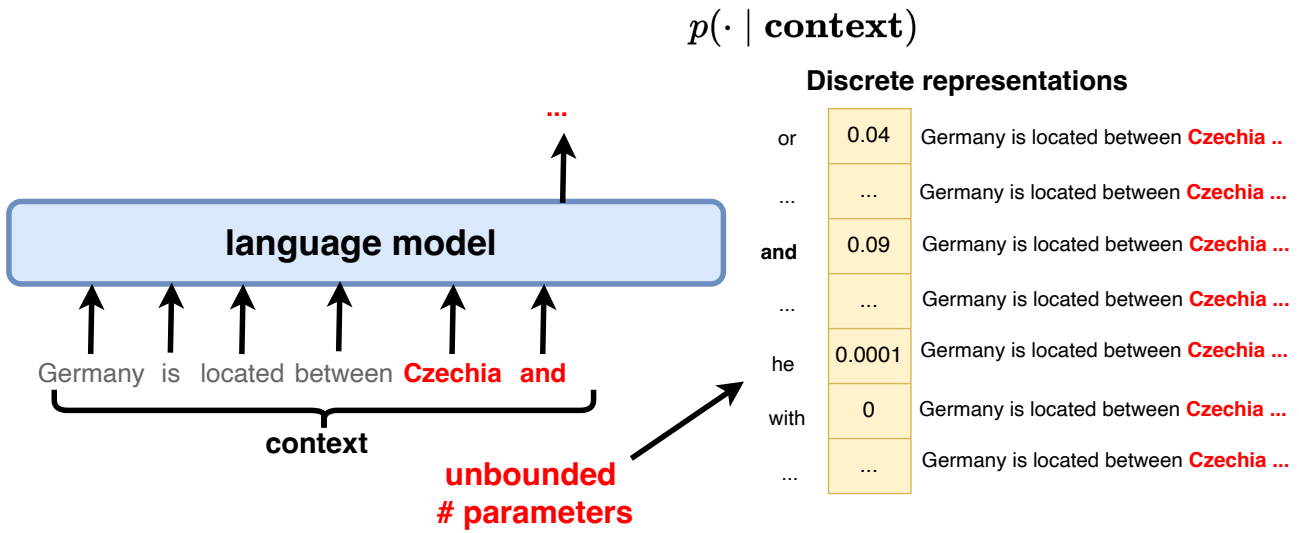
Discrete representations

or	0.04	Germany is located between **Czechia**
...	...	Germany is located between **Czechia**
and	0.09	Germany is located between **Czechia**
...	...	Germany is located between **Czechia**
he	0.0001	Germany is located between **Czechia**
with	0	Germany is located between **Czechia**
...	...	Germany is located between **Czechia**

The problem with traditional solutions

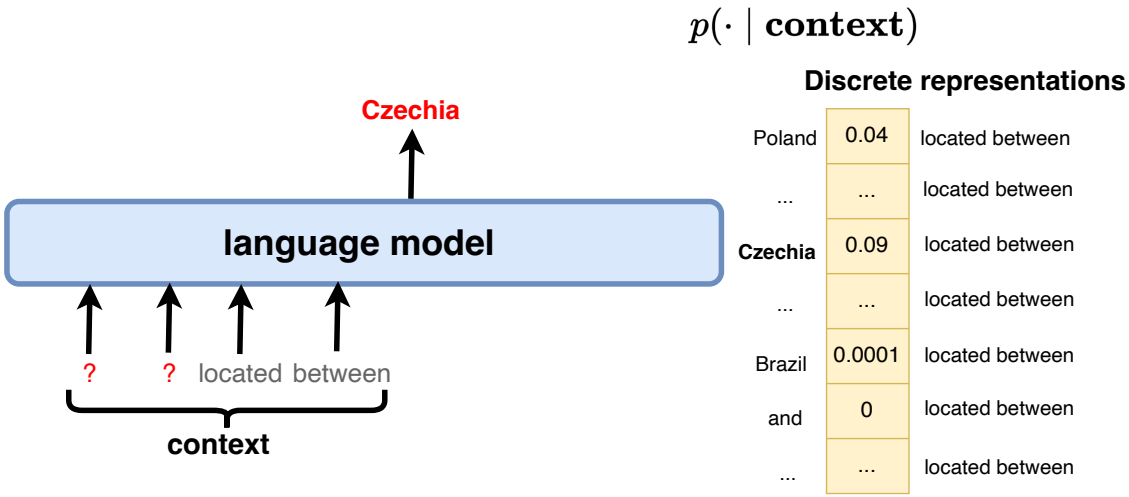


The problem with traditional solutions



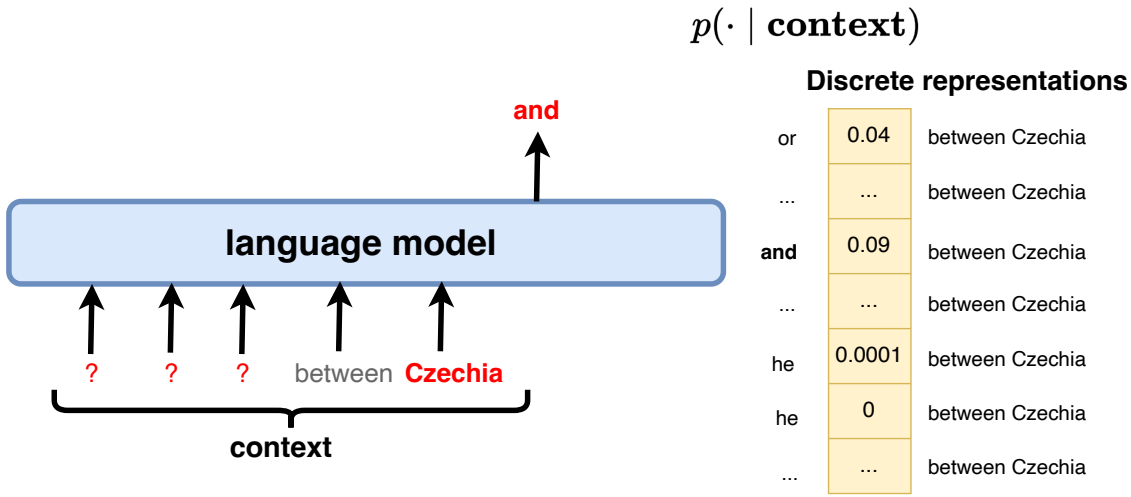
Make models tractable through **independence** or **Markov** assumptions; limit the allowable context.

The problem with traditional solutions



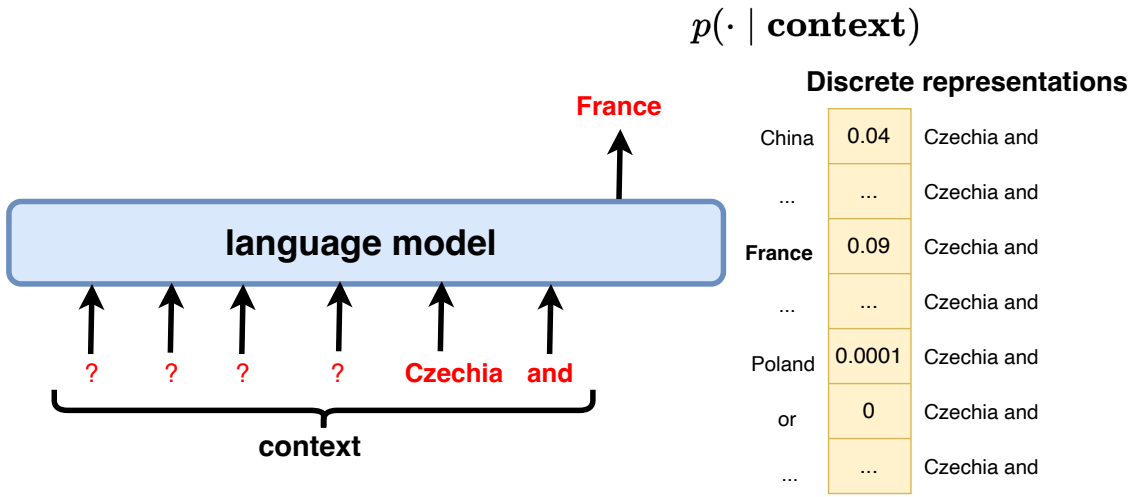
Make models tractable through **independence** or **Markov** assumptions; limit the allowable context.

The problem with traditional solutions



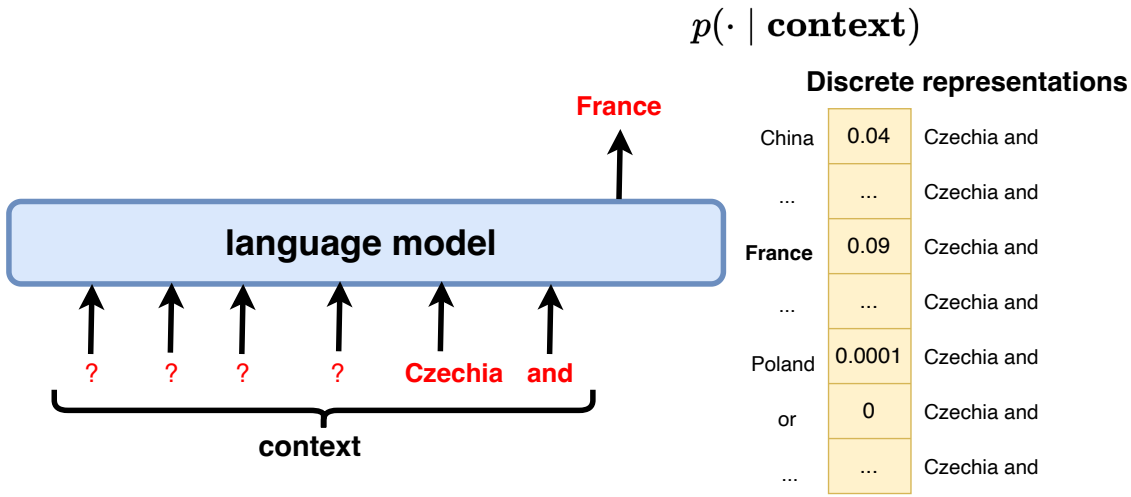
Make models tractable through **independence** or **Markov** assumptions; limit the allowable context.

The problem with traditional solutions



Make models tractable through **independence** or **Markov** assumptions; limit the allowable context.

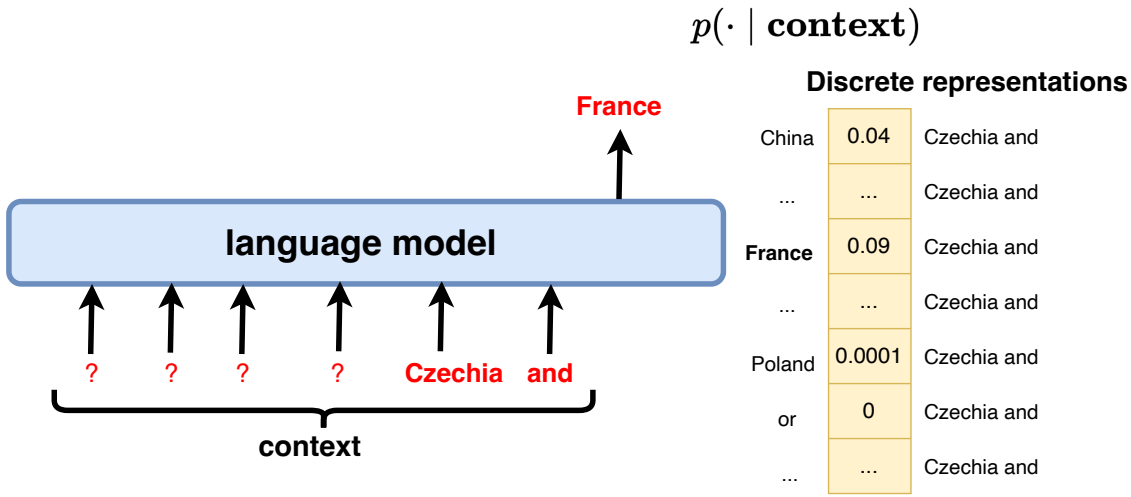
The problem with traditional solutions



Make models tractable through **independence** or **Markov** assumptions; limit the allowable context.

Even limited to 2 previous words and a vocabulary of 100,000 words, results in 10^{15} (**quadrillion**) probability parameters.

The problem with traditional solutions

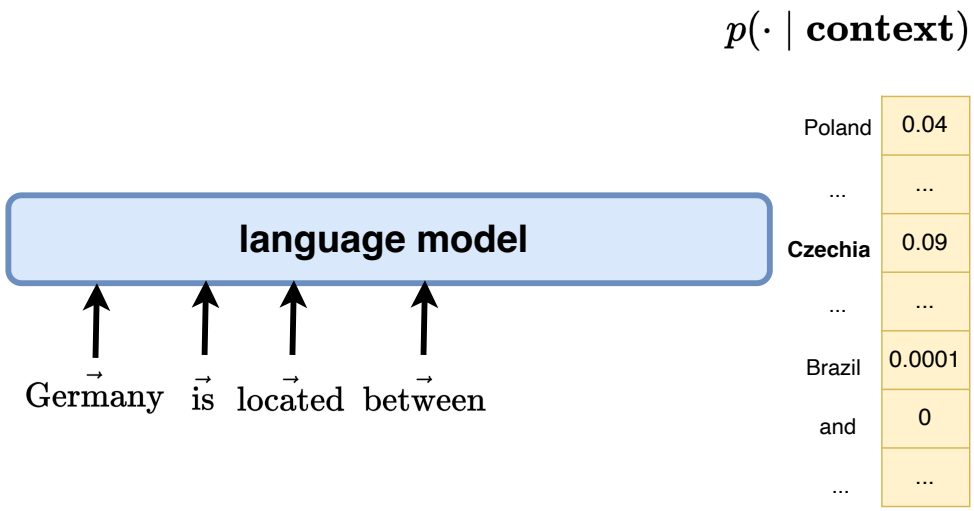


Make models tractable through **independence** or **Markov** assumptions; limit the allowable context.

Do not faithfully model these joint probability distributions!

Moving from the discrete to the continuous

- Modern deep learning systems operate in the continuous space.



What a continuous representation looks like (spaCy)

```
## pip install spacy
## python -m spacy download en_core_web_md
import spacy

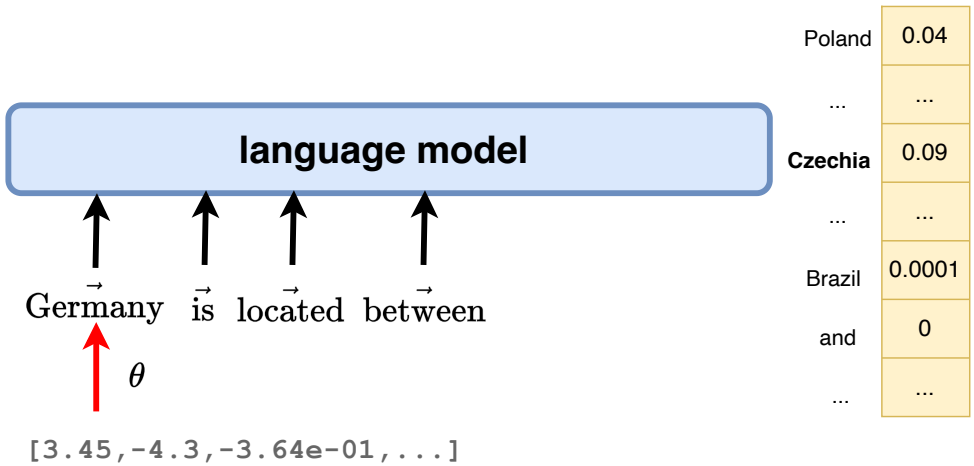
nlp = spacy.load("en_core_web_md")  ## has word vectors

coffee = nlp("coffee")[0]           ## one word (token)
coffee.vector.shape                 ## => (300,) real numbers
coffee.vector[:4]
## => [-0.71252  -0.035167  -0.18155  0.38755]

tea, norway = nlp("tea")[0], nlp("Norway")[0]
coffee.similarity(tea)              ## => 0.57  similar
coffee.similarity(norway)           ## => 0.06  unrelated
```

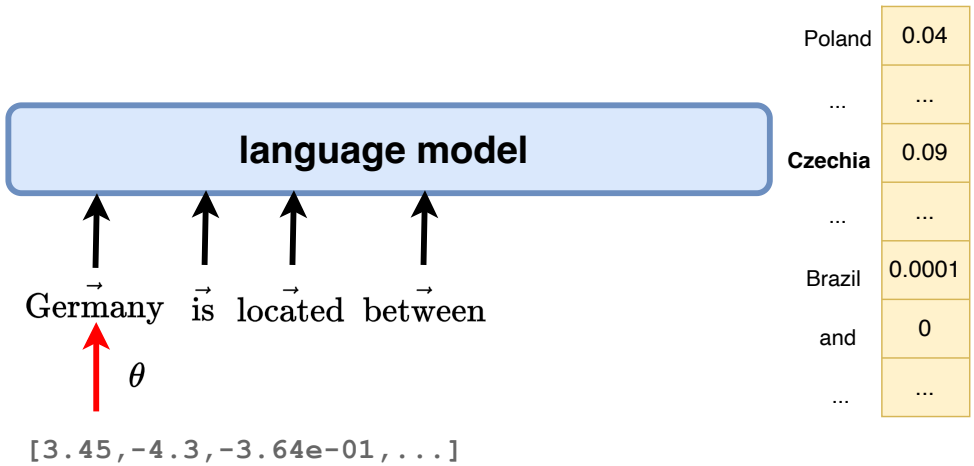
Moving from the discrete to the continuous

- Modern deep learning systems operate in the continuous space.



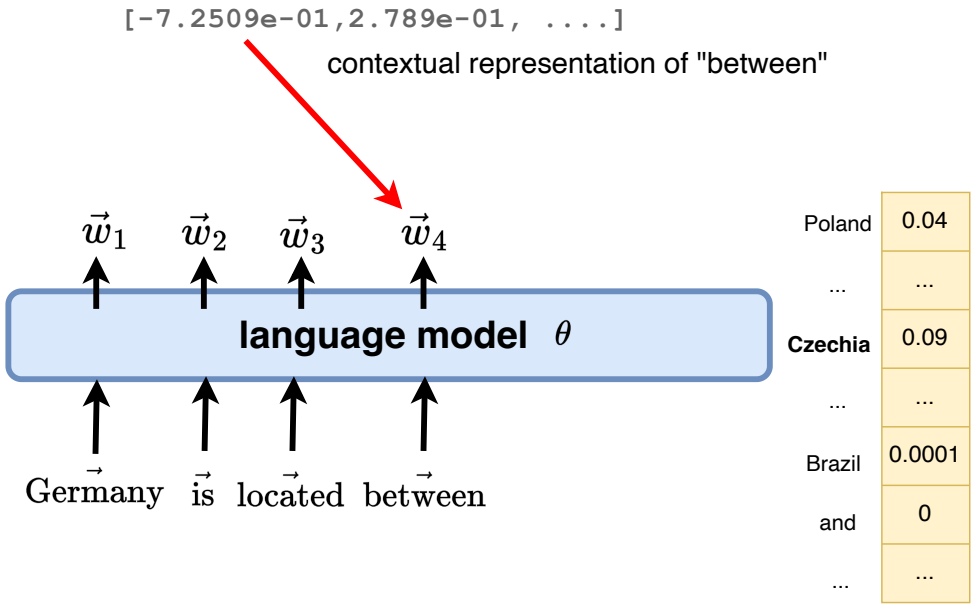
Moving from the discrete to the continuous

- Modern deep learning systems operate in the continuous space.



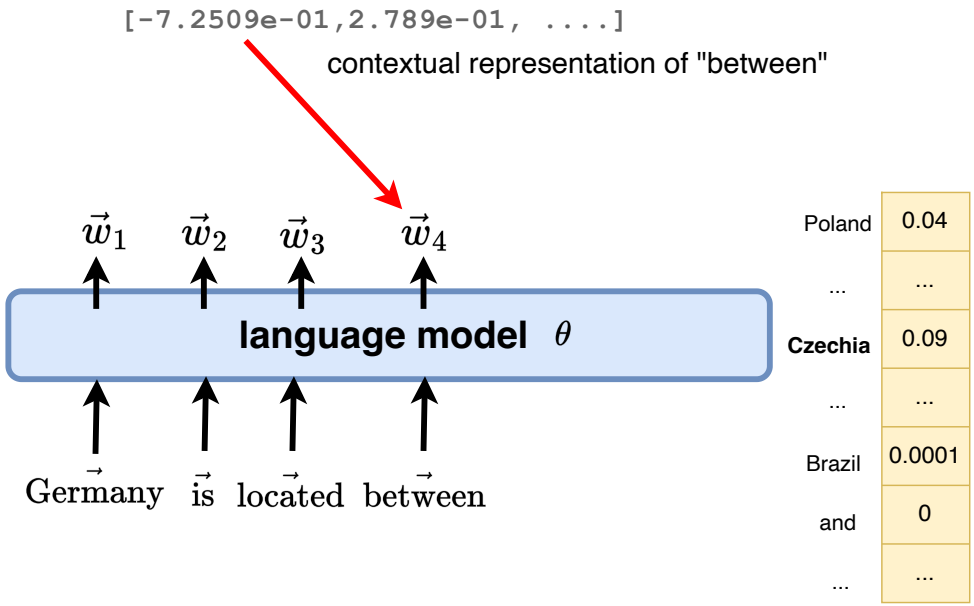
Moving from the discrete to the continuous

- Modern deep learning systems operate in the continuous space.



Moving from the discrete to the continuous

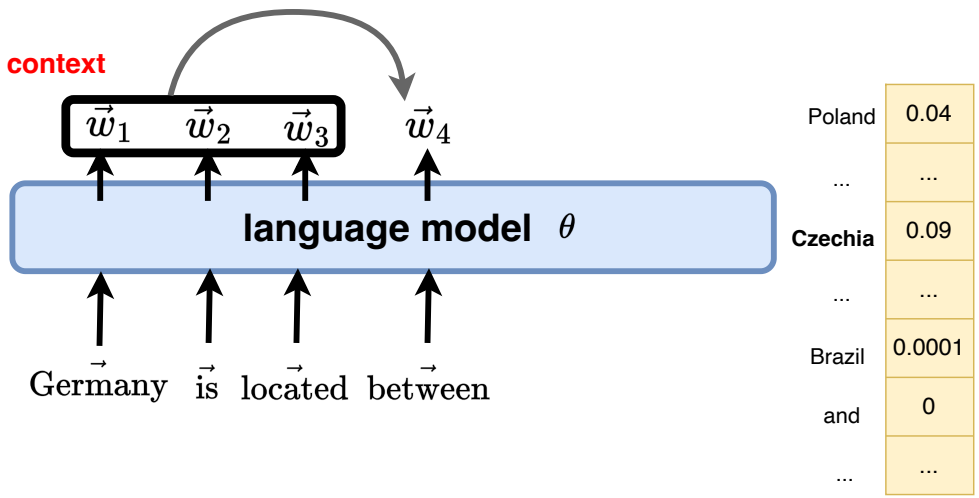
- Modern deep learning systems operate in the continuous space.



How information flows / contextual representations are built, relates to the **model architecture**. Popular: **transformer architecture**.

Moving from the discrete to the continuous

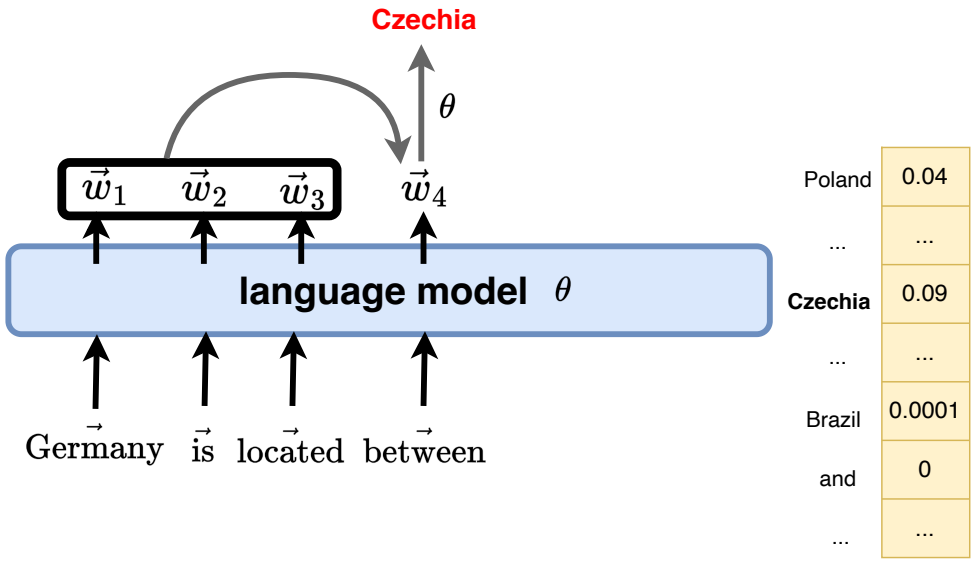
- Modern deep learning systems operate in the continuous space.



How information flows / contextual representations are built, relates to the **model architecture**. Popular: **transformer architecture**.

Moving from the discrete to the continuous

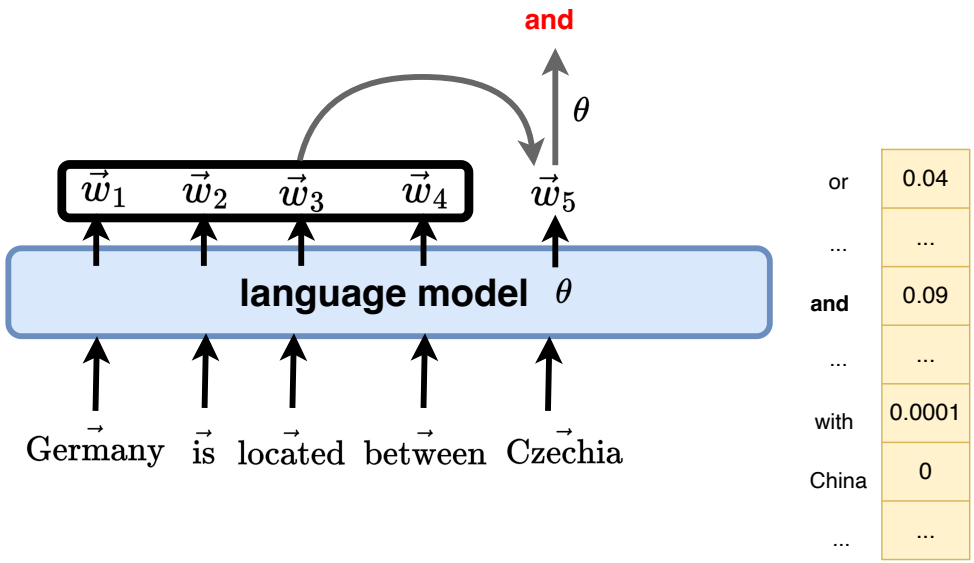
- Modern deep learning systems operate in the continuous space.



Do not make independence assumption, can faithfully model these joint probability distributions.

Moving from the discrete to the continuous

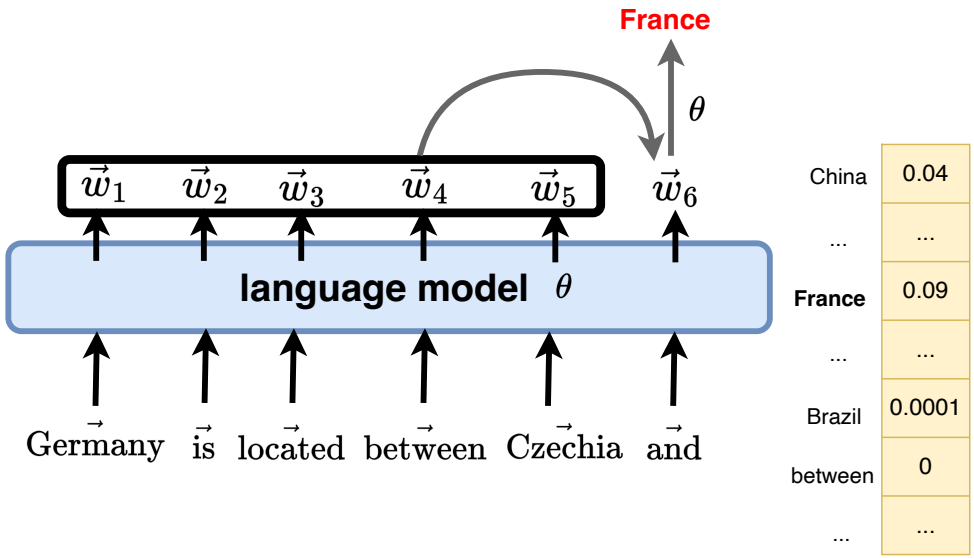
- Modern deep learning systems operate in the continuous space.



Do not make independence assumption, can faithfully model these joint probability distributions.

Moving from the discrete to the continuous

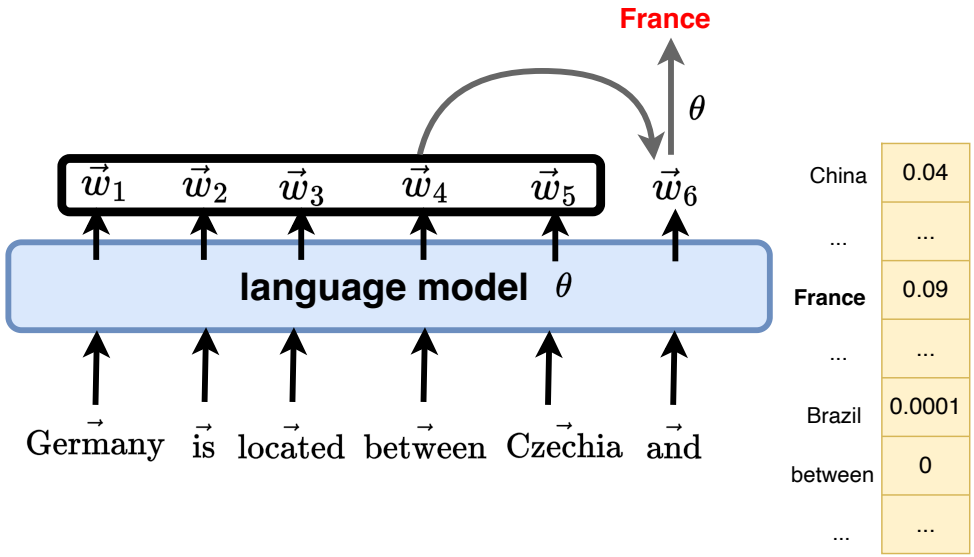
- Modern deep learning systems operate in the continuous space.



Do not make independence assumption, can faithfully model these joint probability distributions.

Moving from the discrete to the continuous

- Modern deep learning systems operate in the continuous space.



Fundamental problem: learning these underlying representations.

It took time to get this right...

Machine translation: a rare word goes wrong

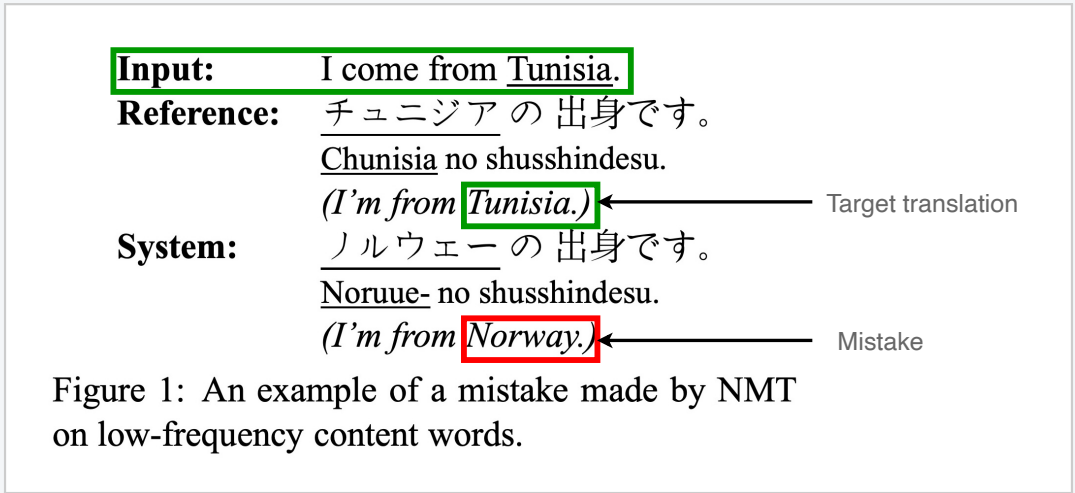


Figure 1: An example of a mistake made by NMT on low-frequency content words.

It took time to get this right...

Machine translation: a rare word goes wrong

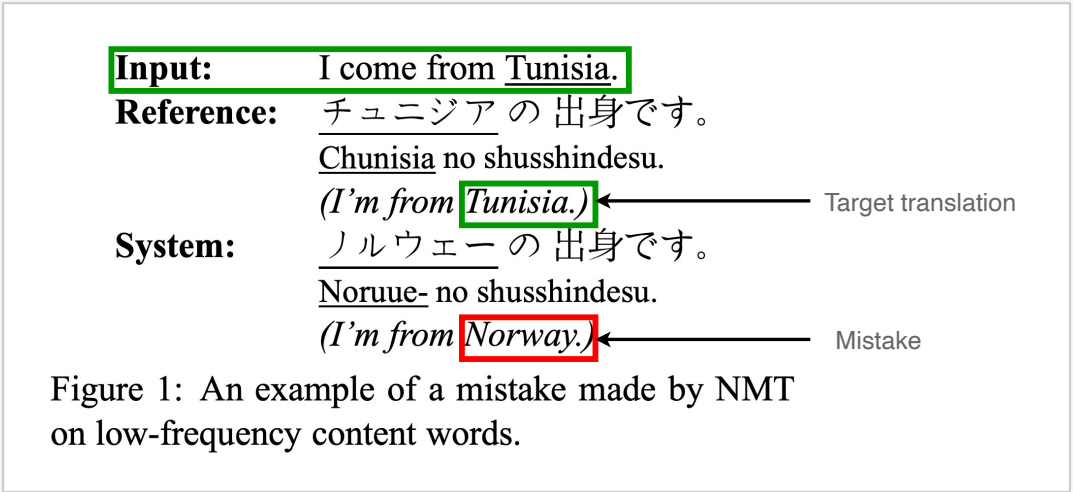


Figure 1: An example of a mistake made by NMT on low-frequency content words.

Text normalization: plausible, fluent, and wrong

Table 2: *Errors from the RNN.*

Input	Correct	Prediction
2 mA	two milliamperes	two million liters
11/10/2008	the tenth of november two thousand eight	the tenth of october two thousand eight
1/2 cc	half a c c	one minute c c
18:00:00Z	eighteen hours zero minutes and zero seconds z	eighteen hundred cubic minutes

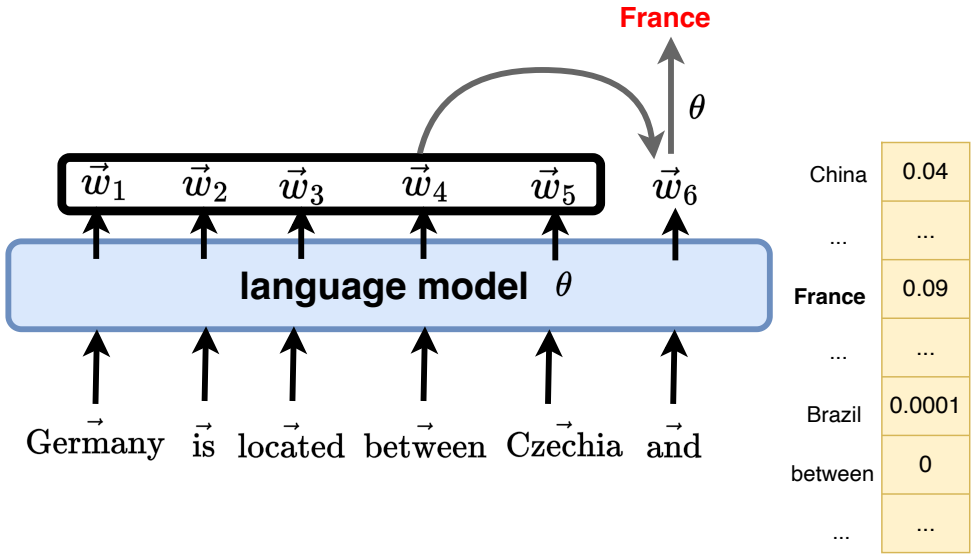
Tunisia and Norway are just nearby vectors

```
import spacy
nlp = spacy.load("en_core_web_md")

tunisia, norway = nlp("Tunisia")[0], nlp("Norway")[0]
tunisia.vector[:5]
## => [-0.60028 -0.24248  0.42605  0.58576  0.1129 ]
norway.vector[:5]
## => [-0.72509  0.2789   0.26907 -0.28551  0.32117]

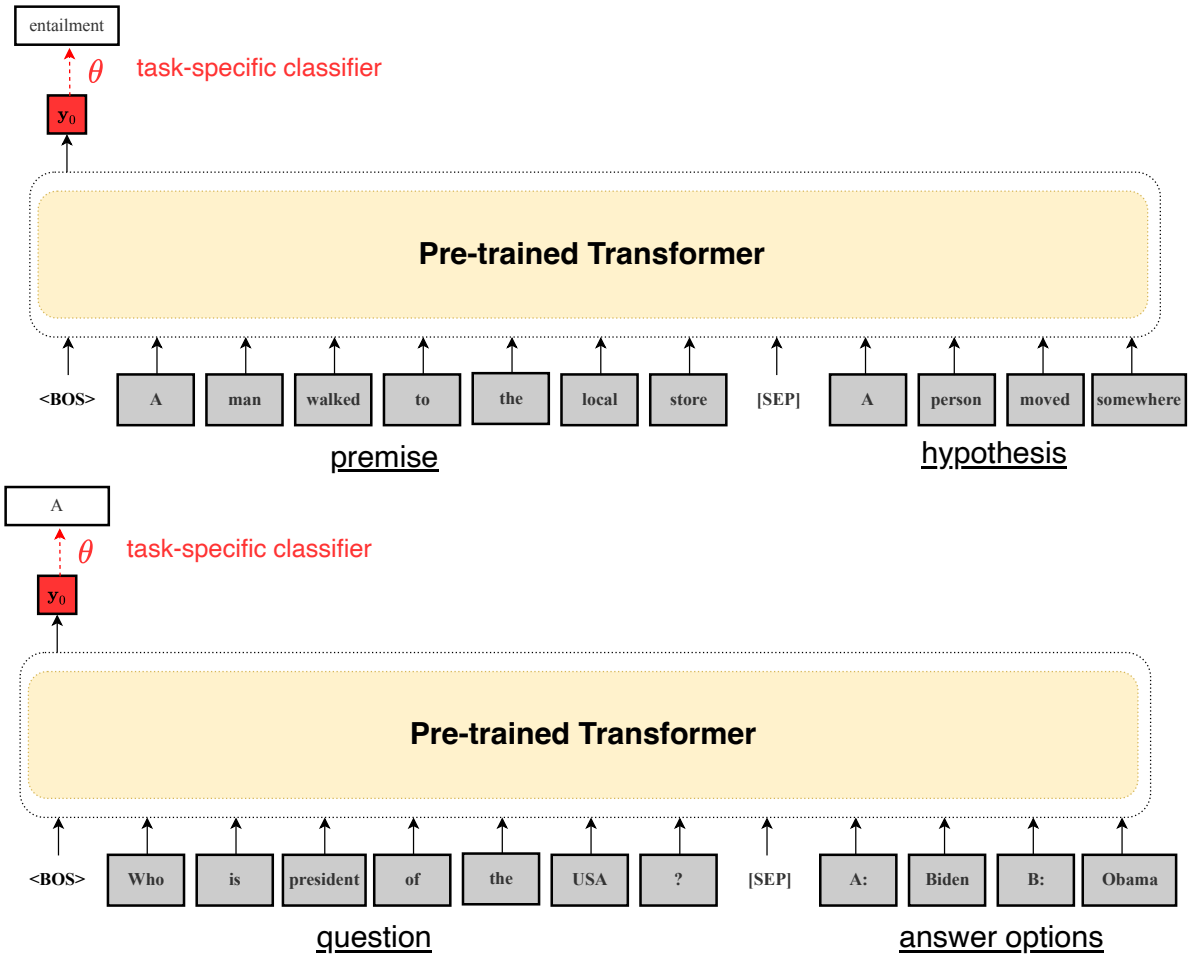
tunisia.similarity(norway)              ## => 0.54  close
tunisia.similarity(nlp("coffee")[0])    ## => 0.02  far
tunisia.similarity(nlp("Morocco")[0]) ## => 1.0   same vector!
```

(Pre-)Training in a nutshell



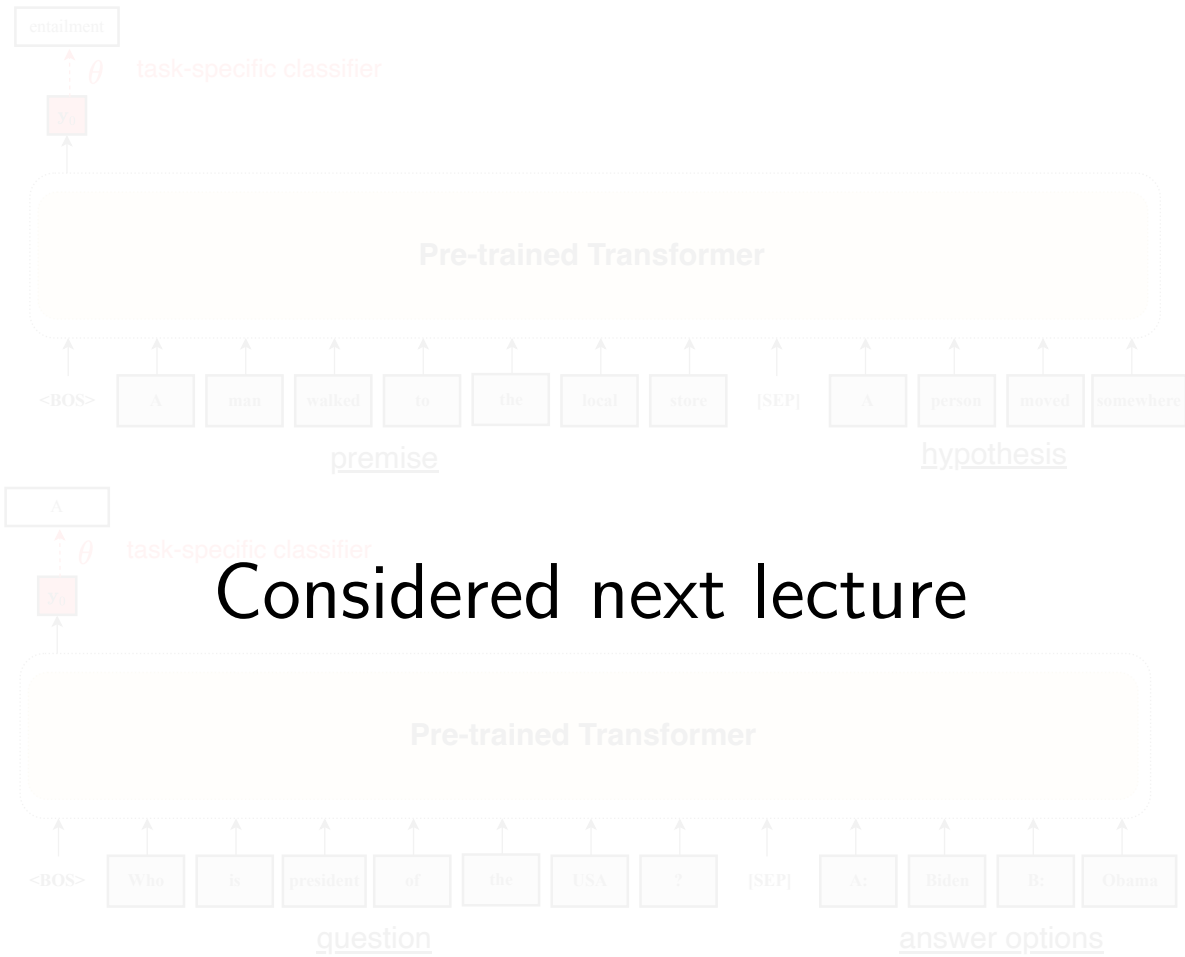
- Estimate parameters θ from signal you get from *auto-completing* example data. Straightforward to get lots of **data for free**.

Model Fine-tuning



Fine-tuning: customizing models to target tasks using additional parameters; **idea**: bootstrap off of pre-training knowledge.

Model Fine-tuning



Considered next lecture

Fine-tuning: customizing models to target tasks using additional parameters; **idea**: bootstrap off of pre-training knowledge.

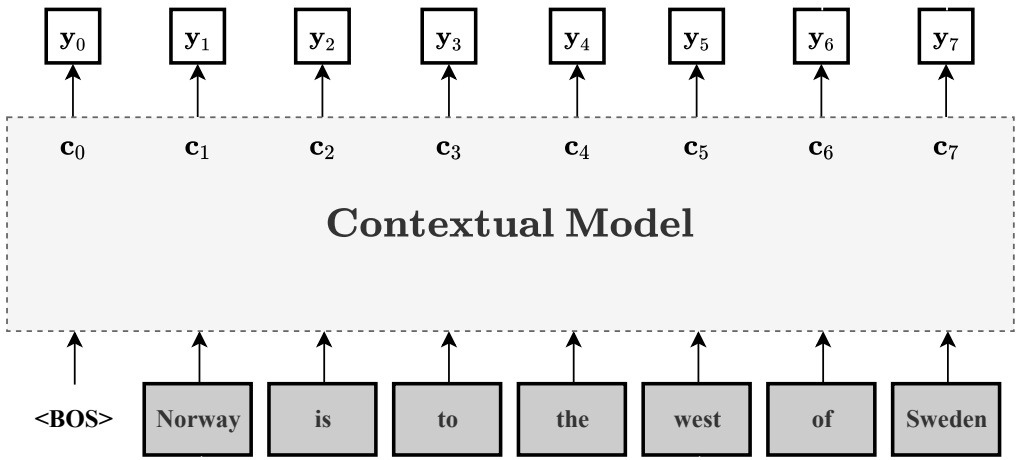
Contextual models and transformers

Contextual Models

- **Role**: assign continuous vectors representations $\mathbf{y}_j \in \mathbb{R}^d$ to elements in a sequence that capture their meaning of those elements in context.

Contextual Models

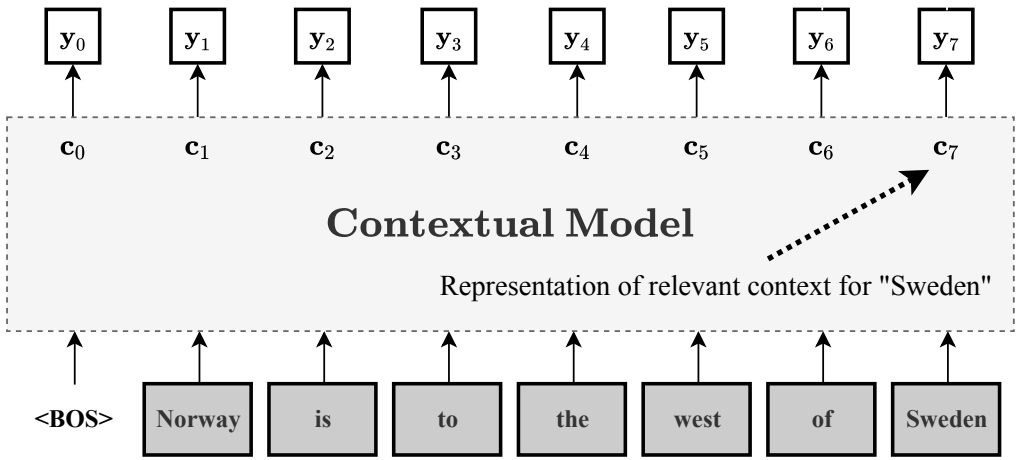
- **Role**: assign continuous vectors representations $\mathbf{y}_j \in \mathbb{R}^d$ to elements in a sequence that capture their meaning of those elements in context.



Operationally: neural network models, often large and opaque.

Contextual Models

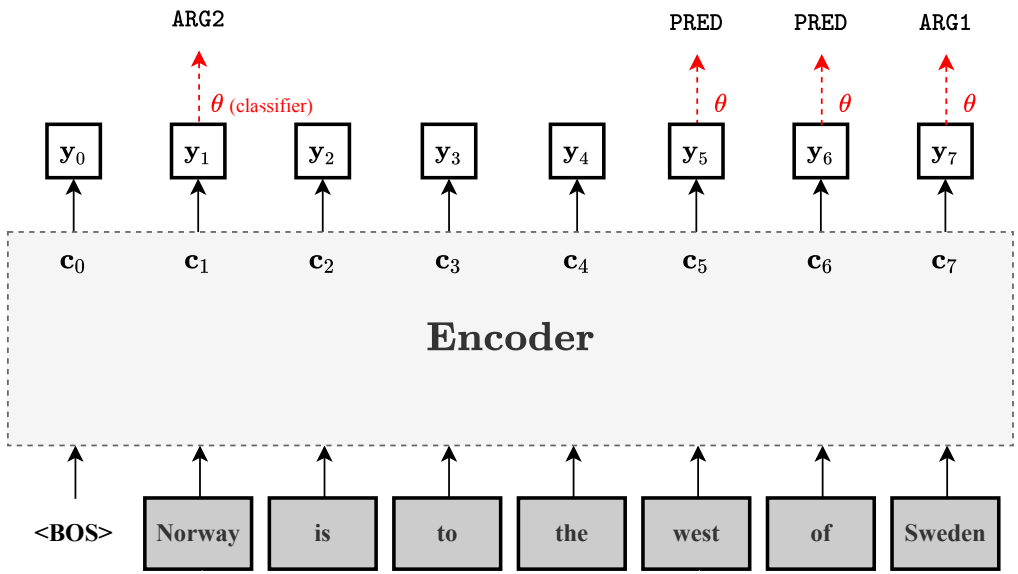
- **Role**: assign continuous vectors representations $\mathbf{y}_j \in \mathbb{R}^d$ to elements in a sequence that capture their meaning of those elements in context.



Semantics: $\mathbf{y}_7$ captures the meaning of *Sweden* grounded in this this particular sentential context, output of a compositional procedure.

Contextual Models

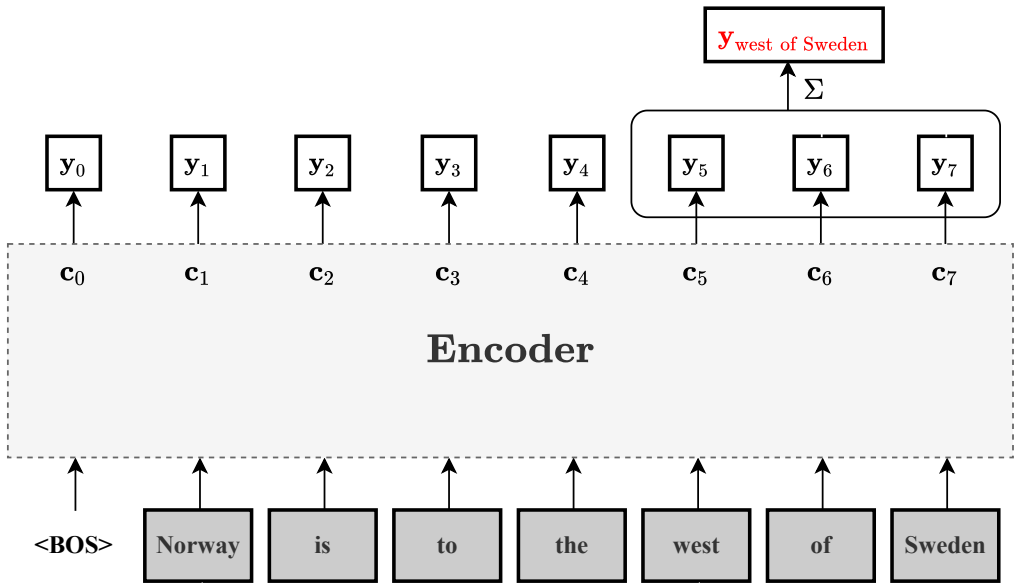
- **Role**: assign continuous vectors representations $\mathbf{y}_j \in \mathbb{R}^d$ to elements in a sequence that capture their meaning of those elements in context.



Why? Build representations that allow you make predictions, **success**: learned representations that effectively solve problems.

Contextual Models

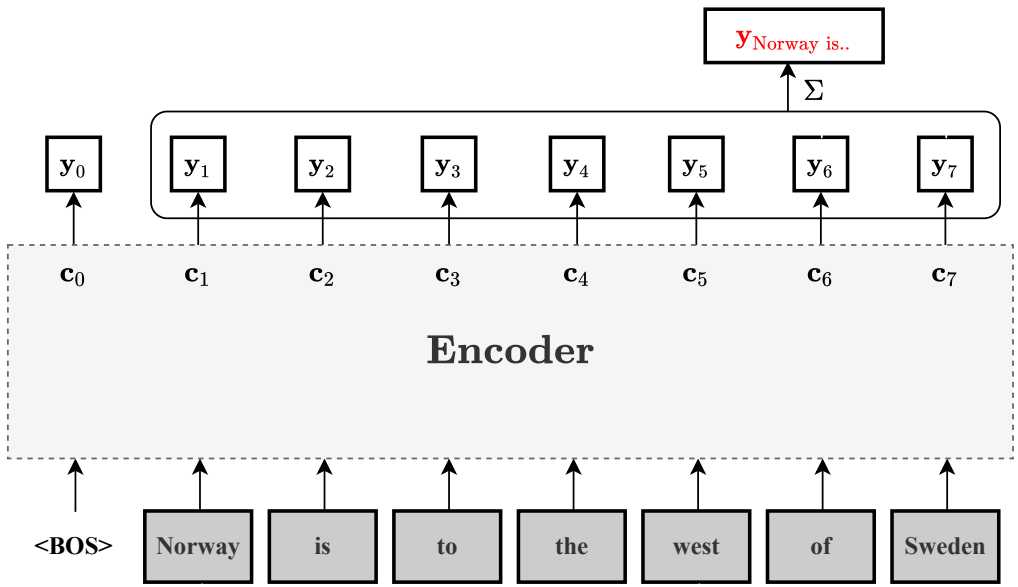
- **Role**: assign continuous vectors representations $\mathbf{y}_j \in \mathbb{R}^d$ to elements in a sequence that capture their meaning of those elements in context.



Why? Build representations that allow you make predictions, **success**: learned representations that effectively solve problems.

Contextual Models

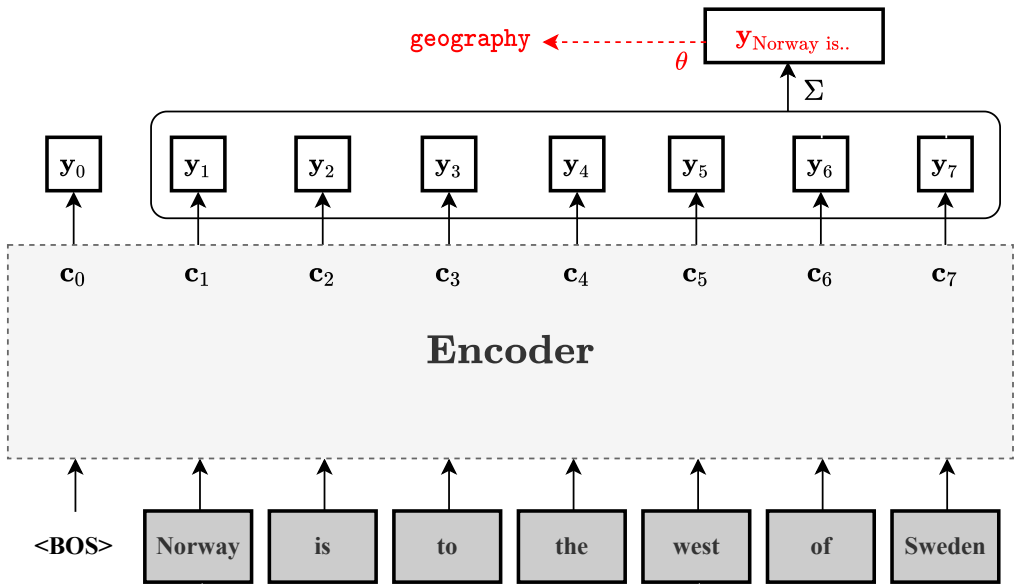
- **Role**: assign continuous vectors representations $\mathbf{y}_j \in \mathbb{R}^d$ to elements in a sequence that capture their meaning of those elements in context.



Why? Build representations that allow you make predictions, **success**: learned representations that effectively solve problems.

Contextual Models

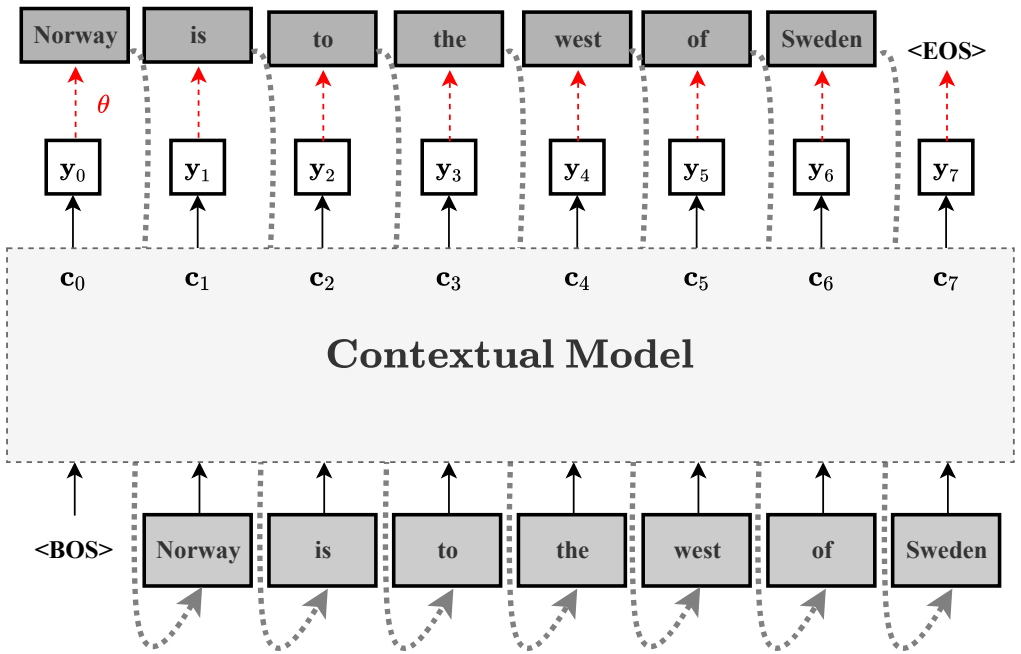
- **Role**: assign continuous vectors representations $\mathbf{y}_j \in \mathbb{R}^d$ to elements in a sequence that capture their meaning of those elements in context.



Why? Build representations that allow you make predictions, **success**: learned representations that effectively solve problems.

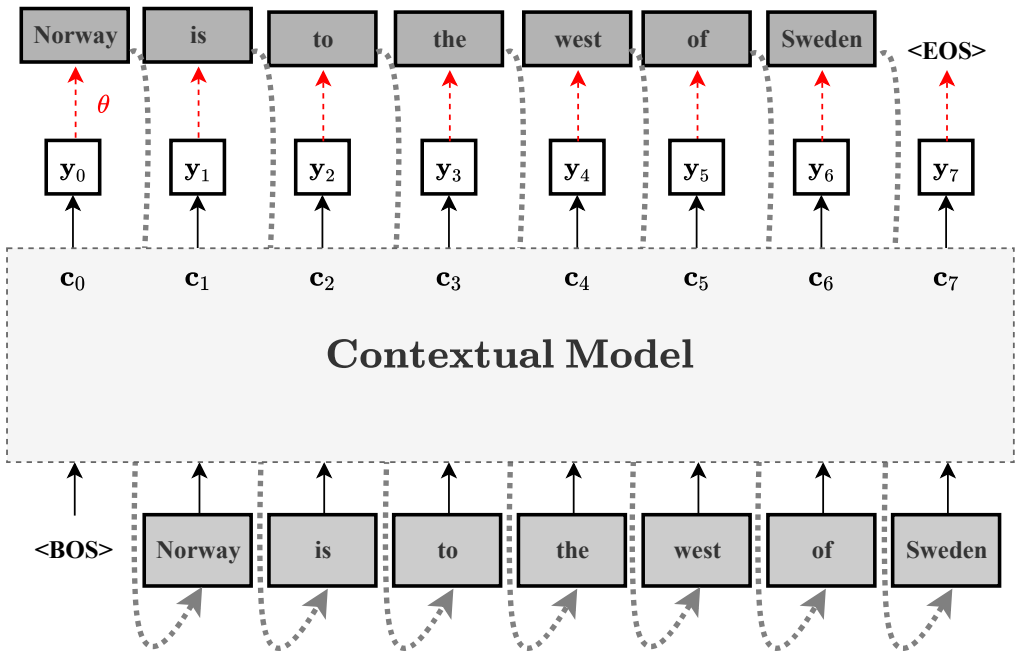
Contextual Models

- **Role**: assign continuous vectors representations $\mathbf{y}_j \in \mathbb{R}^d$ to elements in a sequence that capture their meaning of those elements in context.



Contextual Models

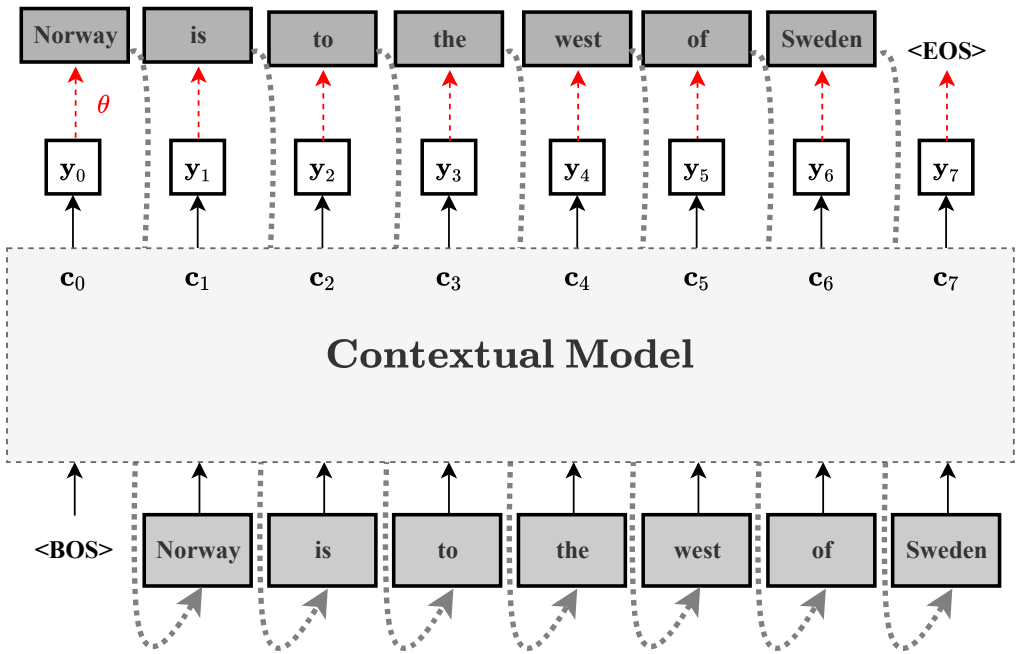
- **Role**: assign continuous vectors representations $\mathbf{y}_j \in \mathbb{R}^d$ to elements in a sequence that capture their meaning of those elements in context.



as LMs: $p(w_j \mid w_1, \ldots, w_{j-1}) = p(w_j \mid \mathbf{c}_{j-1})$, **Important:** Can condition on full contexts, *faithfully* model complex joint probability distributions

Contextual Models

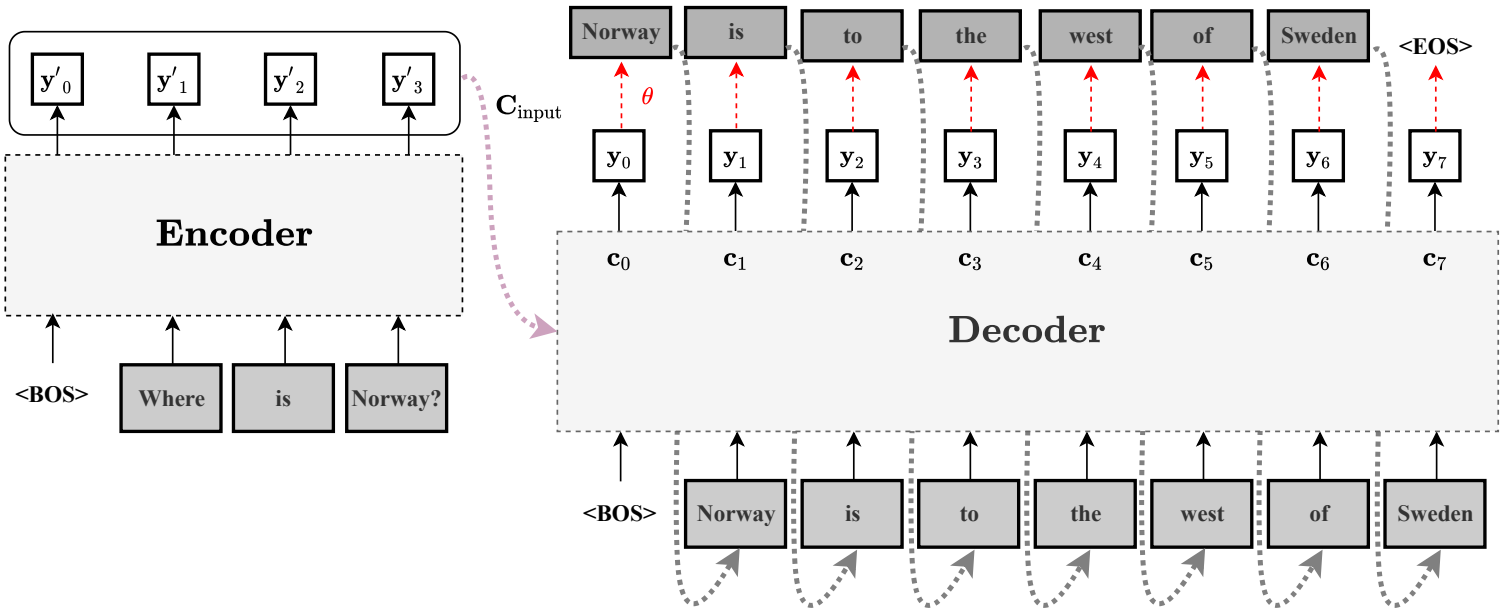
- **Role**: assign continuous vectors representations $\mathbf{y}_j \in \mathbb{R}^d$ to elements in a sequence that capture their meaning of those elements in context.



Model Architectures: how information is processed, internal representations are constructed. **Common**: RNNs, Transformers.

Contextual Models

- **Role**: assign continuous vectors representations $\mathbf{y}_j \in \mathbb{R}^d$ to elements in a sequence that capture their meaning of those elements in context.

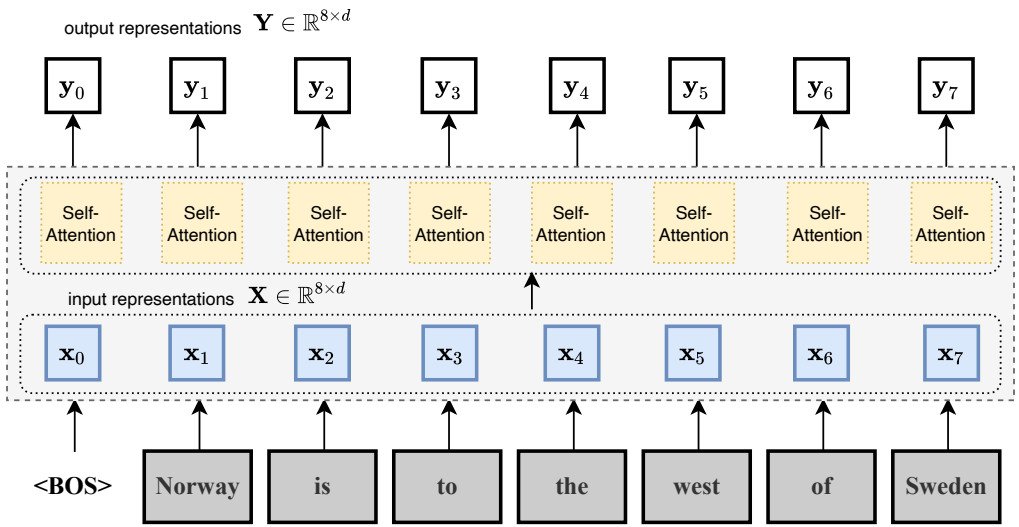


Text2Text models: estimate $p(y^{\text{output}} \mid x^{\text{input}})$, natural way to express many language understanding problems.

Transformers and attention

Self-Attention: Simple Encoder Example

Attention: Mechanism for building contextual representations (see Vaswani et al.2017)).



Embedding representations in PyTorch

```
import torch ## to install: pip install torch

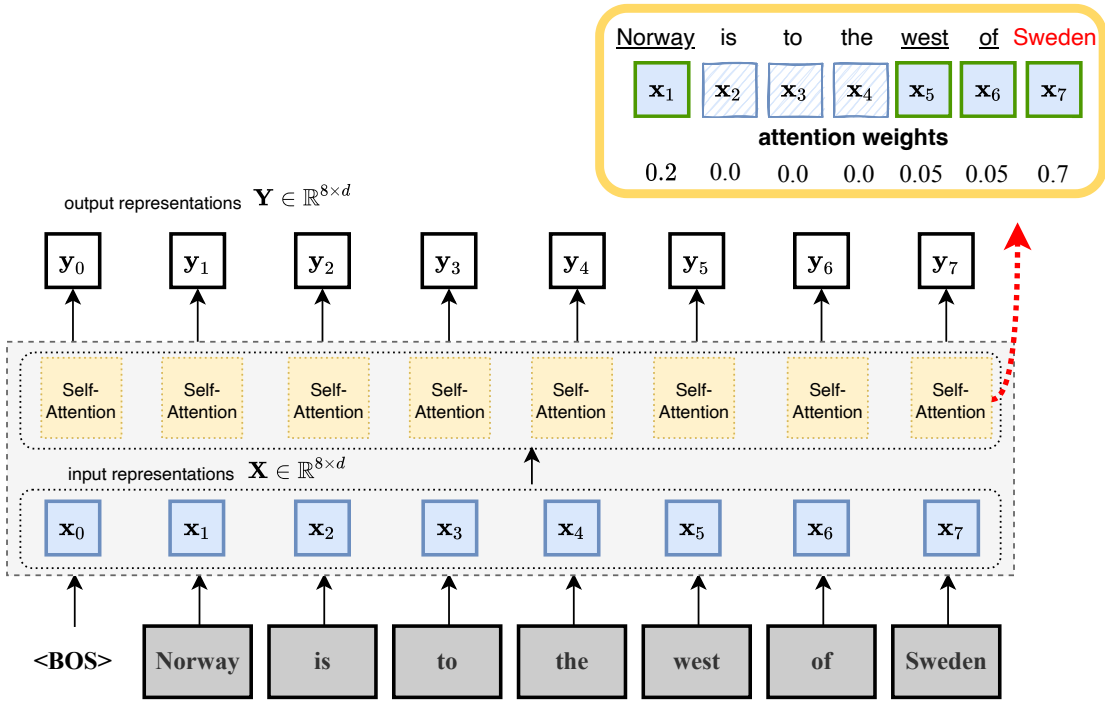
### word embedding parameters and matrix E
E = torch.nn.Embedding(
    embedding_dim=768,  ##<--- dimensionality
    num_embeddings=3000, ##<--- # words
)

##e.g., Representation of our Input
X = E(torch.tensor([
  0, # <BOS>
  1, # Norway
  2, # is
  3, # to
  4, # the
  5, # west
  6, # of
  7, # Sweden
])) ### => matrix 7 * 768

x1 = X[1] ##-> initial representation of Norway
print(x1)
###=> [-1.1344e+00, -3.0359e-01,  7.3585e-02, .....]
```

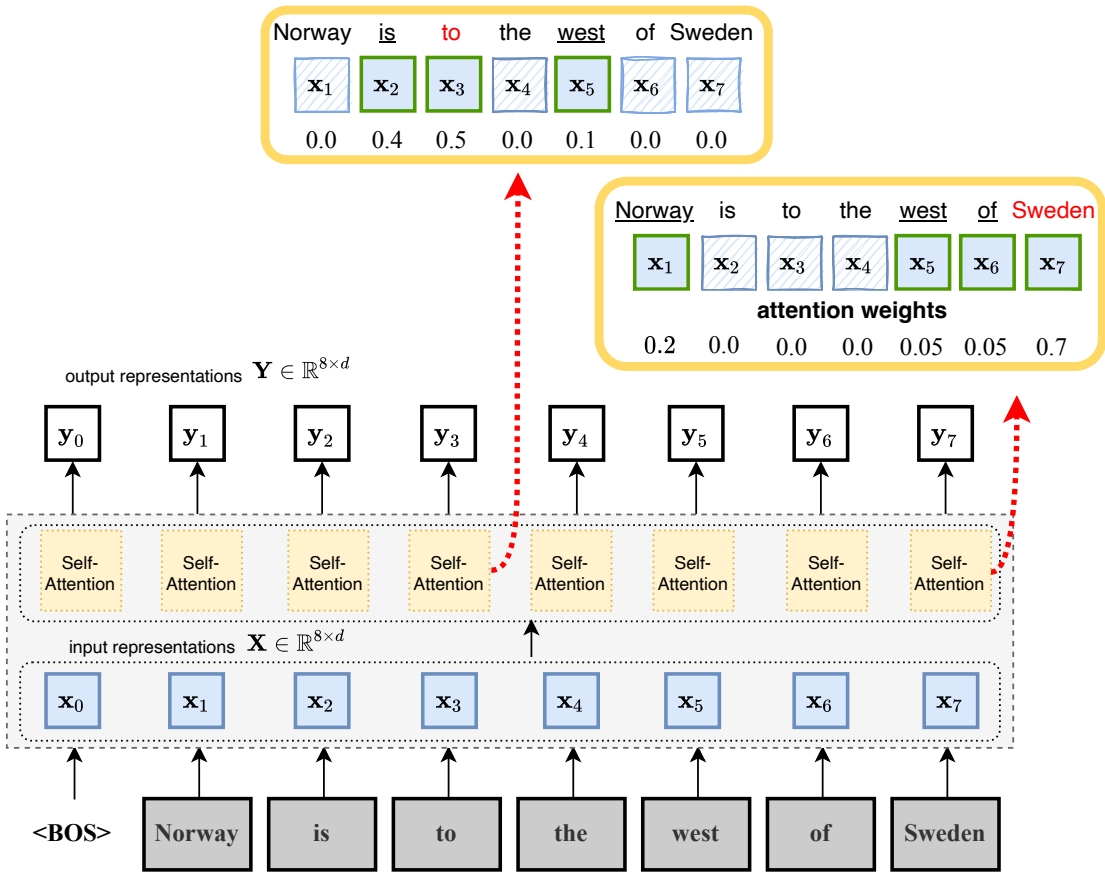
Self-Attention: Simple Encoder Example

Attention: Mechanism for building contextual representations (see Vaswani et al.2017)).



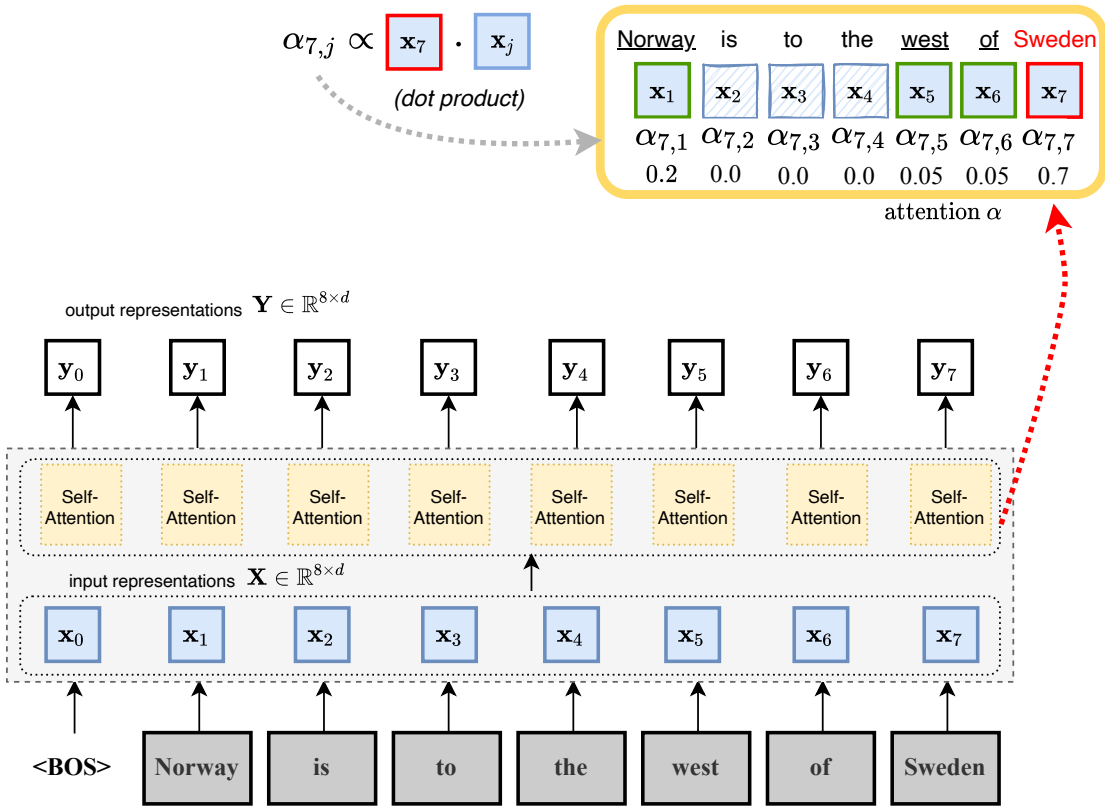
Self-Attention: Simple Encoder Example

Attention: Mechanism for building contextual representations (see Vaswani et al.2017)).



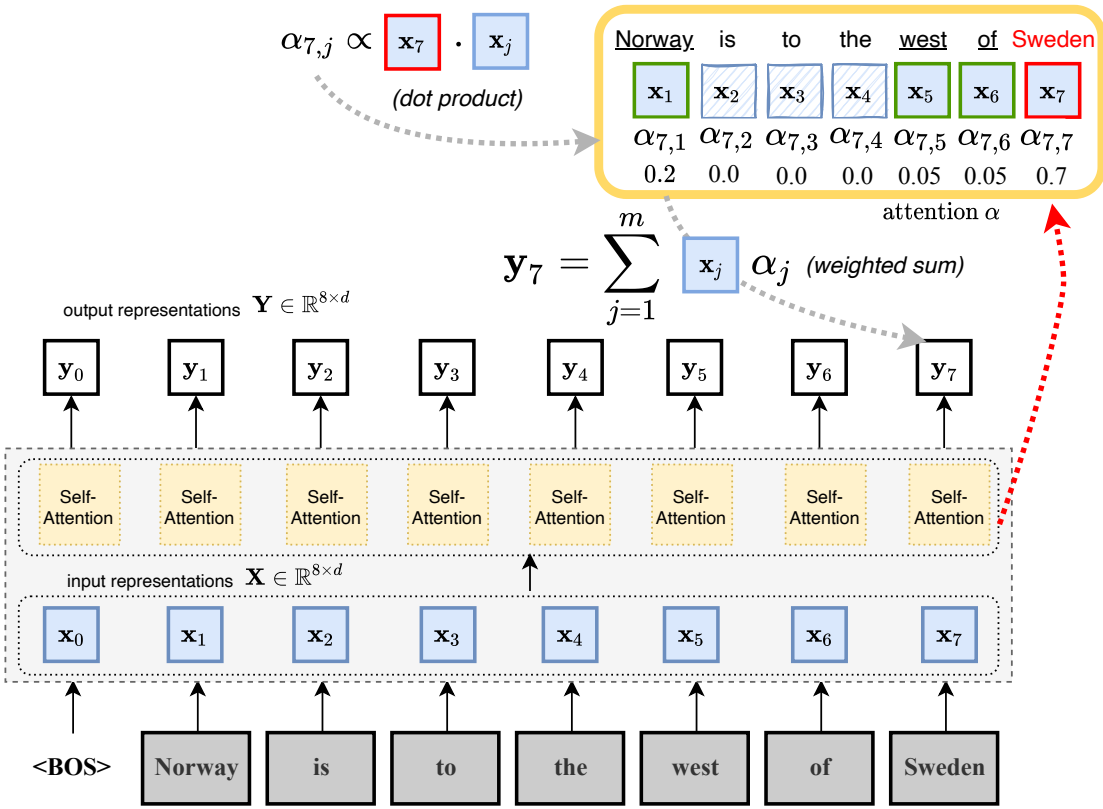
Self-Attention: Simple Encoder Example

Attention: Mechanism for building contextual representations (see Vaswani et al.2017)).



Self-Attention: Simple Encoder Example

Attention: Mechanism for building contextual representations (see Vaswani et al.2017)).



Self-Attention: Computing Final Representations

The full computation

$$\alpha'_{i,j} = \mathbf{x}_i \cdot \mathbf{x}_j \qquad \textbf{dot product}$$

$$\alpha_{i,j} = \frac{e^{\alpha'_{i,j}}}{\sum_j e^{\alpha'_{i,j}}} \qquad \textbf{softmax}$$

$$\mathbf{y}_i = \sum_j \alpha_{i,j} \mathbf{x}_j \qquad \textbf{weighted sum}$$

Self-Attention: Computing Final Representations

The full computation

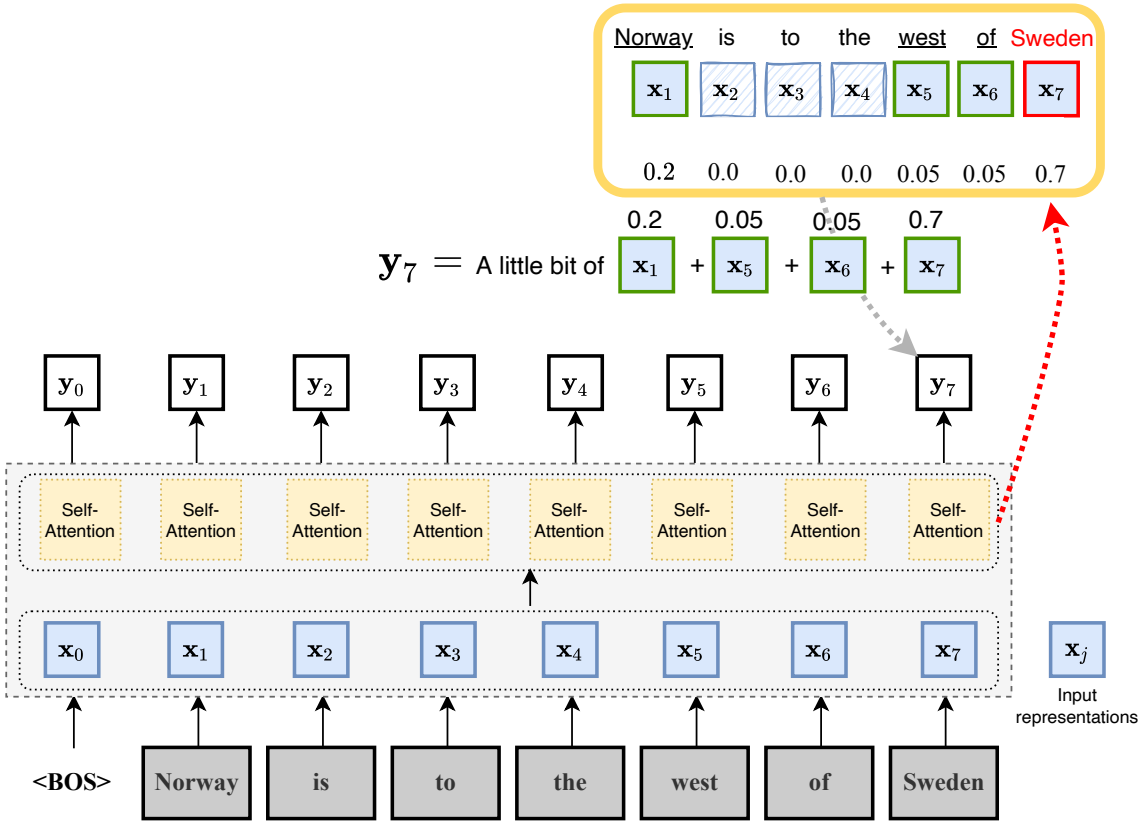
Self-attention in a few lines of PyTorch

```
## Input representations (again),
X = E(torch.tensor([0,1,2,3,4,5,6,7]))

### raw weights (dot product / matrix multiplication)
raw_Alpha =  torch.matmul(X,X.transpose(0,1))
### normalized via softmax, probability distribution
alpha =  raw_Alpha.softmax(dim=-1)
### Final self attention representations
Y =  torch.matmul(alpha,X)
### self attention representation of `Norway`
y1 = Y[1]
```

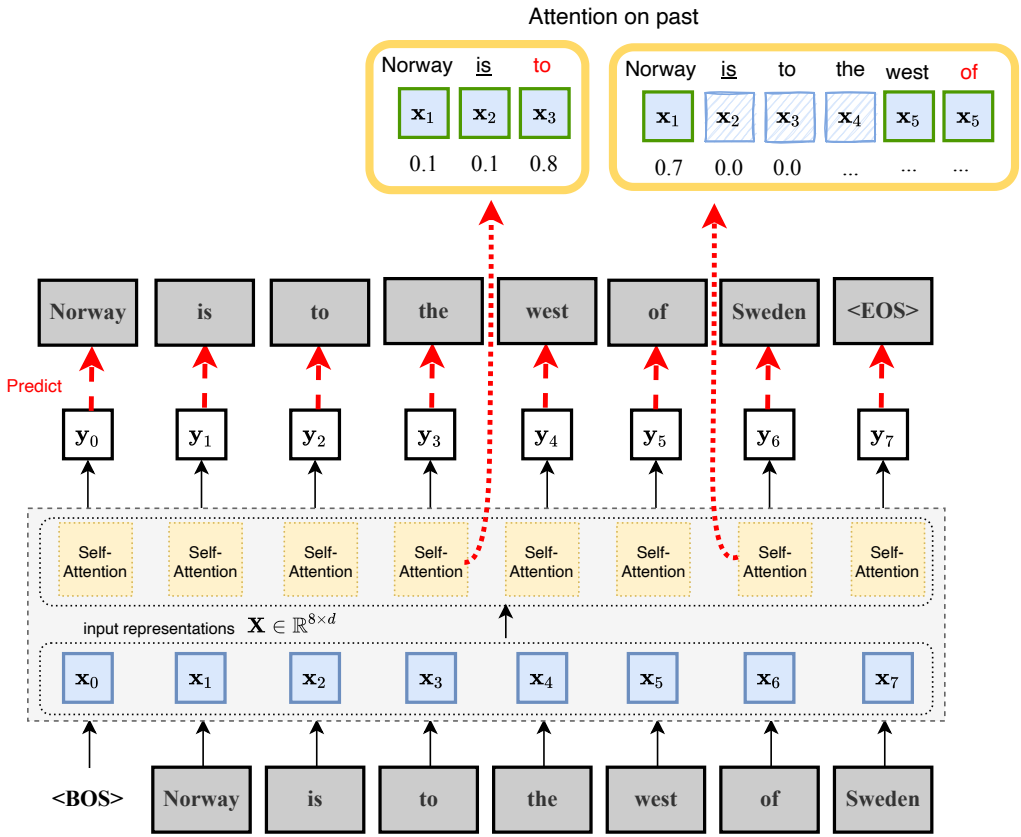
Self-Attention: Intuition

Attention: A kind of brute-force looking around and aggregation of contextual information.



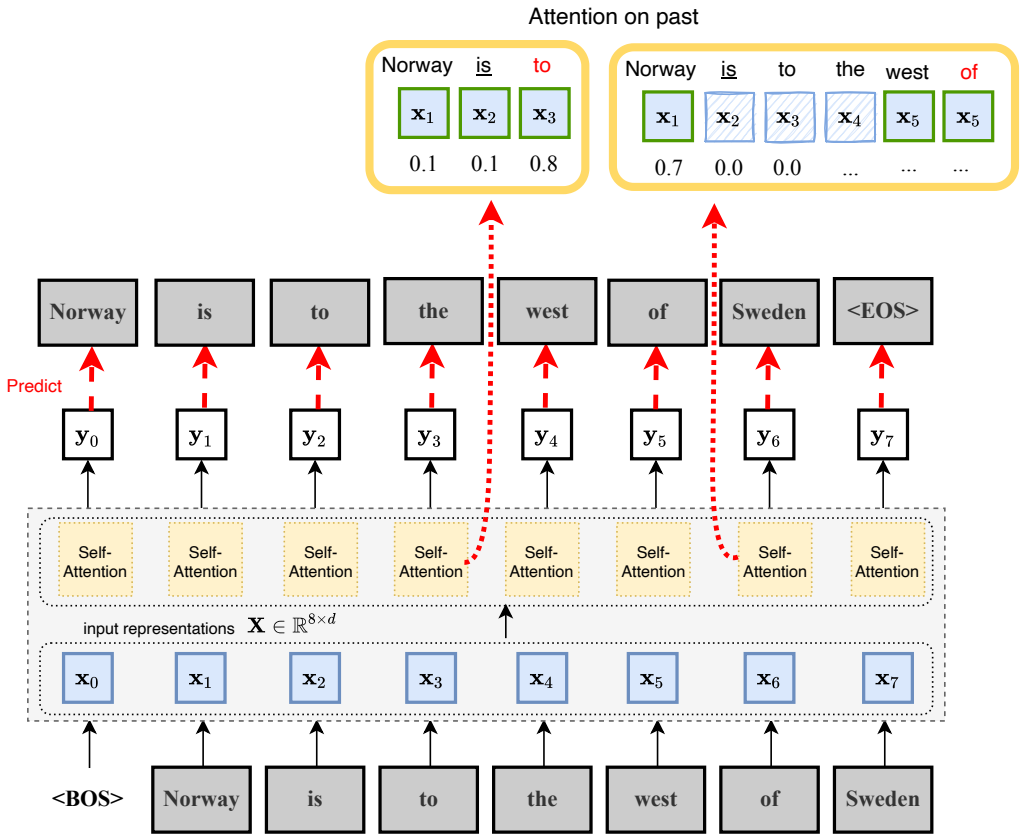
Attention in Other Contexts

Decoders: Attention limited to past context, allows for generation (step-by-step prediction of next word).



Attention in Other Contexts

Decoders: Attention limited to past context, allows for generation (step-by-step prediction of next word).



Note: No independence assumptions, full context.

Causal attention: adding a mask

```
import torch
## Input representations (again),
X = E(torch.tensor([0,1,2,3,4,5,6,7]))

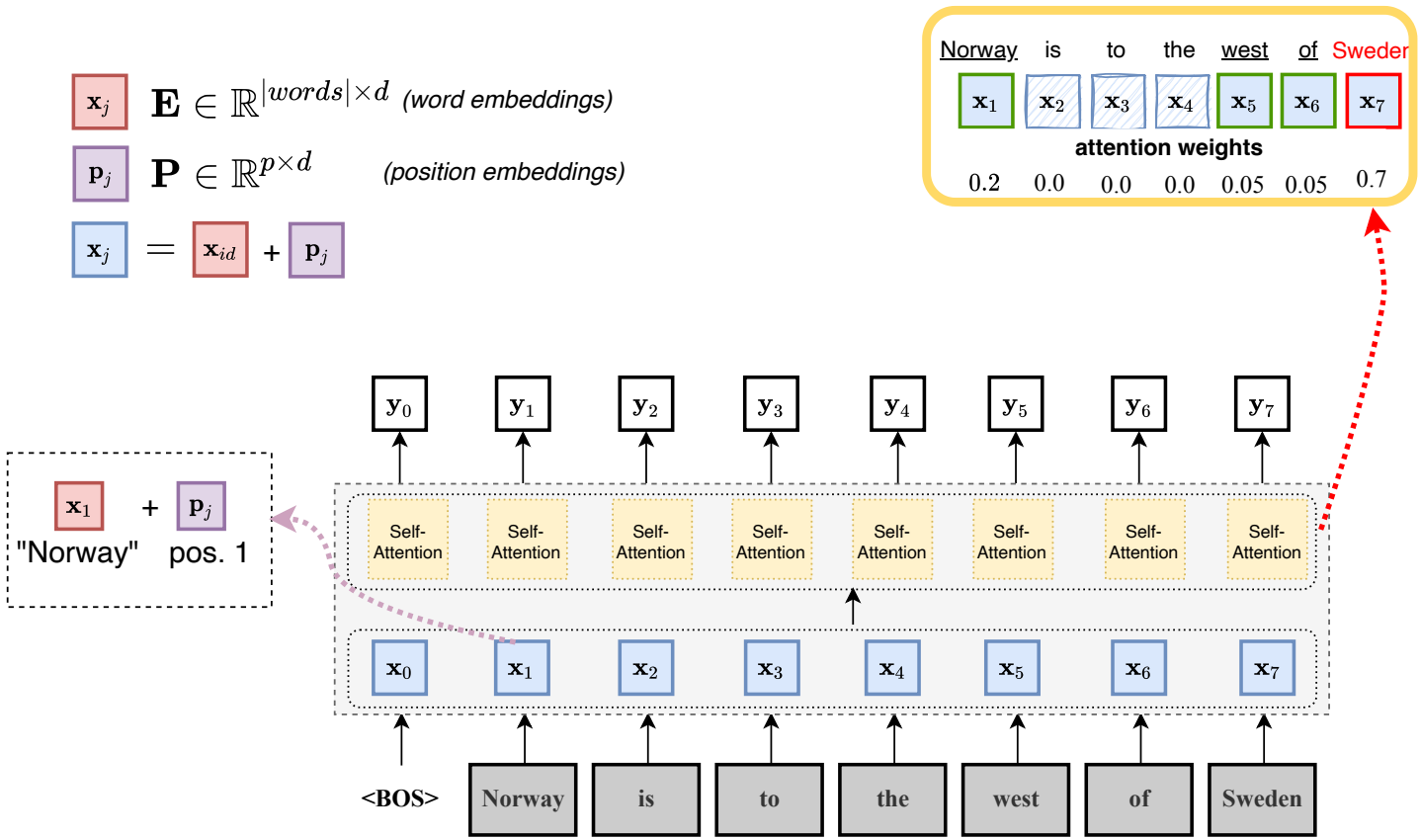
### raw weights (dot product / matrix multiplication)
raw_Alpha =  torch.matmul(X,X.transpose(0,1))

### causal mask
mask = torch.triu_indices(8, 8, offset=1)
raw_Alpha[mask[0], mask[1]] = float('-inf')

### normalized via softmax, probability distribution
alpha =  raw_Alpha.softmax(dim=-1)
### Final self attention representations
Y =  torch.matmul(alpha,X)
### self attention representation of `Norway`
y1 = Y[1]
```

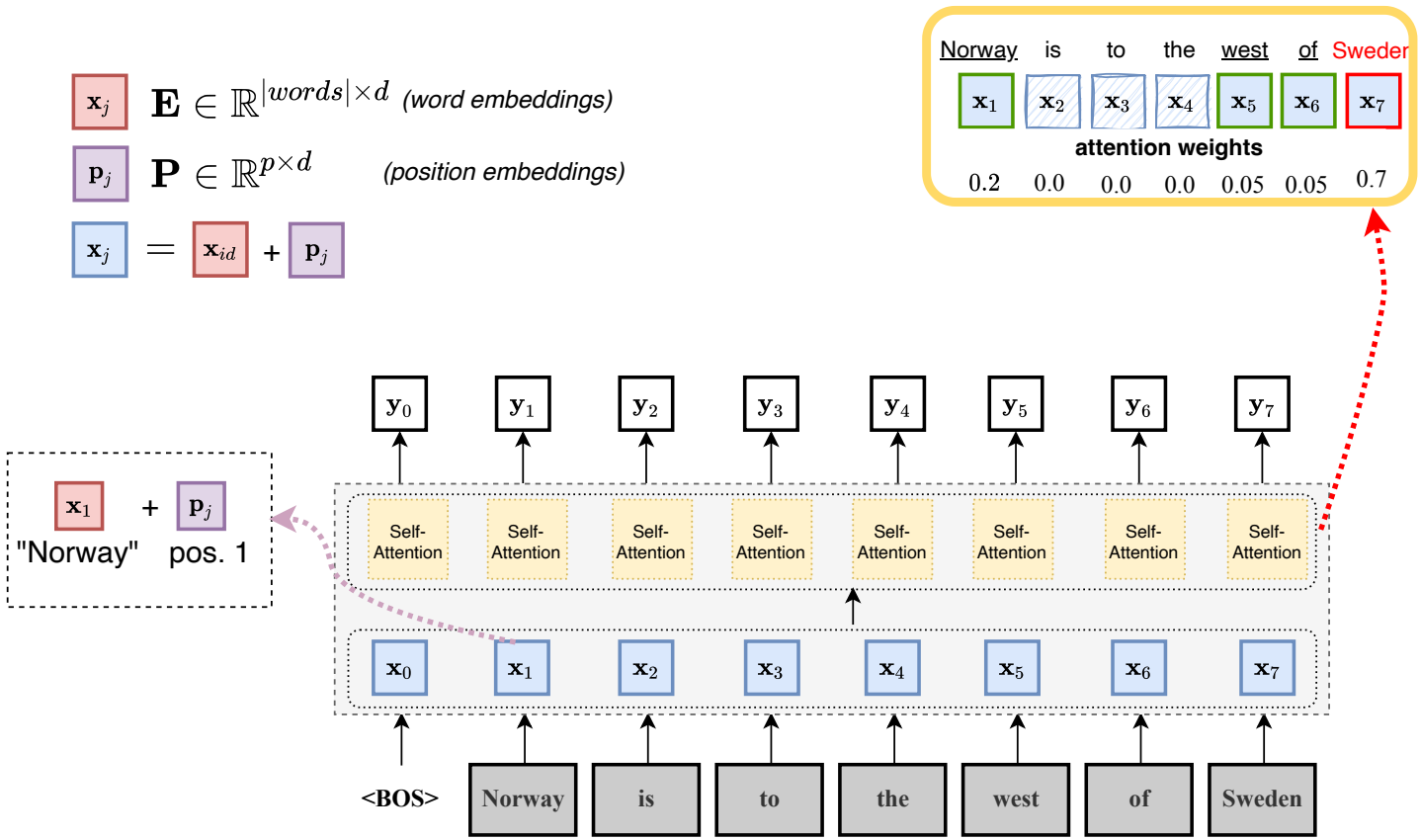
An important detail: positional information

- Word embeddings so far do not encode position information.



An important detail: positional information

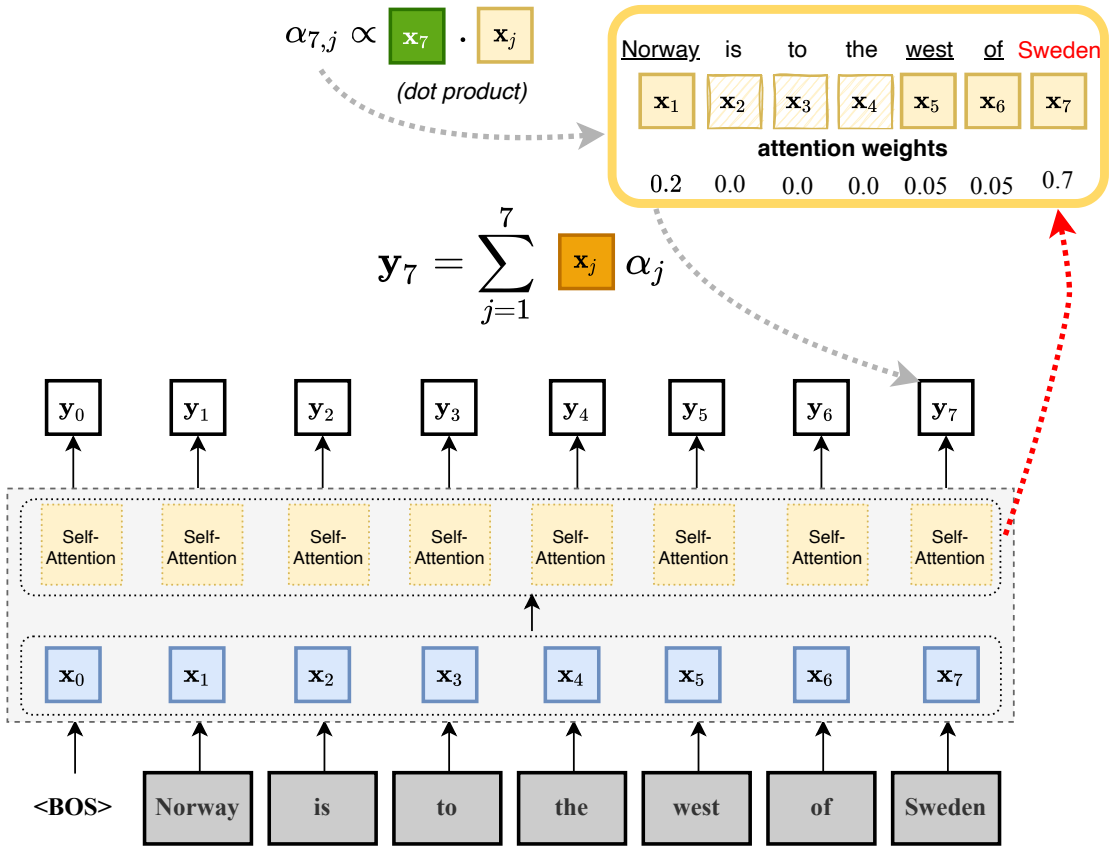
- Word embeddings so far do not encode position information.



Note: we can add any additional information we want, many variations.

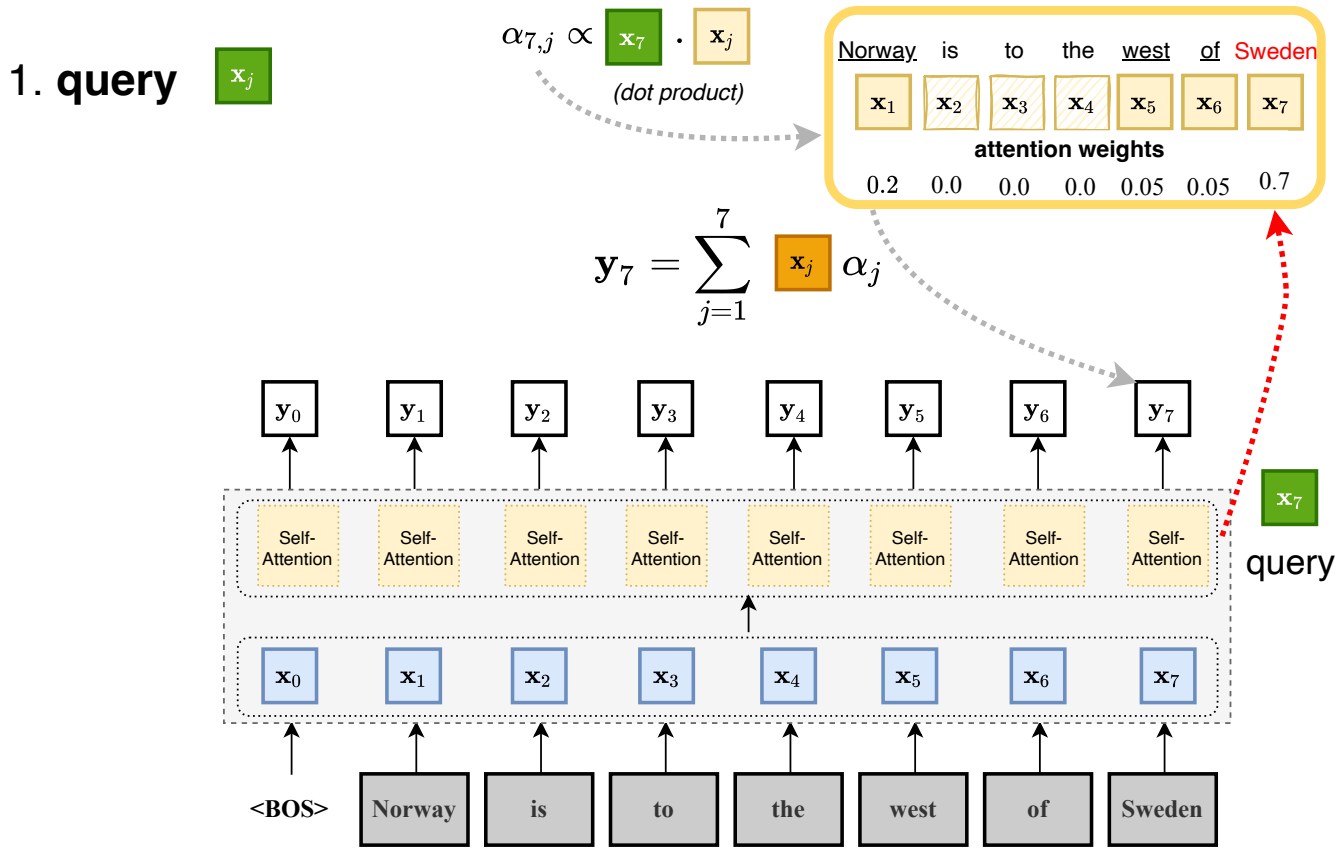
More Parameters: Keys, Queries and Values

- Three components in our computation.



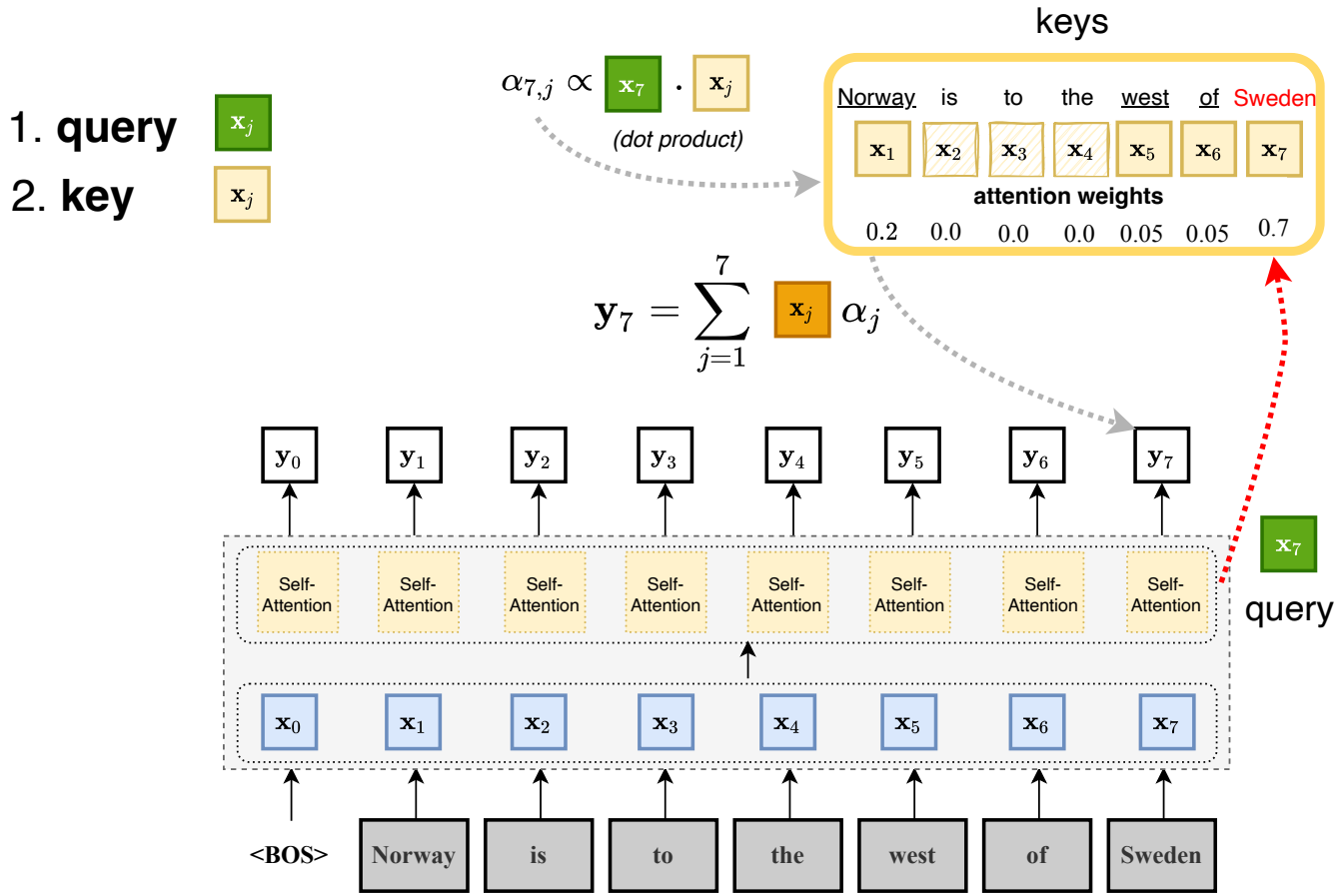
More Parameters: Keys, Queries and Values

- Three components in our computation.



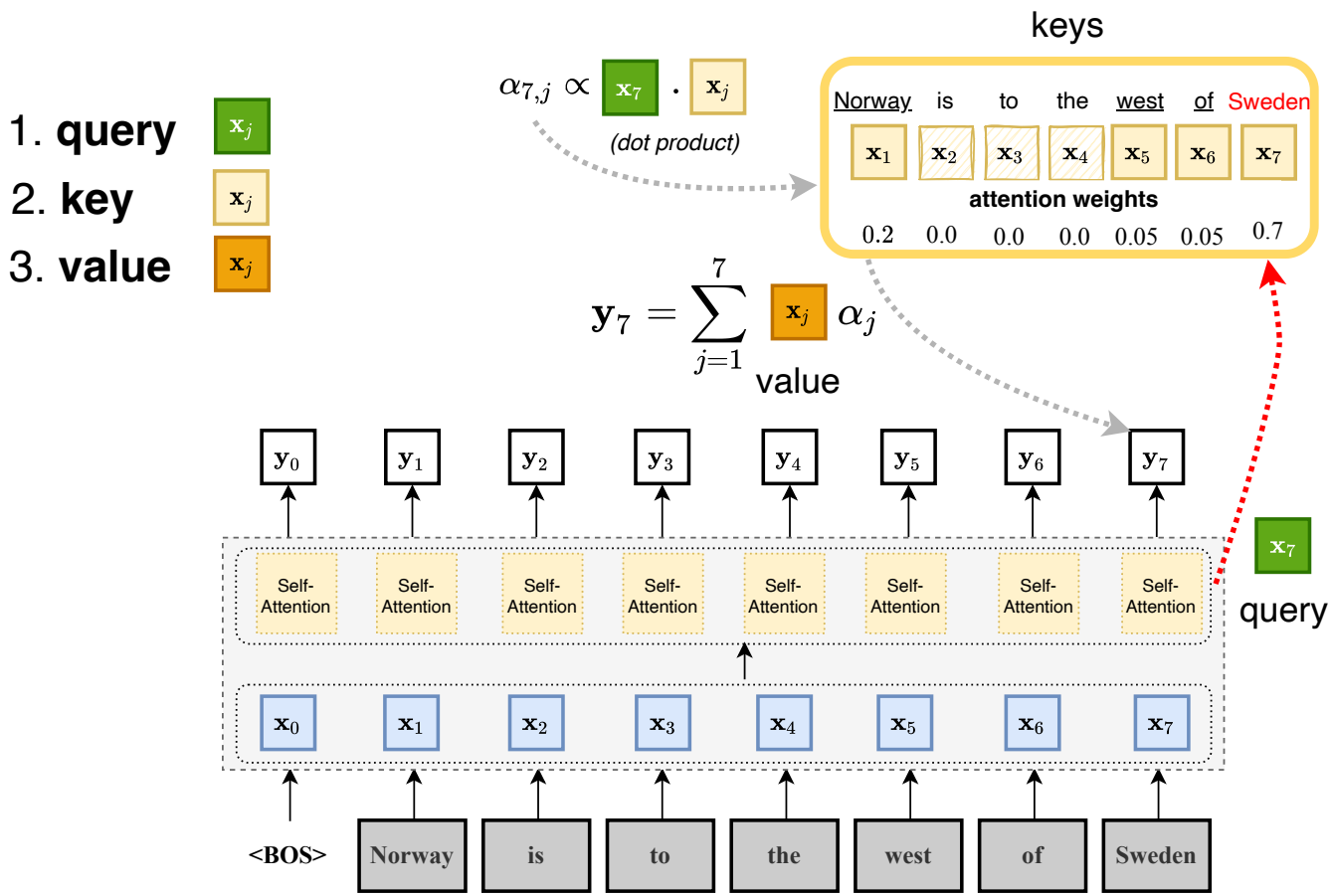
More Parameters: Keys, Queries and Values

- Three components in our computation.



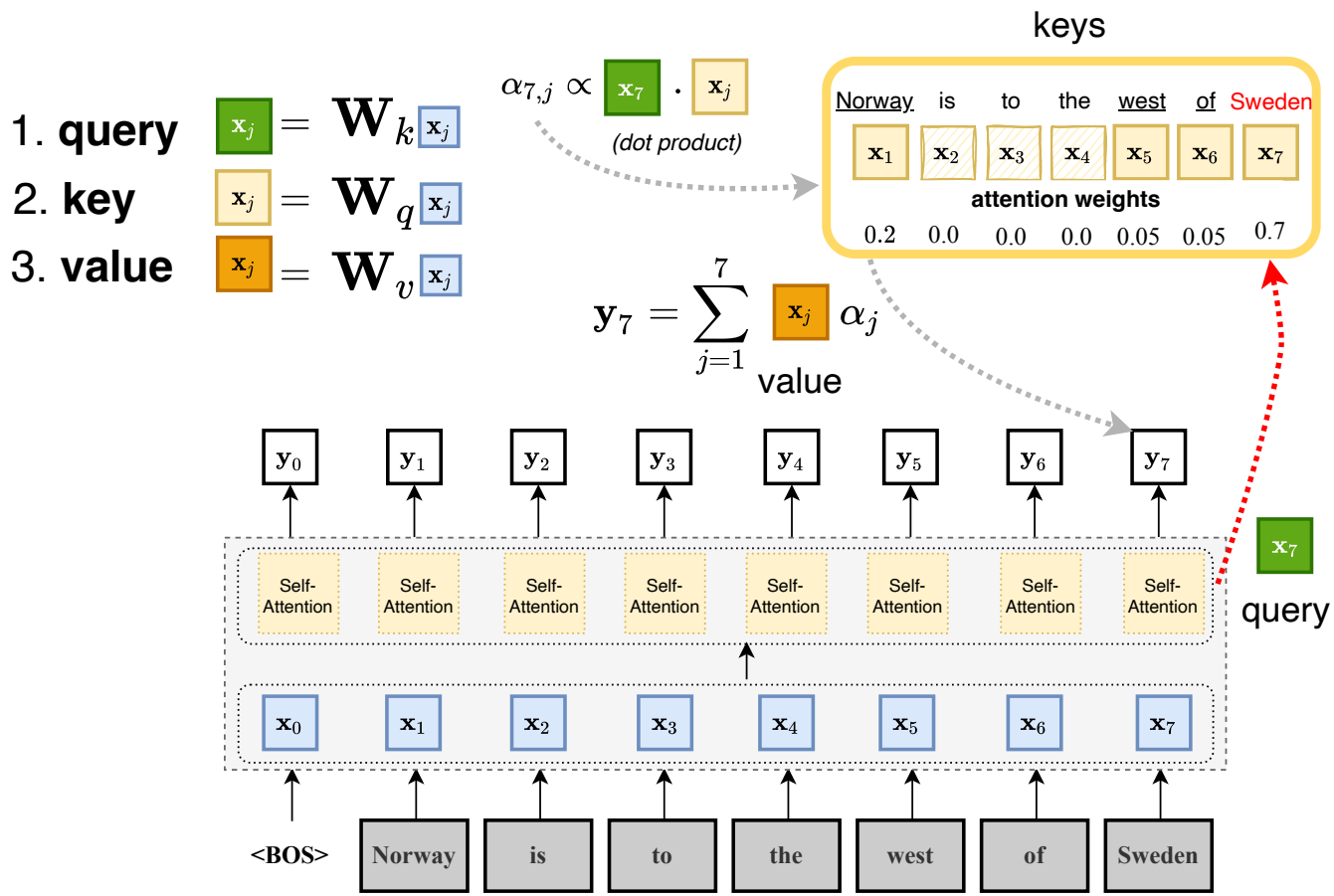
More Parameters: Keys, Queries and Values

- Three components in our computation.



More Parameters: Keys, Queries and Values

- Three components in our computation.



More Parameters: Keys, Queries and Values

Another few additional lines of PyTorch

Keys, queries and values in PyTorch

```
## Input representations (again),
X = E(torch.tensor([0,1,2,3,4,5,6,7]))
D = 768

### key, value, query parameters (linear layer)
W_k   = torch.nn.Linear(D, D, bias=False)
W_q   = torch.nn.Linear(D, D, bias=False)
W_v   = torch.nn.Linear(D, D, bias=False)

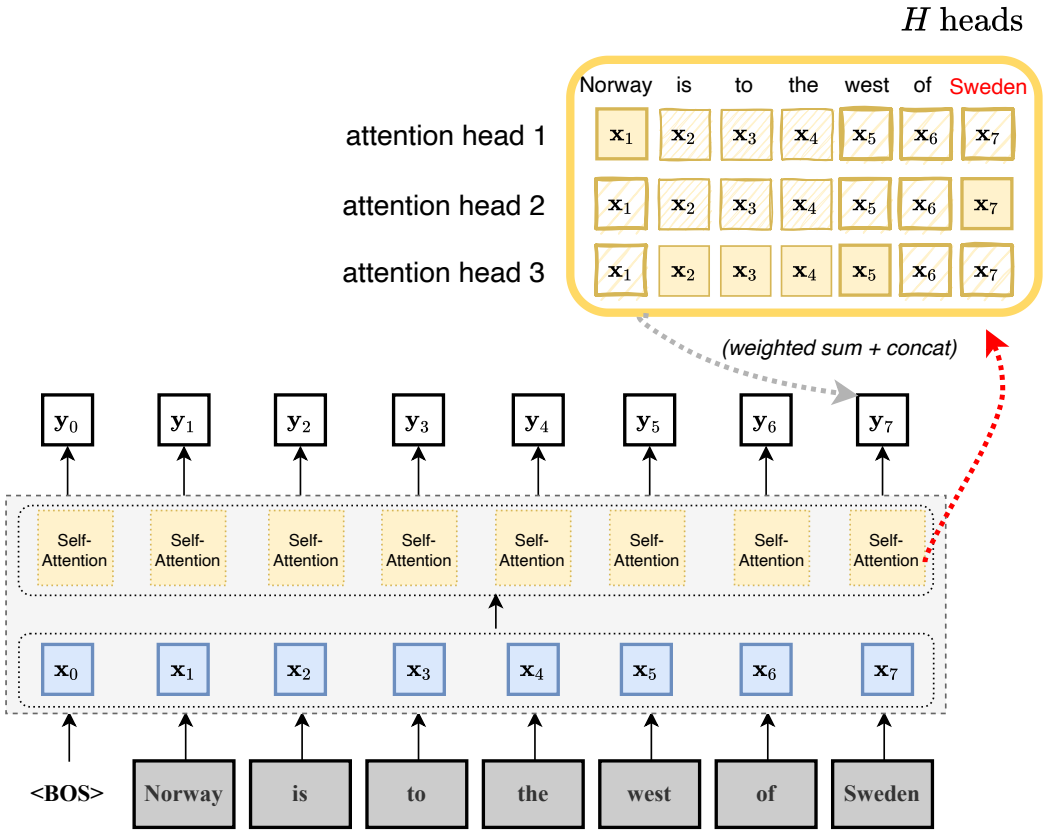
key_rep    = W_k(X)  #<-- rep. of X as keys
query_rep  = W_q(X) #<-- rep. of X as queries
value_rep  = W_v(X) #<-- rep of X as values

### sample computation with parameters applied over `X`
alpha = torch.matmul(
   query_rep,
   key_rep.transpose(0,1)
).softmax(dim=-1)

### same as before
Y = torch.matmul(alpha,value_rep)
```

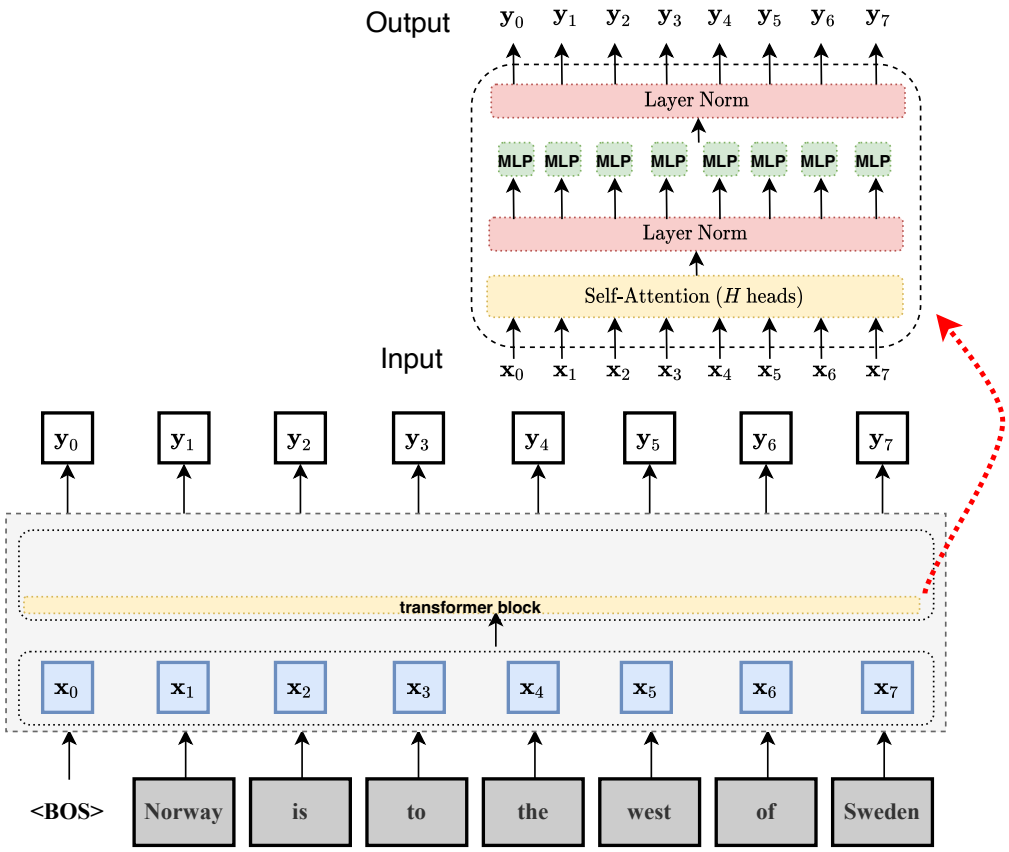
Multi-headed Attention

- Allows the model to simultaneously focus on multiple parts of input.



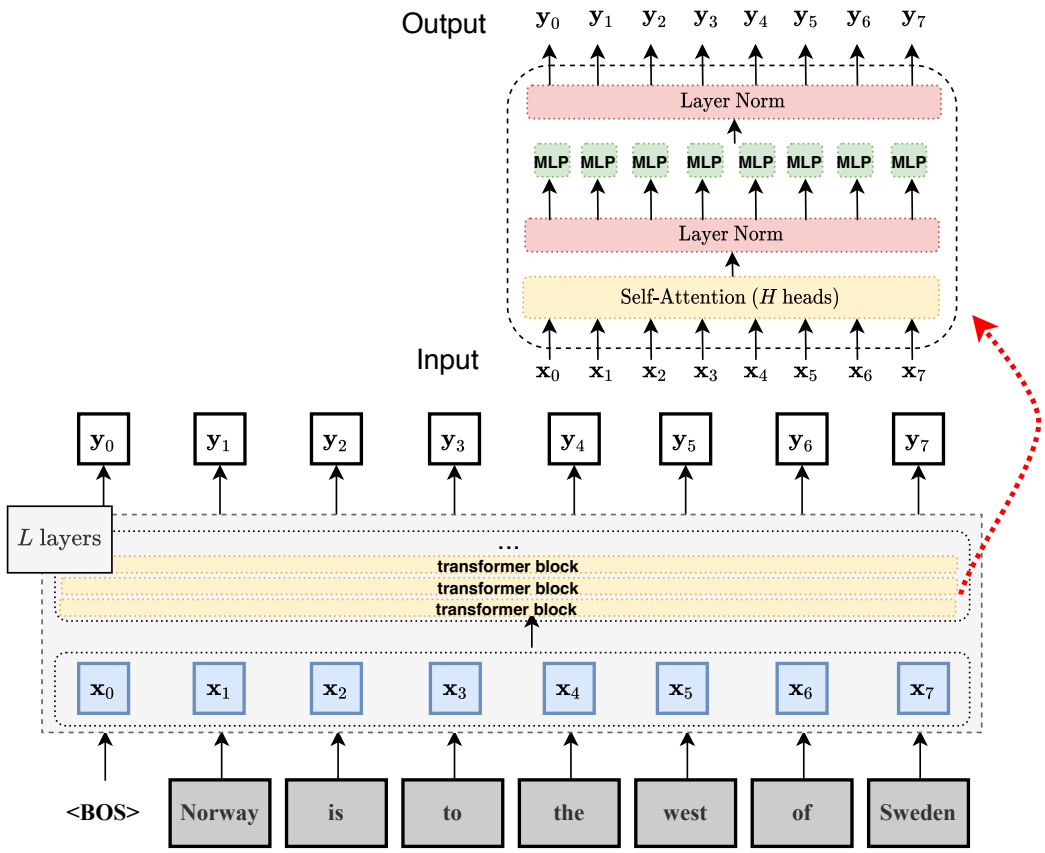
Transformer Blocks and Multiple Layers

- Current models are a bit more complex and multi-layered.



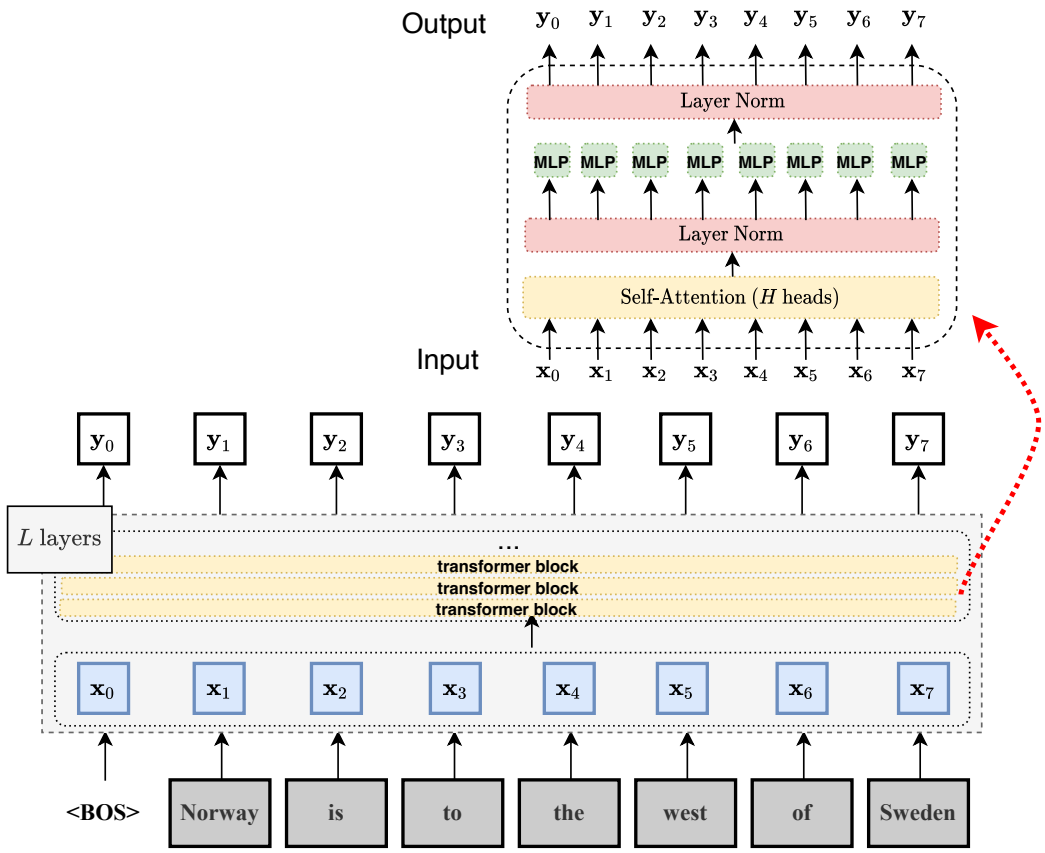
Transformer Blocks and Multiple Layers

- Current models are a bit more complex and multi-layered.



Transformer Blocks and Multiple Layers

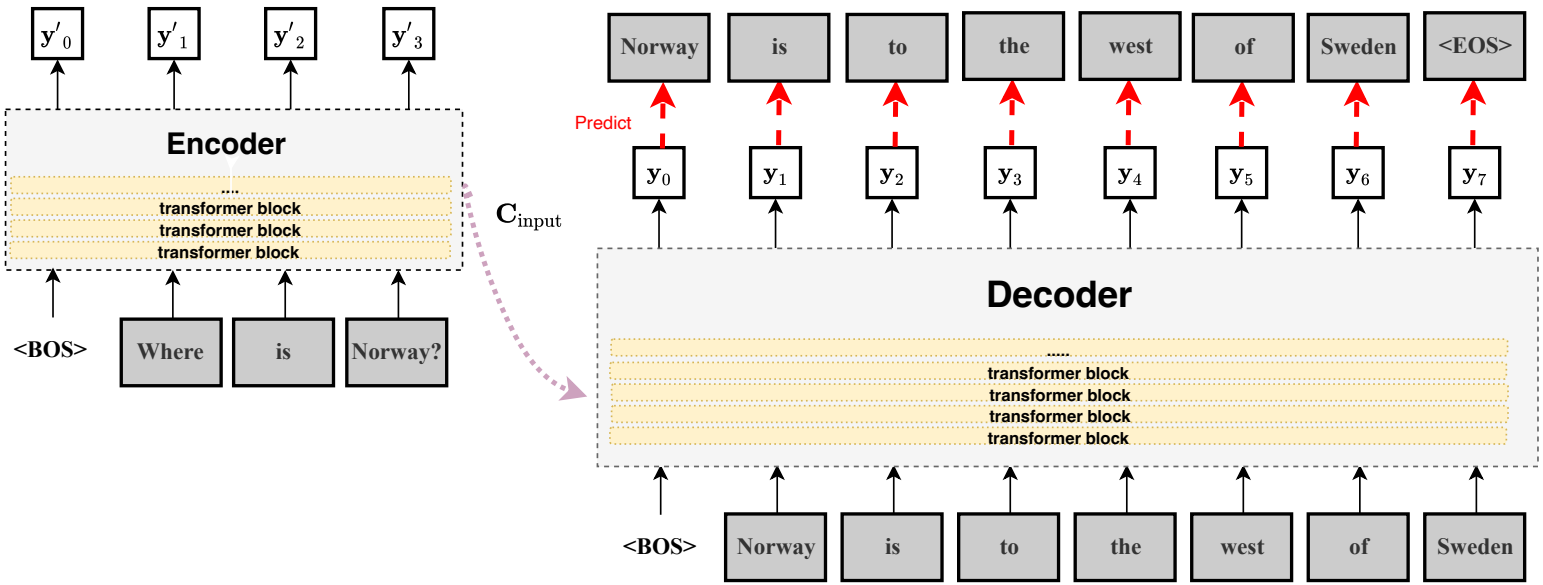
- Current models are a bit more complex and multi-layered.



BERT (Devlin et al.2018): $L = 24$ layers each with $H = 16$ heads, *340M* parameters, embedding dimension=1024

Transformer Blocks and Multiple Layers

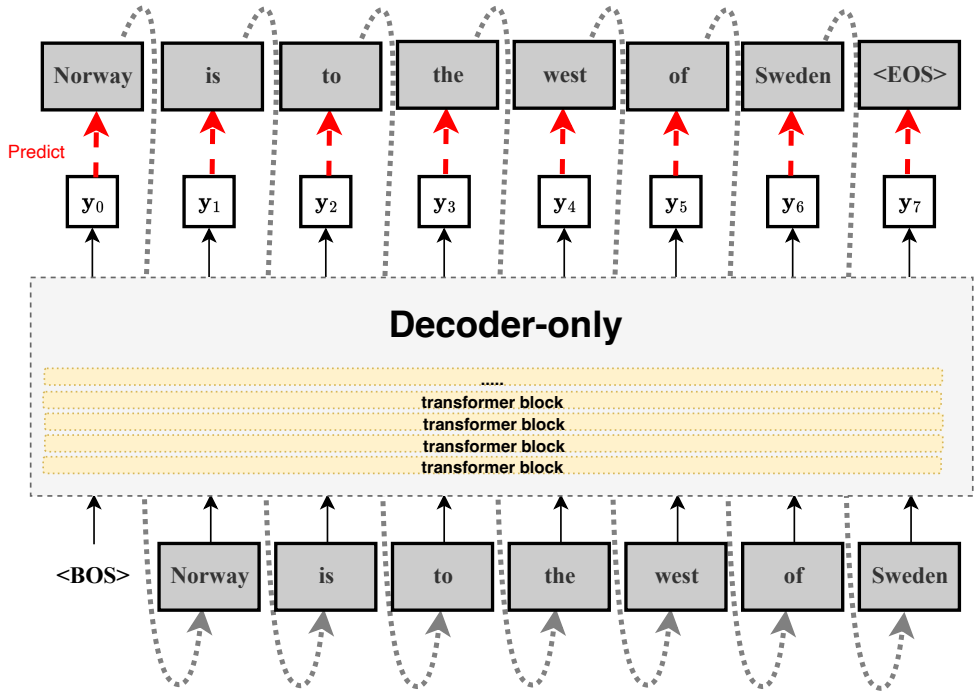
- Current models are a bit more complex and multi-layered.



T5 model (Raffel et al.2020), text2text architecture: **encoder** and **decoder**, 24 layers, 128 heads, cross-attention (11B parameters).

Transformer Blocks and Multiple Layers

- Current models are a bit more complex and multi-layered.



GPT3 (Brown et al.2020): $L = 96$ layers each with $H = 96$ heads (*175B parameters*), embedding dimension=12288

A transformer block (nanoGPT)

```
## nanoGPT: https://github.com/karpathy/nanoGPT
class Block(nn.Module):

    def __init__(self, config):
        super().__init__()
        self.ln_1 = LayerNorm(config.n_embd, bias=config.bias)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = LayerNorm(config.n_embd, bias=config.bias)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(self.ln_1(x))
        x = x + self.mlp(self.ln_2(x))
        return x
```

A transformer model: a stack of blocks

```
## sketch, simplified from nanoGPT
class GPT(nn.Module):
    def __init__(self, config):
        super().__init__()
        V, d, L = config.vocab_size, config.n_embd, config.n_layer
        self.wte = nn.Embedding(V, d)                    ## word embeddings
        self.wpe = nn.Embedding(config.block_size, d)  ## positions
        self.blocks = nn.ModuleList(
            [Block(config) for _ in range(L)])           ## L layers
        self.ln_f = nn.LayerNorm(d)
        self.lm_head = nn.Linear(d, V, bias=False)

    def forward(self, idx):
        pos = torch.arange(idx.size(1))
        x = self.wte(idx) + self.wpe(pos)   ## input representations
        for block in self.blocks:           ## layer after layer
            x = block(x)
        return self.lm_head(self.ln_f(x))   ## scores for the next word
```

Go through this yourself

Interim summary

1. **Language models**

$p(w_1, \ldots, w_m) = \prod_j p(w_j \mid w_{<j})$

score sequences; generate text

Interim summary

1. **Language models**

$p(w_1, \ldots, w_m) = \prod_j p(w_j \mid w_{<j})$

score sequences; generate text

2. **Classical models**

words are **discrete** symbols

$\approx 10^{15}$ parameters; **not** faithful

Interim summary

1. **Language models**

$p(w_1, \ldots, w_m) = \prod_j p(w_j \mid w_{<j})$

score sequences; generate text

2. **Classical models**

words are **discrete** symbols

$\approx 10^{15}$ parameters; **not** faithful

3. **Deep learning models**

words are **vectors** $\mathbf{x} \in \mathbb{R}^d$

full context: a **faithful** joint

Interim summary

1. Language models

$p(w_1, \ldots, w_m) = \prod_j p(w_j \mid w_{<j})$

score sequences; generate text

2. Classical models

words are **discrete** symbols

$\approx 10^{15}$ parameters; **not** faithful

3. Deep learning models

words are **vectors** $\mathbf{x} \in \mathbb{R}^d$

full context: a **faithful** joint

4. Contextual models

$\mathbf{y}_j$: meaning of w_j *in context*

$p(w_j \mid w_{<j}) = p(w_j \mid \mathbf{c}_{j-1})$

Interim summary

1. Language models

$p(w_1, \ldots, w_m) = \prod_j p(w_j \mid w_{<j})$

score sequences; generate text

2. Classical models

words are **discrete** symbols

$\approx 10^{15}$ parameters; **not** faithful

3. Deep learning models

words are **vectors** $\mathbf{x} \in \mathbb{R}^d$

full context: a **faithful** joint

4. Contextual models

$\mathbf{y}_j$: meaning of w_j *in context*

$p(w_j \mid w_{<j}) = p(w_j \mid \mathbf{c}_{j-1})$

5. Attention

$\mathbf{y}_i = \sum_j \alpha_{i,j} \mathbf{x}_j$

keys, queries, values; many heads

Interim summary

1. Language models

$p(w_1, \ldots, w_m) = \prod_j p(w_j \mid w_{<j})$

score sequences; generate text

2. Classical models

words are **discrete** symbols

$\approx 10^{15}$ parameters; **not** faithful

3. Deep learning models

words are **vectors** $\mathbf{x} \in \mathbb{R}^d$

full context: a **faithful** joint

4. Contextual models

$\mathbf{y}_j$: meaning of w_j *in context*

$p(w_j \mid w_{<j}) = p(w_j \mid \mathbf{c}_{j-1})$

5. Attention

$\mathbf{y}_i = \sum_j \alpha_{i,j} \mathbf{x}_j$

keys, queries, values; many heads

6. Encoders vs. decoders

encoder: **all**, decoder: **past**

causal mask: decoders generate

Questions?

Quick check: true or false?

1. $\prod_j p(w_j \mid w_{<j})$ assumes independence
2. a bigram model uses $p(w_j \mid w_{j-1})$
3. 100k words, 2 context words: 10^{15} params
4. an attention weight $\alpha_{i,j}$ can be negative
5. no positions: attention ignores word order
6. in a decoder, $\mathbf{y}_3$ can depend on w_5
7. encoders (BERT) generate word by word

Quick check: true or false?

1. $\prod_j p(w_j \mid w_{<j})$ assumes independence **false** exact (chain rule)
2. a bigram model uses $p(w_j \mid w_{j-1})$
3. 100k words, 2 context words: 10^{15} params
4. an attention weight $\alpha_{i,j}$ can be negative
5. no positions: attention ignores word order
6. in a decoder, $\mathbf{y}_3$ can depend on w_5
7. encoders (BERT) generate word by word

Quick check: true or false?

1. $\prod_j p(w_j \mid w_{<j})$ assumes independence — **false** — exact (chain rule)
2. a bigram model uses $p(w_j \mid w_{j-1})$ — **true** — Markov assumption
3. 100k words, 2 context words: 10^{15} params
4. an attention weight $\alpha_{i,j}$ can be negative
5. no positions: attention ignores word order
6. in a decoder, $\mathbf{y}_3$ can depend on w_5
7. encoders (BERT) generate word by word

Quick check: true or false?

1.	$\prod_j p(w_j \mid w_{<j})$ assumes independence	**false**	exact (chain rule)
2.	a bigram model uses $p(w_j \mid w_{j-1})$	**true**	Markov assumption
3.	100k words, 2 context words: 10^{15} params	**true**	$(10^5)^3$
4.	an attention weight $\alpha_{i,j}$ can be negative		
5.	no positions: attention ignores word order		
6.	in a decoder, $\mathbf{y}_3$ can depend on w_5		
7.	encoders (BERT) generate word by word		

Quick check: true or false?

1.	$\prod_j p(w_j \mid w_{<j})$ assumes independence	**false**	exact (chain rule)
2.	a bigram model uses $p(w_j \mid w_{j-1})$	**true**	Markov assumption
3.	100k words, 2 context words: 10^{15} params	**true**	$(10^5)^3$
4.	an attention weight $\alpha_{i,j}$ can be negative	**false**	softmax: positive
5.	no positions: attention ignores word order		
6.	in a decoder, $\mathbf{y}_3$ can depend on w_5		
7.	encoders (BERT) generate word by word		

Quick check: true or false?

1.	$\prod_j p(w_j \mid w_{<j})$ assumes independence	**false**	exact (chain rule)
2.	a bigram model uses $p(w_j \mid w_{j-1})$	**true**	Markov assumption
3.	100k words, 2 context words: 10^{15} params	**true**	$(10^5)^3$
4.	an attention weight $\alpha_{i,j}$ can be negative	**false**	softmax: positive
5.	no positions: attention ignores word order	**true**	a weighted sum
6.	in a decoder, $\mathbf{y}_3$ can depend on w_5		
7.	encoders (BERT) generate word by word		

Quick check: true or false?

1.	$\prod_j p(w_j \mid w_{<j})$ assumes independence	**false**	exact (chain rule)
2.	a bigram model uses $p(w_j \mid w_{j-1})$	**true**	Markov assumption
3.	100k words, 2 context words: 10^{15} params	**true**	$(10^5)^3$
4.	an attention weight $\alpha_{i,j}$ can be negative	**false**	softmax: positive
5.	no positions: attention ignores word order	**true**	a weighted sum
6.	in a decoder, $\mathbf{y}_3$ can depend on w_5	**false**	causal mask
7.	encoders (BERT) generate word by word		

Quick check: true or false?

1.	$\prod_j p(w_j \mid w_{<j})$ assumes independence	**false**	exact (chain rule)
2.	a bigram model uses $p(w_j \mid w_{j-1})$	**true**	Markov assumption
3.	100k words, 2 context words: 10^{15} params	**true**	$(10^5)^3$
4.	an attention weight $\alpha_{i,j}$ can be negative	**false**	softmax: positive
5.	no positions: attention ignores word order	**true**	a weighted sum
6.	in a decoder, $\mathbf{y}_3$ can depend on w_5	**false**	causal mask
7.	encoders (BERT) generate word by word	**false**	decoders (GPT) do

Quick check: causal masking

8. causal mask: $\alpha_{i,j} = 0$ for all $j > i$
9. the mask is applied *after* the softmax
10. the first position attends only to itself
11. `triu_indices`, `offset=1`: pairs $j > i$
12. masking: train one position at a time
13. BERT-style encoders use a causal mask
14. no mask: $\mathbf{y}_j$ could just copy w_{j+1}

Quick check: causal masking

8. causal mask: $\alpha_{i,j} = 0$ for all $j > i$ **true** $e^{-\infty} = 0$
9. the mask is applied *after* the softmax
10. the first position attends only to itself
11. `triu_indices`, `offset=1`: pairs $j > i$
12. masking: train one position at a time
13. BERT-style encoders use a causal mask
14. no mask: $\mathbf{y}_j$ could just copy w_{j+1}

Quick check: causal masking

8. causal mask: $\alpha_{i,j} = 0$ for all $j > i$ — **true** — $e^{-\infty} = 0$
9. the mask is applied *after* the softmax — **false** — before it
10. the first position attends only to itself
11. `triu_indices`, `offset=1`: pairs $j > i$
12. masking: train one position at a time
13. BERT-style encoders use a causal mask
14. no mask: $\mathbf{y}_j$ could just copy w_{j+1}

Quick check: causal masking

8. causal mask: $\alpha_{i,j} = 0$ for all $j > i$ — **true** — $e^{-\infty} = 0$
9. the mask is applied *after* the softmax — **false** — before it
10. the first position attends only to itself — **true** — $\alpha_{1,1} = 1$
11. `triu_indices`, `offset=1`: pairs $j > i$
12. masking: train one position at a time
13. BERT-style encoders use a causal mask
14. no mask: $\mathbf{y}_j$ could just copy w_{j+1}

Quick check: causal masking

8.	causal mask: $\alpha_{i,j} = 0$ for all $j > i$	**true**	$e^{-\infty} = 0$
9.	the mask is applied *after* the softmax	**false**	before it
10.	the first position attends only to itself	**true**	$\alpha_{1,1} = 1$
11.	`triu_indices`, `offset=1`: pairs $j > i$	**true**	above diagonal
12.	masking: train one position at a time		
13.	BERT-style encoders use a causal mask		
14.	no mask: $\mathbf{y}_j$ could just copy w_{j+1}		

Quick check: causal masking

8. causal mask: $\alpha_{i,j} = 0$ for all $j > i$ — **true** — $e^{-\infty} = 0$
9. the mask is applied *after* the softmax — **false** — before it
10. the first position attends only to itself — **true** — $\alpha_{1,1} = 1$
11. `triu_indices`, `offset=1`: pairs $j > i$ — **true** — above diagonal
12. masking: train one position at a time — **false** — all in parallel
13. BERT-style encoders use a causal mask
14. no mask: $\mathbf{y}_j$ could just copy w_{j+1}

Quick check: causal masking

8.	causal mask: $\alpha_{i,j} = 0$ for all $j > i$	**true**	$e^{-\infty} = 0$
9.	the mask is applied *after* the softmax	**false**	before it
10.	the first position attends only to itself	**true**	$\alpha_{1,1} = 1$
11.	`triu_indices`, `offset=1`: pairs $j > i$	**true**	above diagonal
12.	masking: train one position at a time	**false**	all in parallel
13.	BERT-style encoders use a causal mask	**false**	they see all
14.	no mask: $\mathbf{y}_j$ could just copy w_{j+1}		

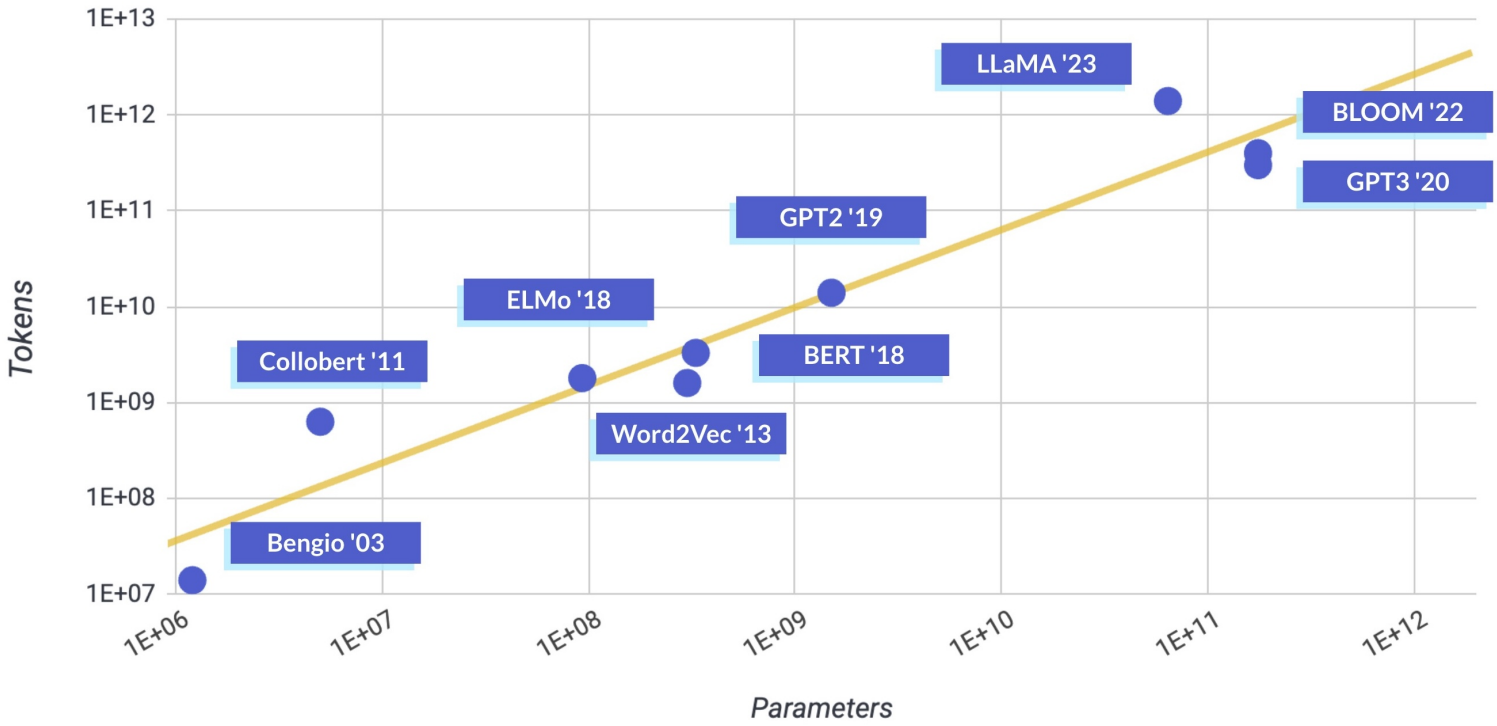
Quick check: causal masking

8.	causal mask: $\alpha_{i,j} = 0$ for all $j > i$	**true**	$e^{-\infty} = 0$
9.	the mask is applied *after* the softmax	**false**	before it
10.	the first position attends only to itself	**true**	$\alpha_{1,1} = 1$
11.	`triu_indices`, `offset=1`: pairs $j > i$	**true**	above diagonal
12.	masking: train one position at a time	**false**	all in parallel
13.	BERT-style encoders use a causal mask	**false**	they see all
14.	no mask: $\mathbf{y}_j$ could just copy w_{j+1}	**true**	it sees the answer

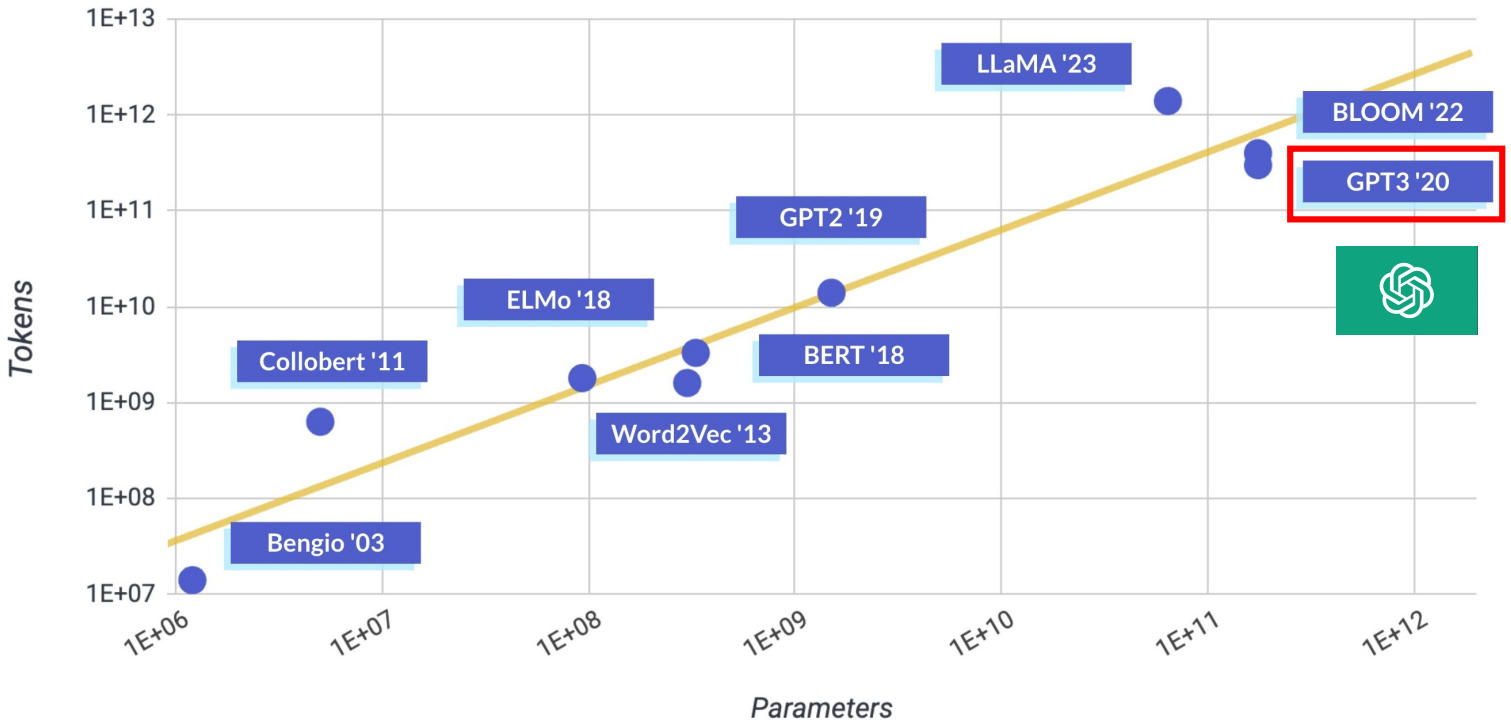
The kinds of questions on the exam

The nature of their training

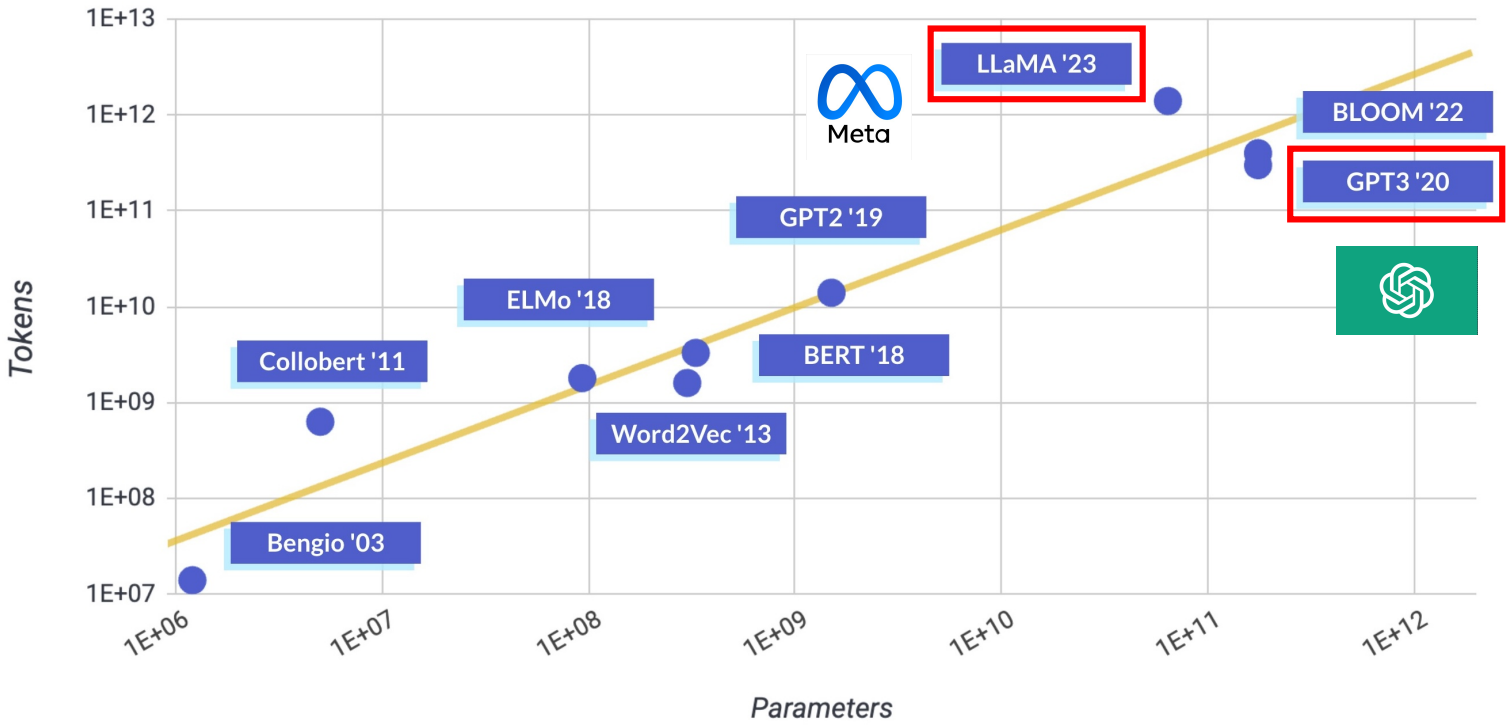
Model and data scale: a glance



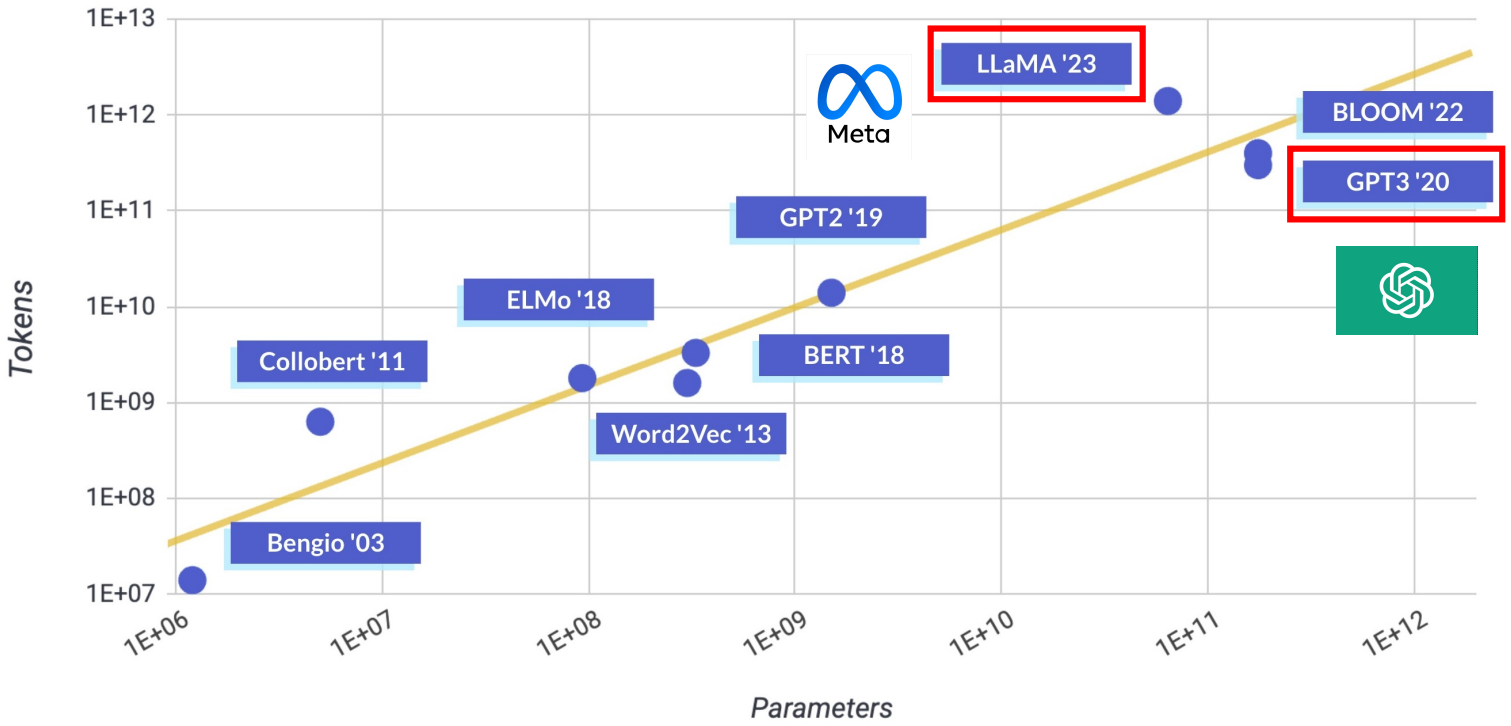
Model and data scale: a glance



Model and data scale: a glance



Model and data scale: a glance

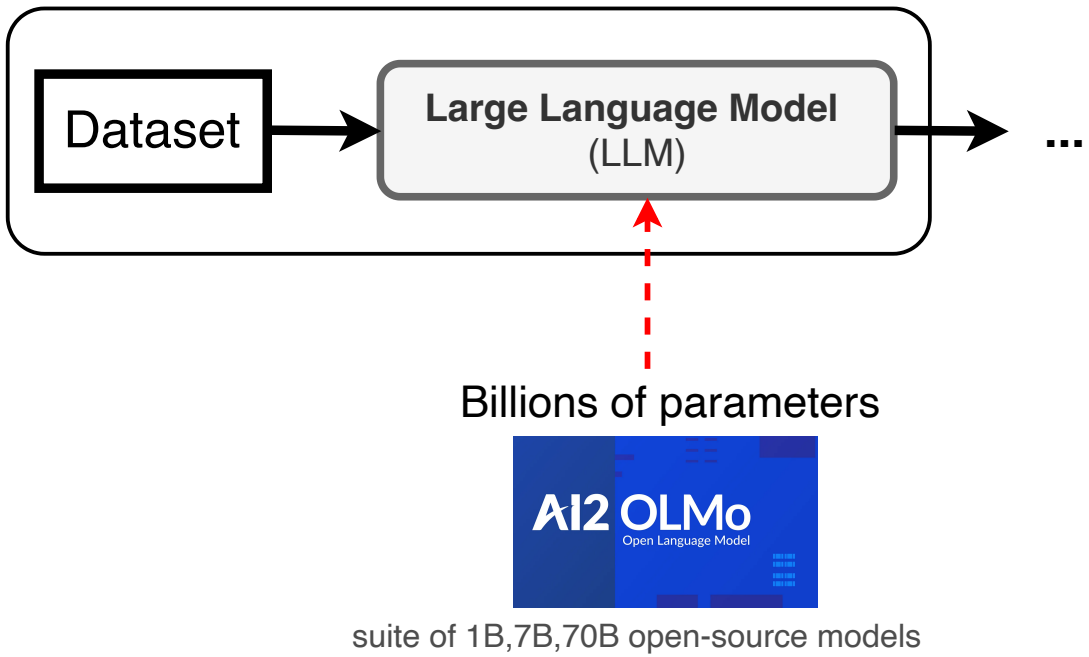


- Industry is currently dominating, we know increasingly less about what these models are and how they are trained.

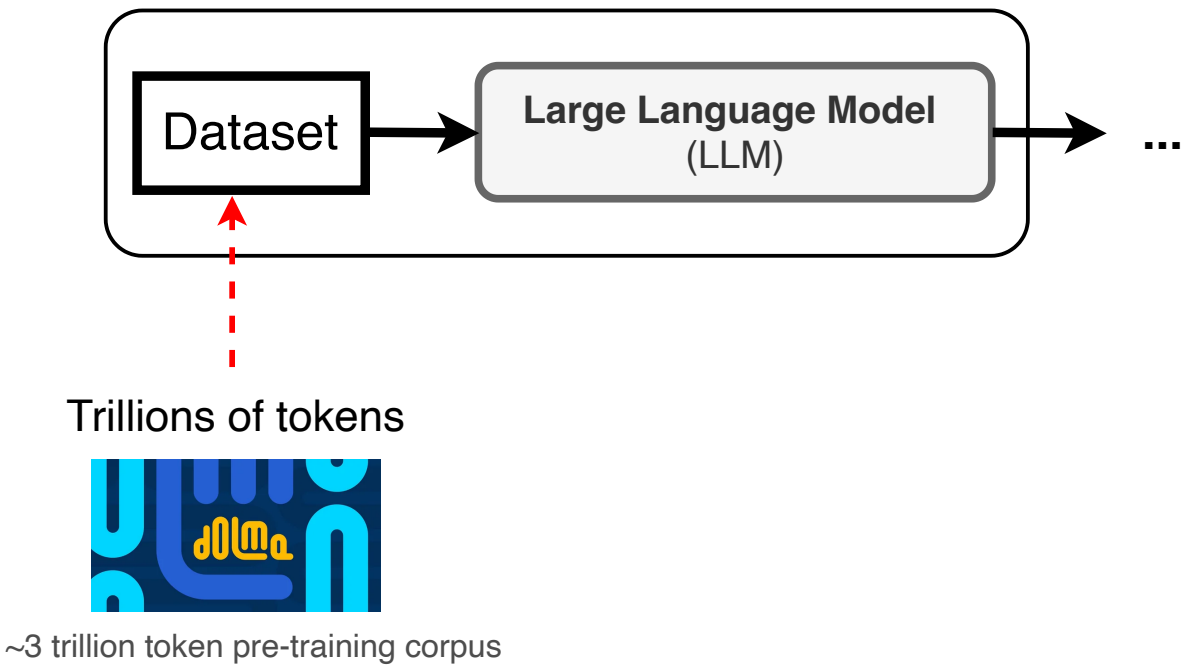
Open Language Model (OLMo) project at AI2



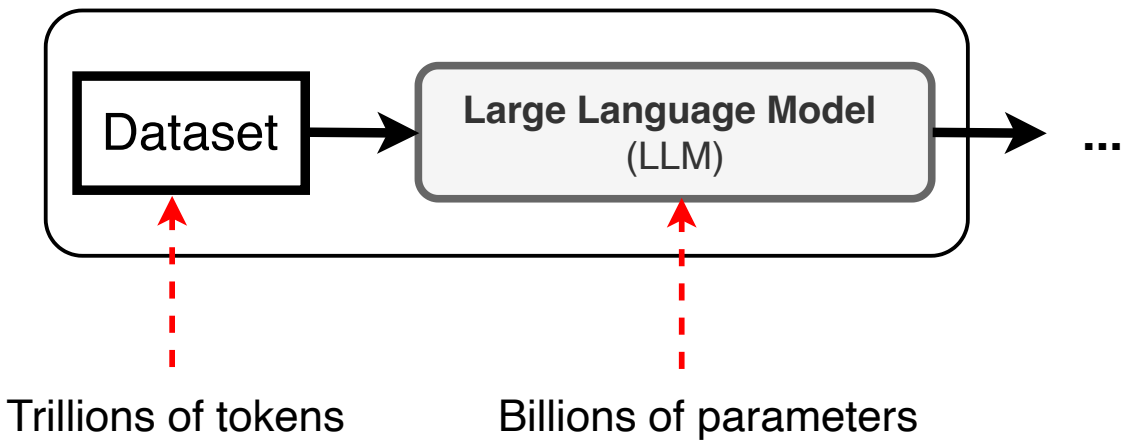
Open Language Model (OLMo) project at AI2



Open Language Model (OLMo) project at AI2



Open Language Model (OLMo) project at AI2



Where does the data come from?

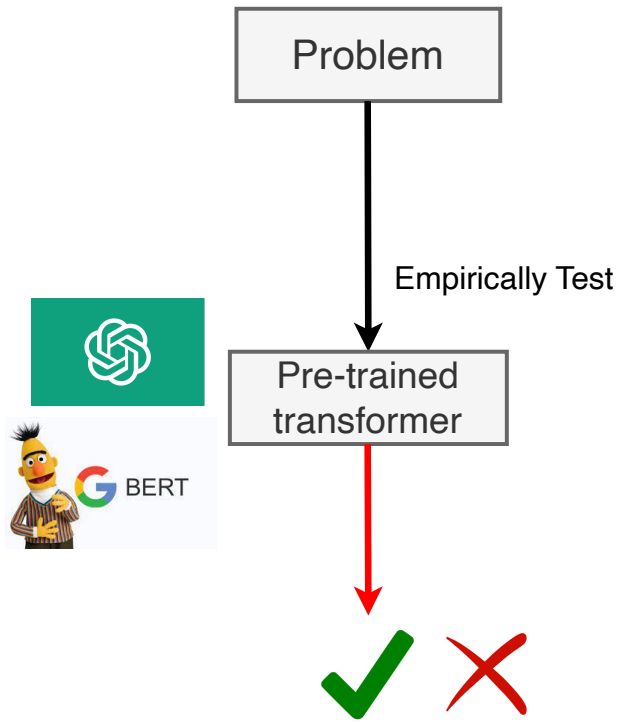
Doc Type	UTF-8 bytes *(GB)*	Documents *(millions)*	Unicode words *(billions)*
web pages	9,022	3,370	1,775
code	1,043	210	260
web pages	790	364	153
social media	339	377	72
STEM papers	268	38.8	50
books	20.4	0.056	4.0
encyclopedic	16.2	6.2	3.7
	11,519	**4,367**	**2,318**

What can models do? What do they know?

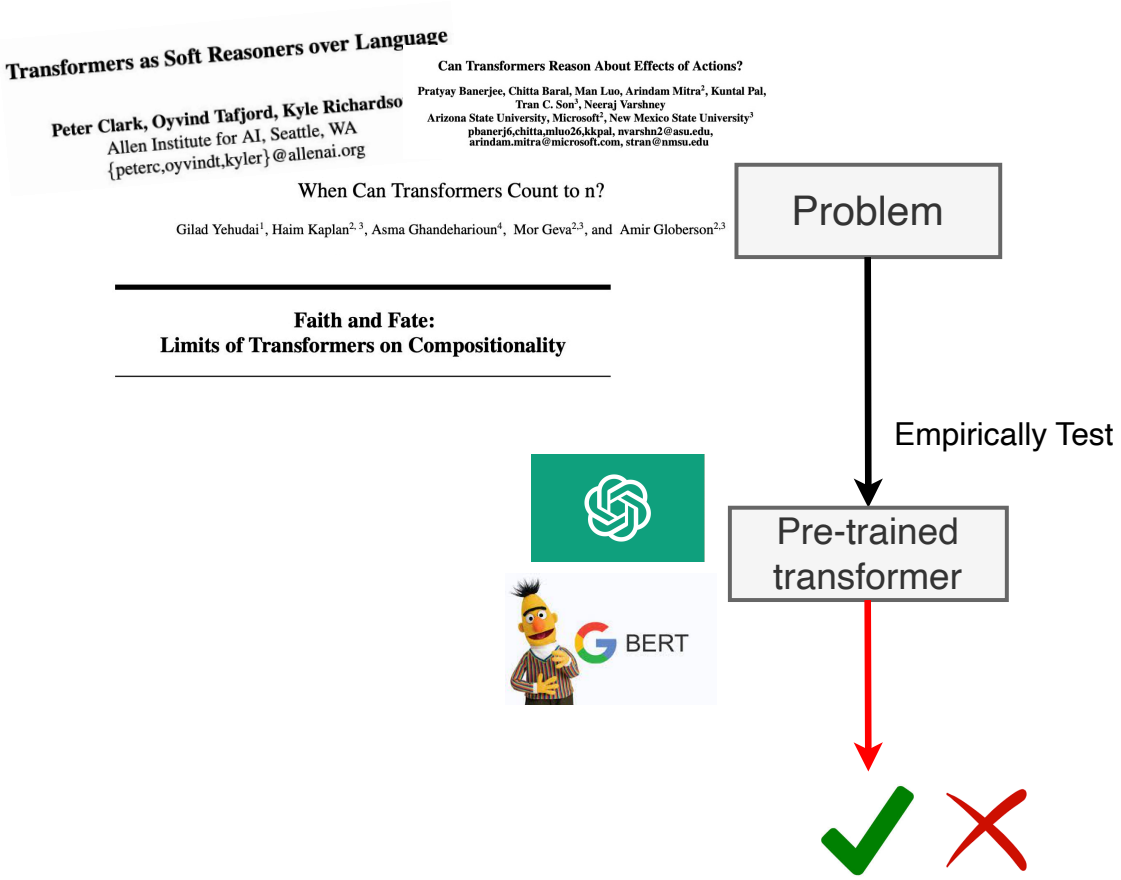
What can models do? Different approaches



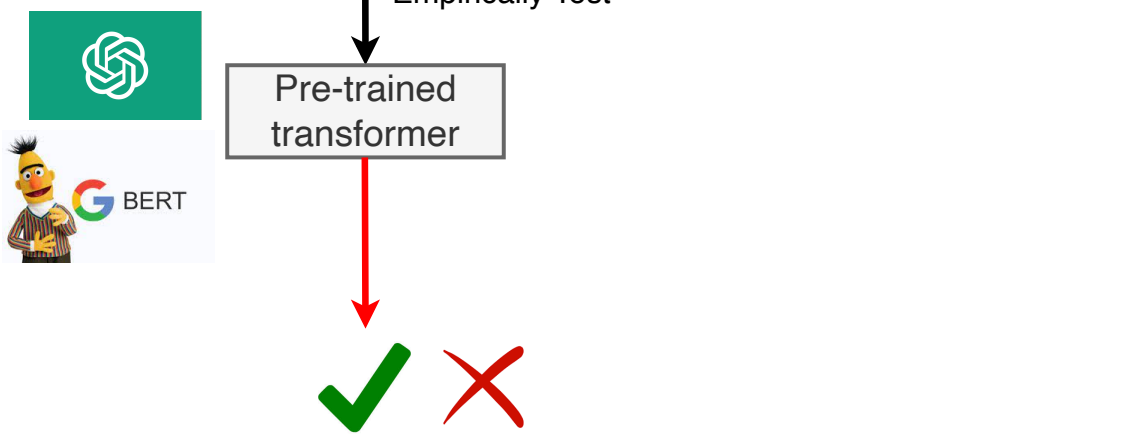
What can models do? Different approaches



What can models do? Different approaches



What can models do? Different approaches




```
def frac_prevs(sop,val){
  prevs = select(indices,indices,<=);
  return aggregate(prevs,
                   indicator(sop==val));
}

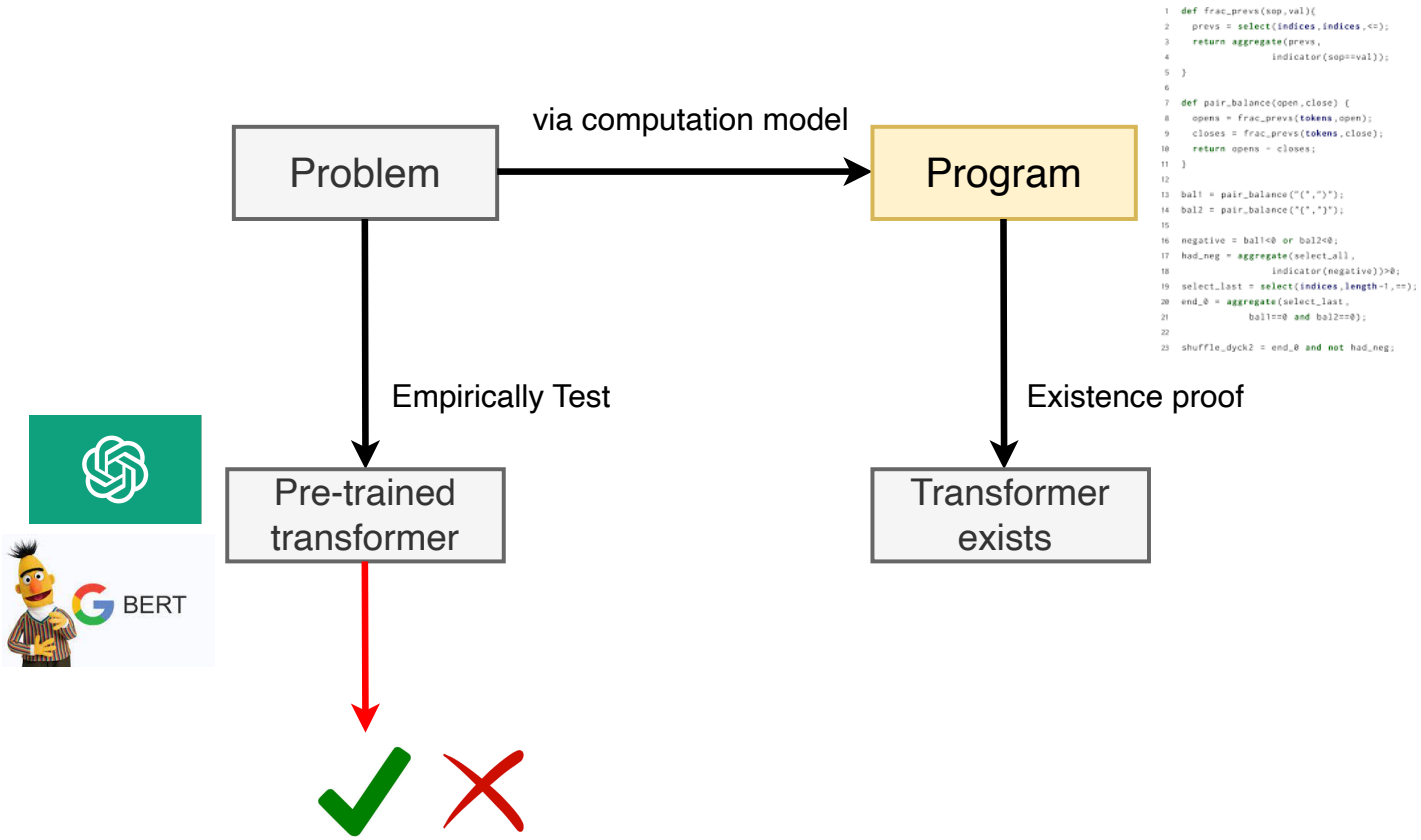
def pair_balance(open,close) {
  opens = frac_prevs(tokens,open);
  closes = frac_prevs(tokens,close);
  return opens > closes;
}

bal1 = pair_balance("(",")");
bal2 = pair_balance("[","]");

negative = bal1<0 or bal2<0;
had_neg = aggregate(select_all,
                    indicator(negative))>0;
select_last = select(indices,length-1,==);
end_0 = aggregate(select_last,
                  bal1==0 and bal2==0);

shuffle_dyck2 = end_0 and not had_neg;
```

What can models do? Different approaches



Thinking like a transformer

Thinking Like Transformers

Gail Weiss [1] **Yoav Goldberg** [2 3] **Eran Yahav** [1]

- `Rasp` (*Restricted Access Sequence Processing Language*): a symbolic functional language for expressing transformer computation.

Thinking like a transformer

```
class SimpleBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.attn = SelfAttention(config)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(x) #<--- looking around
        x = x + self.mlp(x) #<--- element-wise
        #   | residual
        return x
```

Thinking like a transformer

```
class SimpleBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.attn = SelfAttention(config)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(x) #<--- looking around
        x = x + self.mlp(x) #<--- element-wise
        #   | residual
        return x
```

Conditions on information processing

Thinking like a transformer

```
class SimpleBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.attn = SelfAttention(config)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(x) #<--- looking around
        x = x + self.mlp(x) #<--- element-wise
        #   | residual
        return x
```

Conditions on information processing

- Each transformation on x must produce an object of the same size

Thinking like a transformer

```
class SimpleBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.attn = SelfAttention(config)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(x) #<--- looking around
        x = x + self.mlp(x) #<--- element-wise
        #   | residual
        return x
```

Conditions on information processing

- Each transformation on x must produce an object of the same size
- Information across positions can only be shared via attention.

Thinking like a transformer

```
class SimpleBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.attn = SelfAttention(config)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(x) #<--- looking around
        x = x + self.mlp(x) #<--- element-wise
        #   | residual
        return x
```

Conditions on information processing

- Each transformation on x must produce an object of the same size
- Information across positions can only be shared via attention.
- Other transformations are element-wise, must have a way to pass around old information.

Thinking like a transformer

```
class SimpleBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.attn = SelfAttention(config)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(x) #<--- looking around
        x = x + self.mlp(x) #<--- thinking
        # ^ residual
        return x
```

Important details omitted!

Conditions on information processing

- Each transformation on x must produce an object of the same size
- Information across positions can only be shared via attention.
- Other transformations are element-wise, must have a way to pass around old information.

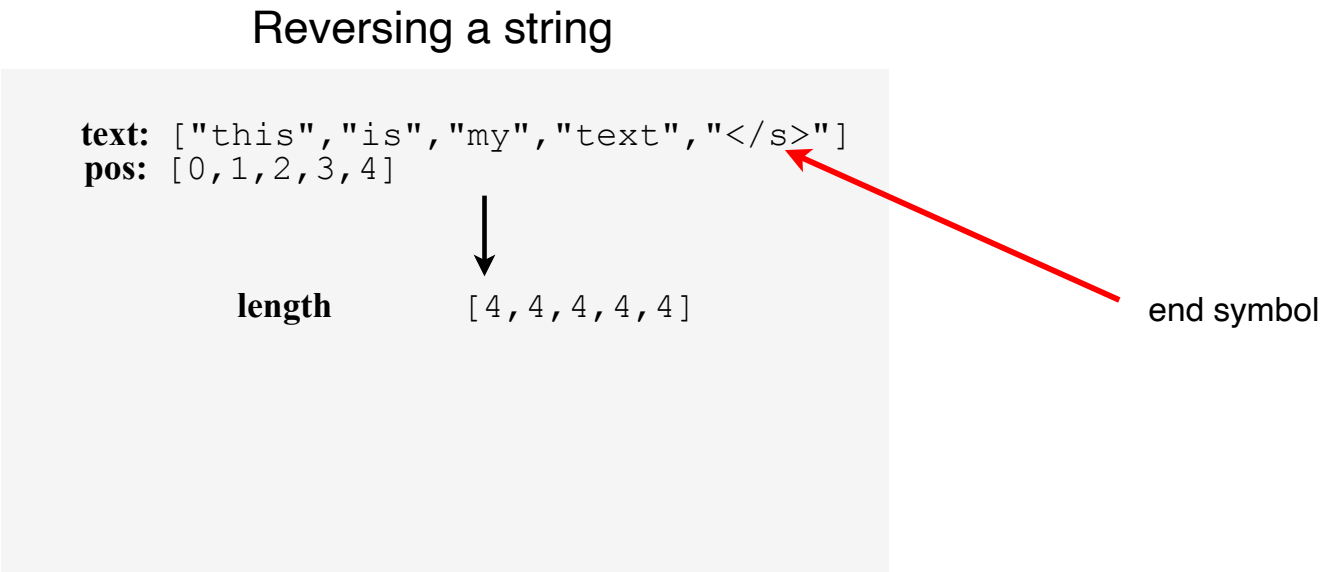
Thinking like a transformer: reversing a string

Reversing a string

text: ["this","is","my","text","</s>"]
pos: [0,1,2,3,4]

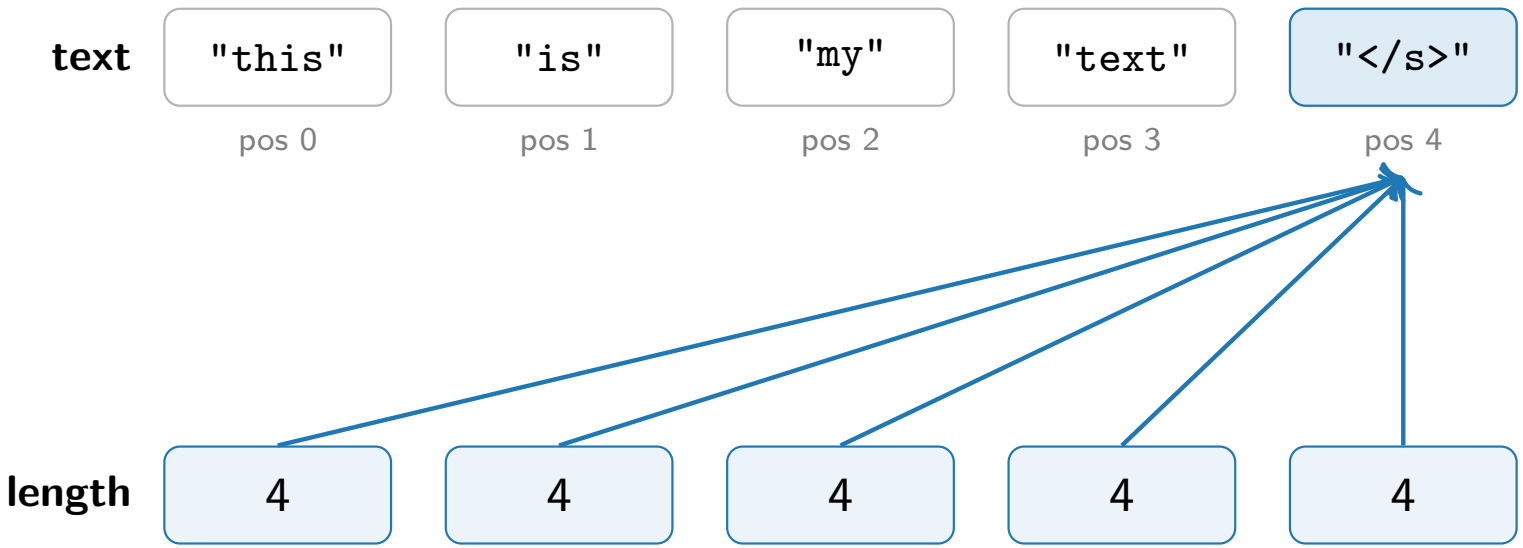
reverse ["<s>","text",...,"this]

Thinking like a transformer: reversing a string



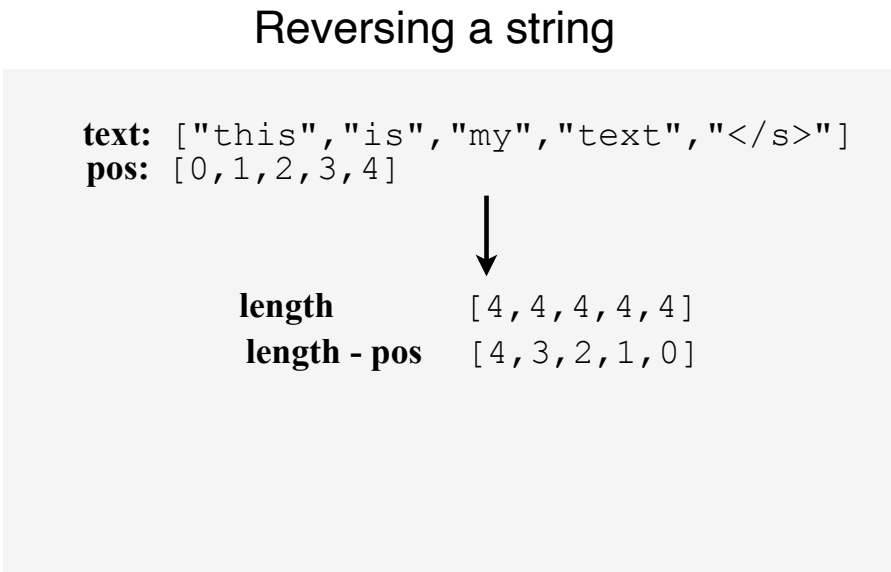
Thinking like a transformer: reversing a string

Step 1 (attention): every position looks at the end symbol

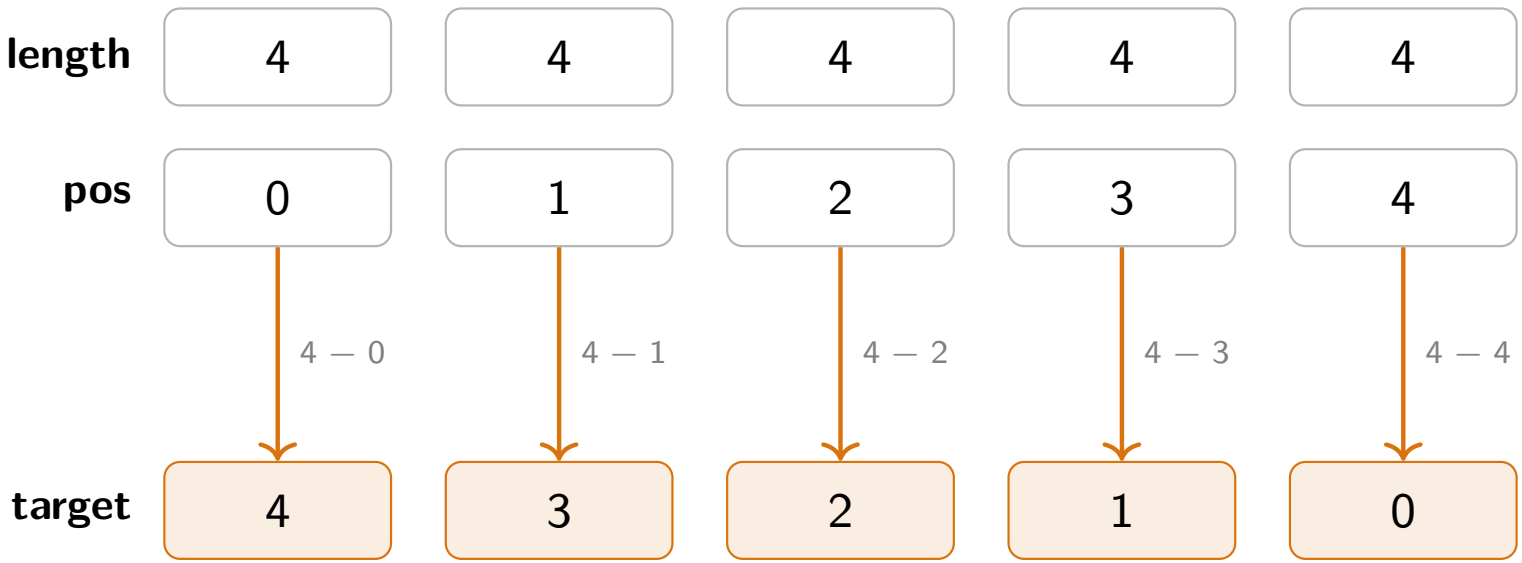


select the key "</s>", then copy its position:
every position now knows **length** = 4

Thinking like a transformer: reversing a string



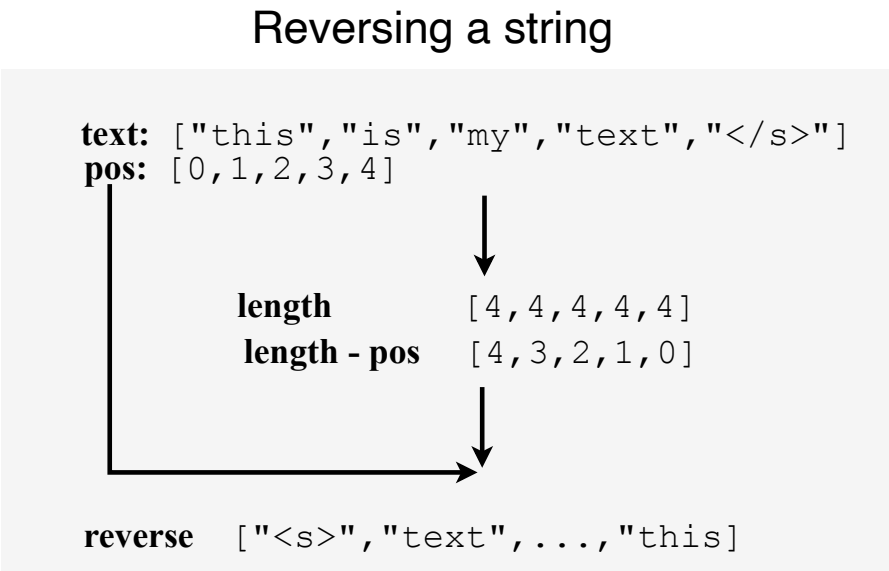
Step 2 (MLP): each position computes on its own



no information moves *between* positions (straight down arrows):

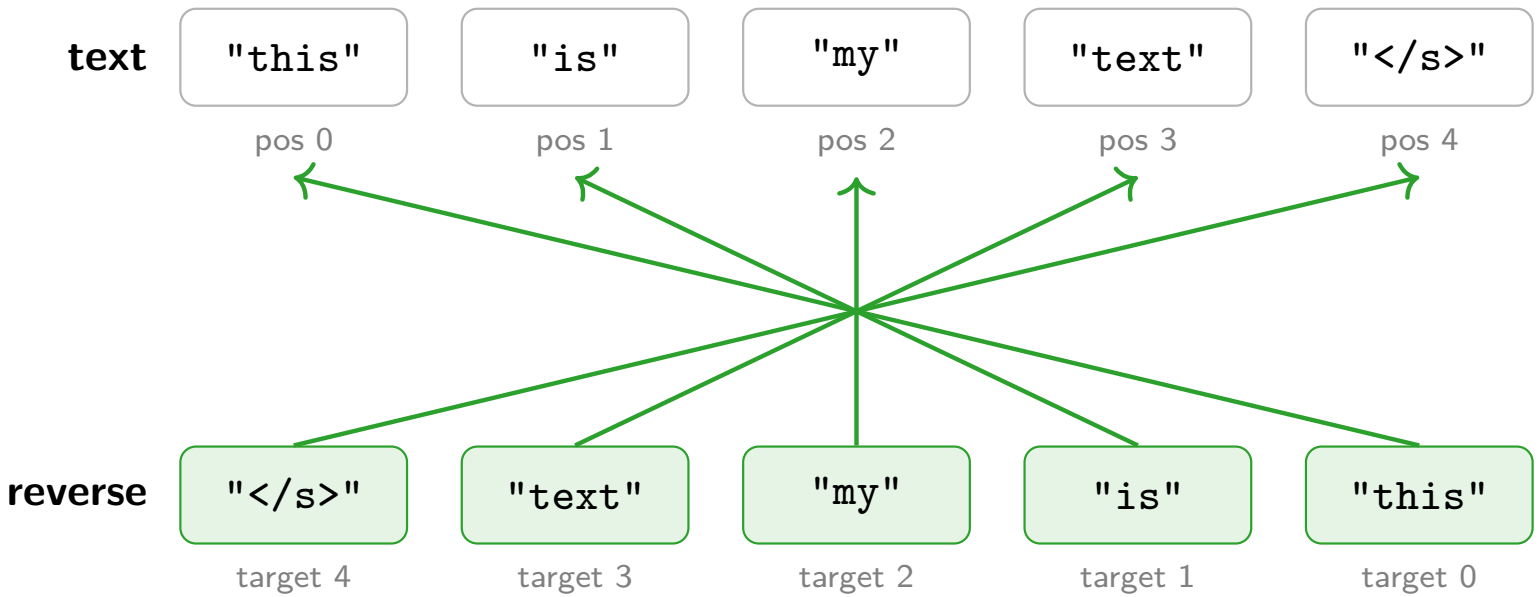
target = length − pos = $[4, 3, 2, 1, 0]$

Thinking like a transformer: reversing a string



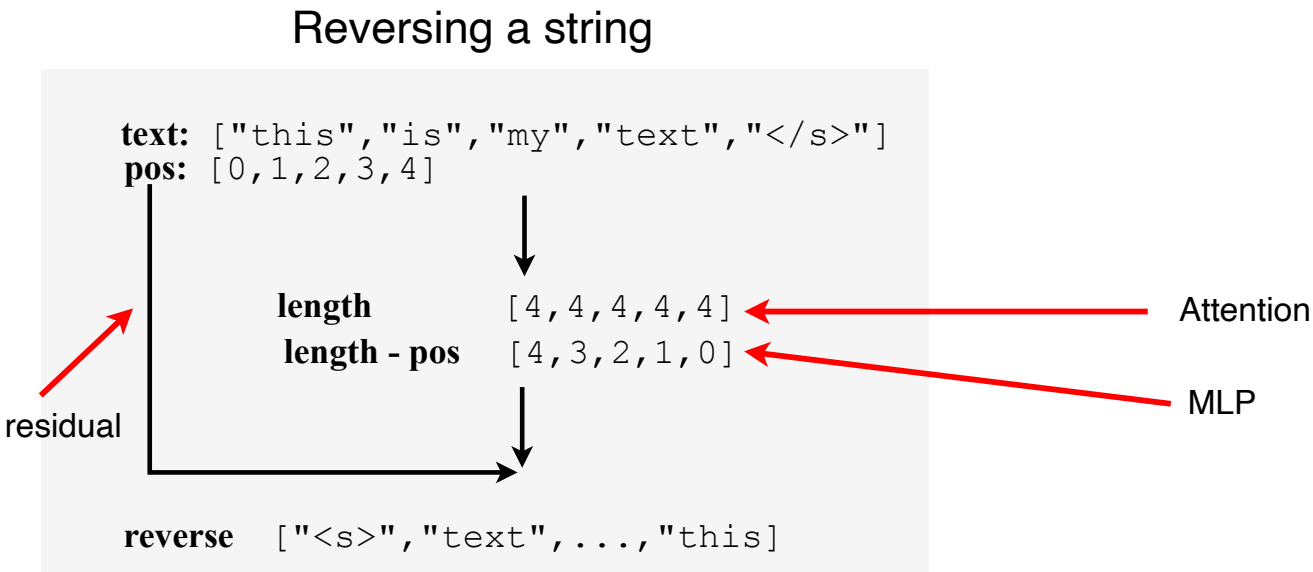
Step 3 (residual + attention): fetch the token at the target

the tokens and their positions are still there, carried by the **residual**

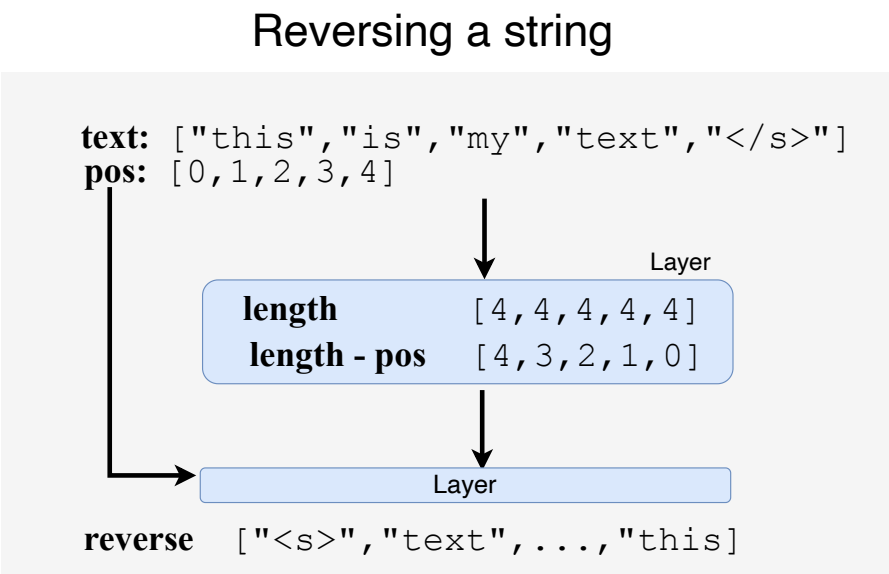


position i attends to the j with $\text{pos}_j = \text{target}_i$ and copies its token:
the **reversed** string

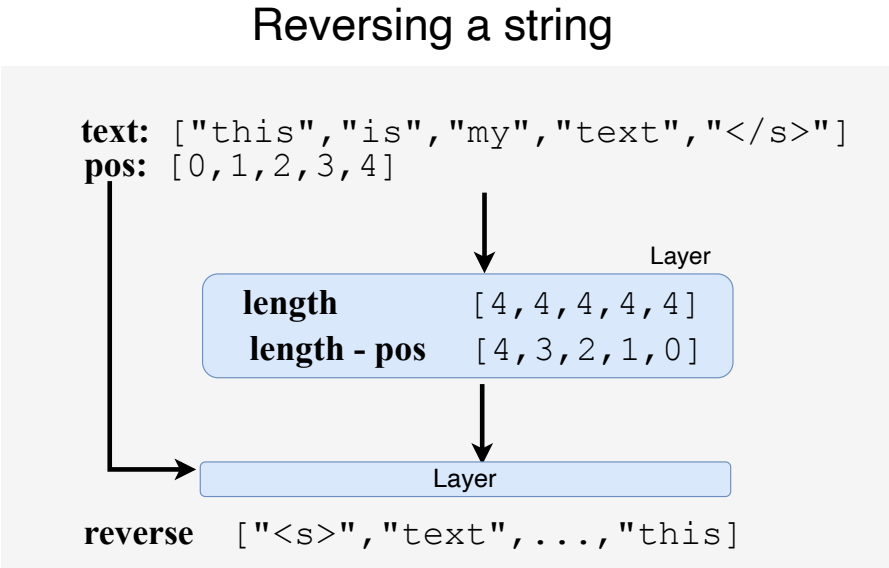
Thinking like a transformer: reversing a string

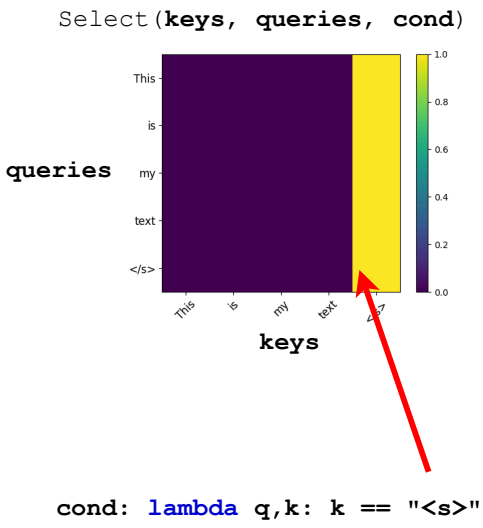


Thinking like a transformer: reversing a string



Thinking like a transformer: reversing a string

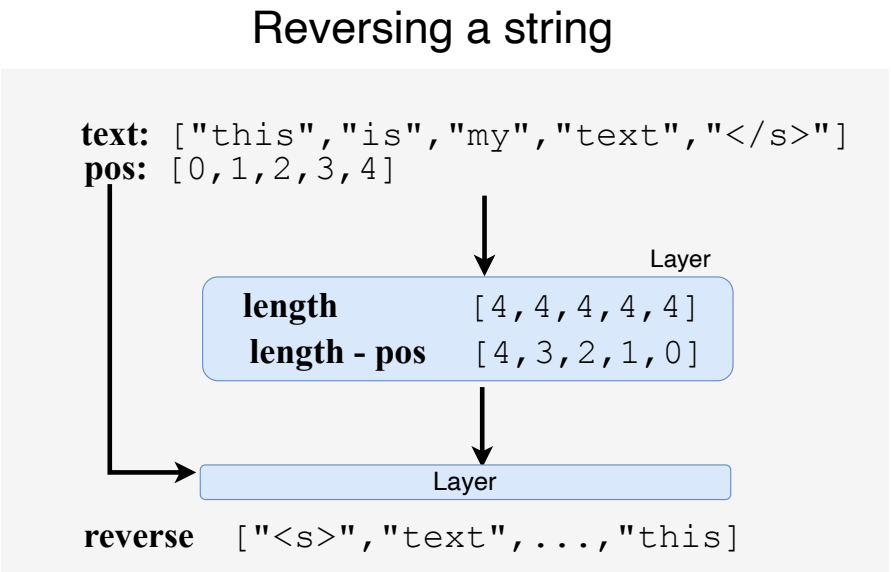


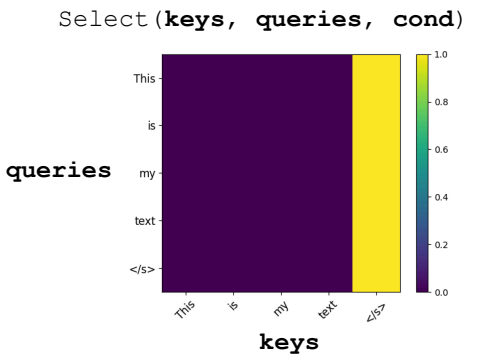


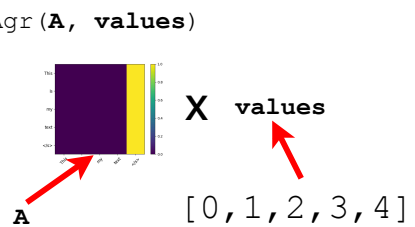
Thinking like a transformer: reversing a string



Thinking like a transformer: reversing a string







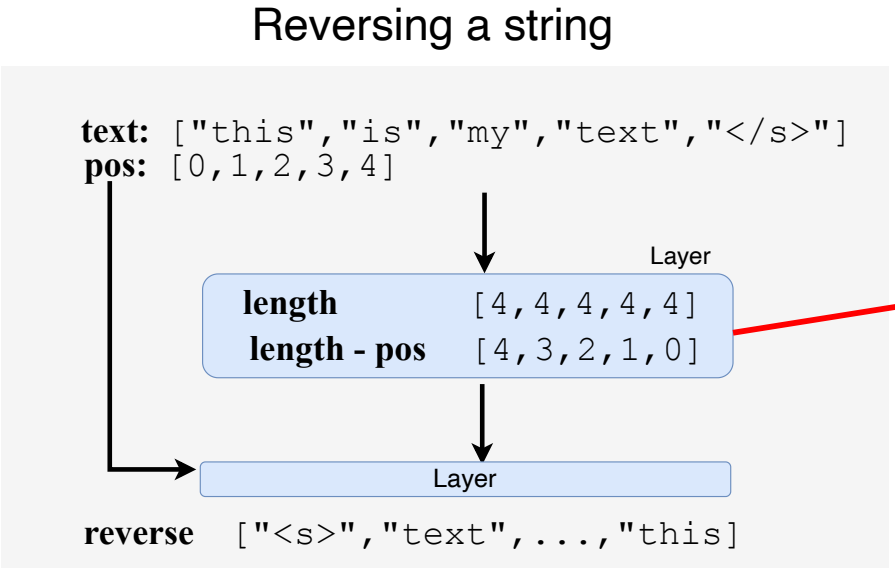
Thinking like a transformer: reversing a string

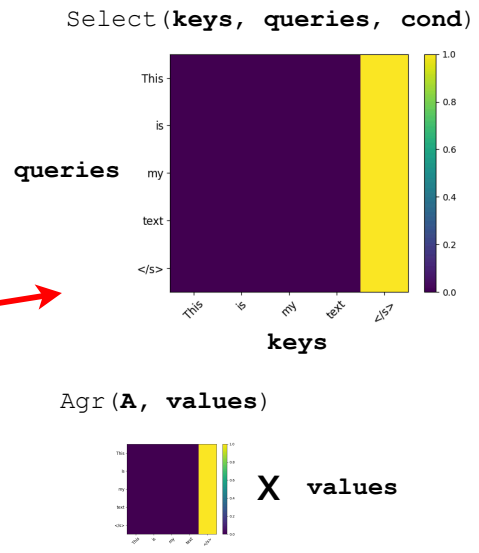
Reversing a string

res=aggregate(s, [4,6,8])

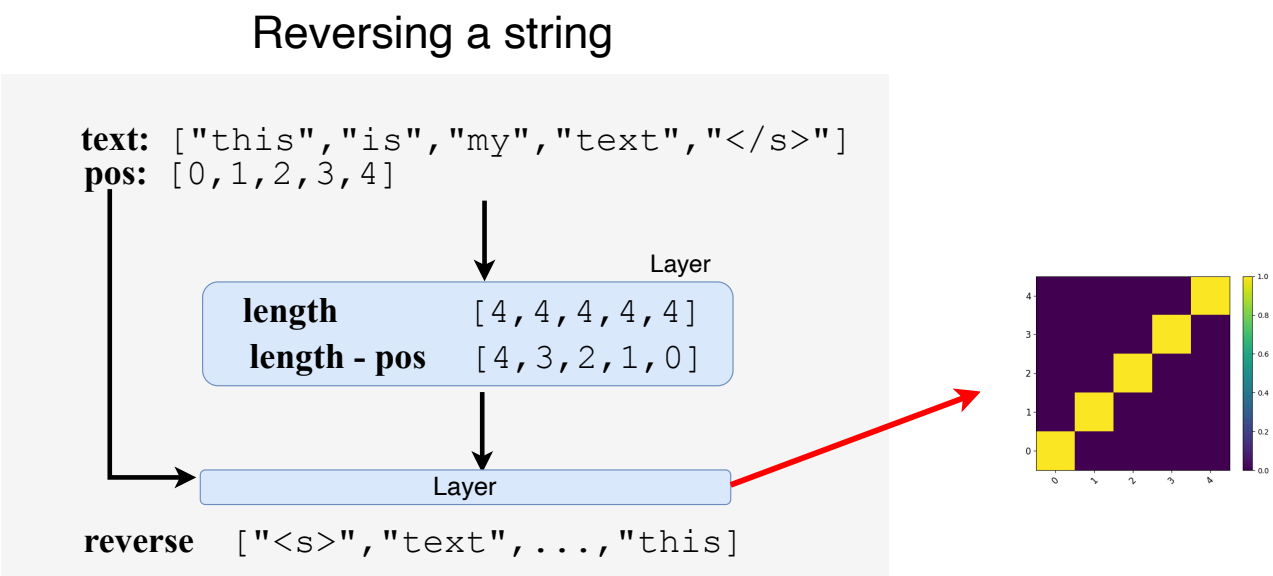
			4	**6**	**8**			
F	F	F	4	6	8	=>	0	
T	F	F	**4**	6	8	=>	4	=> **[0,4,7]**
F	**T**	**T**	4	**6**	**8**	=>	7	

Thinking like a transformer: reversing a string





Thinking like a transformer: reversing a string



Idea of Rasp: associate transformer parts with **symbolic computation**

attention $\mapsto$ `select` + `aggregate`

MLP $\mapsto$ element-wise operations

A small implementation of Rasp

```
## credits: https://github.com/apple/ml-np-rasp
import numpy as np

def indices(x):
    return np.arange(len(x), dtype=int)
def select(k,q,pred):
    s = len(k)
    A = np.zeros((s, s), dtype=bool)
    for i in range(s):
        for j in range(s): A[i,j] = pred(q[i],k[j])
    return A
def sel_width(A):
    return np.dot(A, np.ones(len(A))).astype(int)
def aggr(A, v):
    out = np.dot(A, v)
    norm = sel_width(A)
    out = np.divide(out,norm,where=(norm != 0))
    return out.astype(int)
```

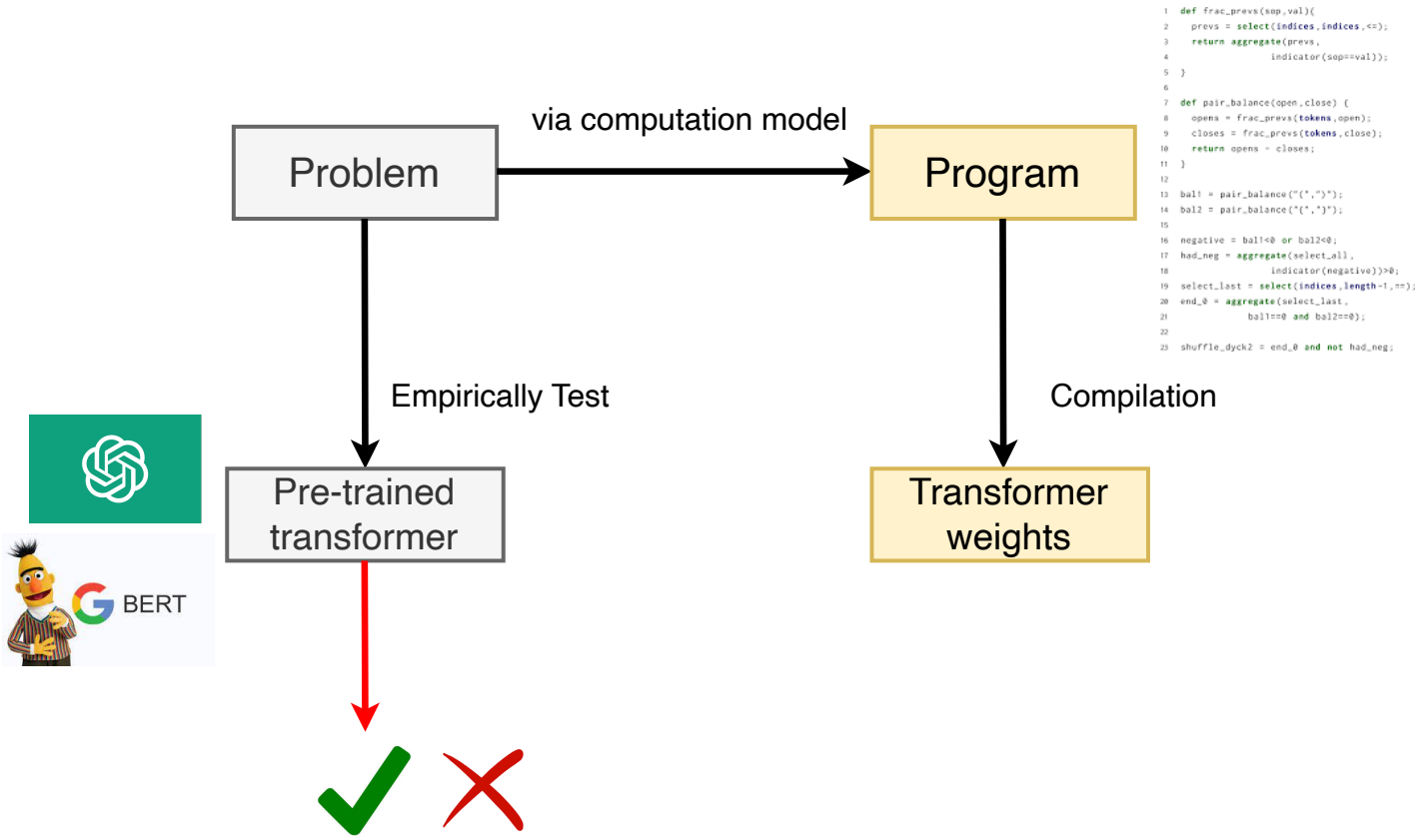
A small implementation of Rasp

```
pos = indices(tokens)
EOS = 0

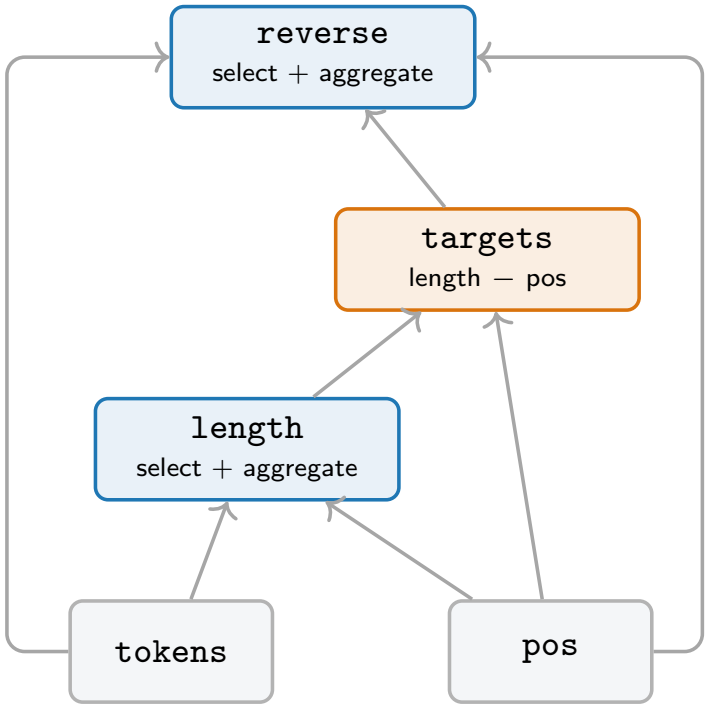
#### layer 1
# attn layer
length = aggr(
 select(tokens,tokens,lambda q,k : k == EOS),
 pos
)
targets = length - pos #<--- MLP

#### layer 2
output = aggr(
 select(pos,targets,lambda k,v: k == v),
 tokens
)
```

The bigger picture now



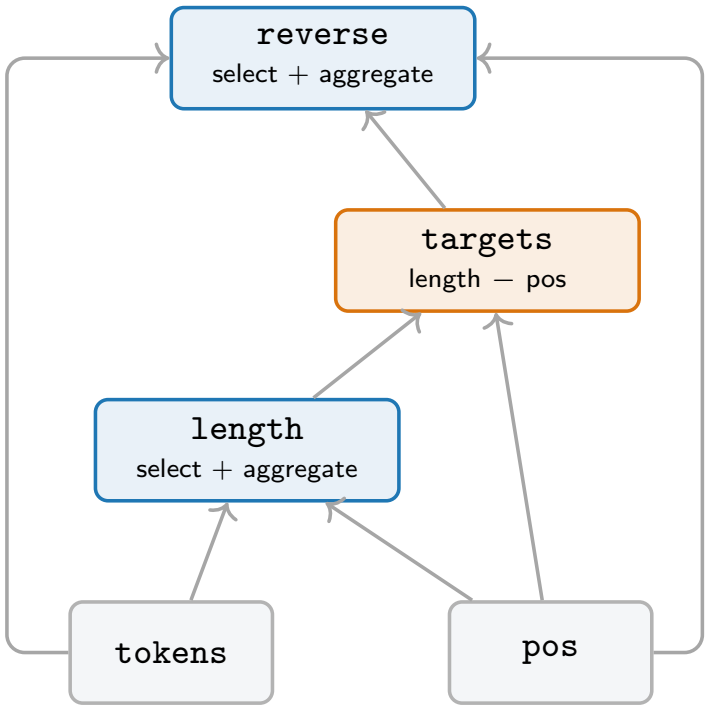
From a program to a transformer, in principle



1. The program is a graph

nodes: operations; edges: what each one reads

From a program to a transformer, in principle



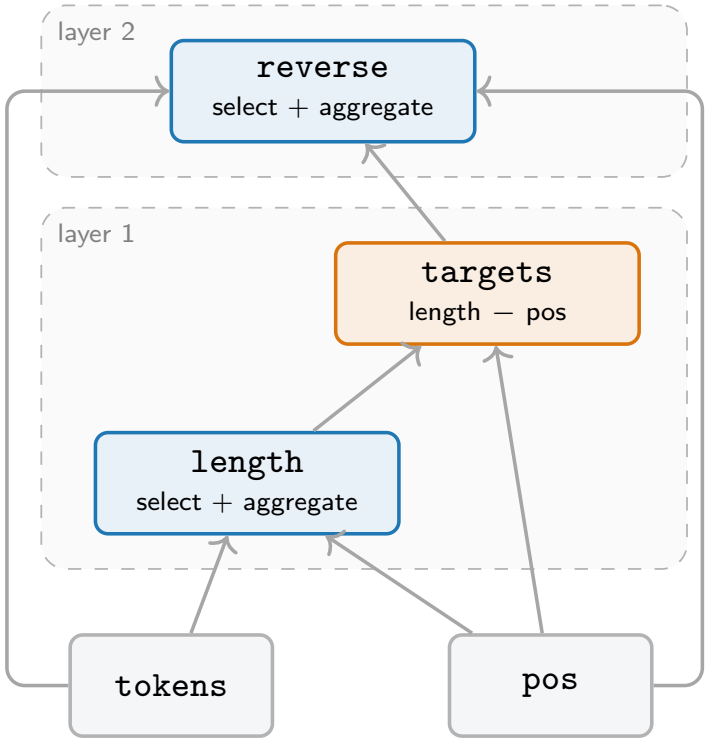
1. The program is a graph

nodes: operations; edges: what each one reads

2. Each node is a transformer part

- select + aggregate → **attention head**
- element-wise → **MLP**
- variables → **residual** stream

From a program to a transformer, in principle



1. The program is a graph

nodes: operations; edges: what each one reads

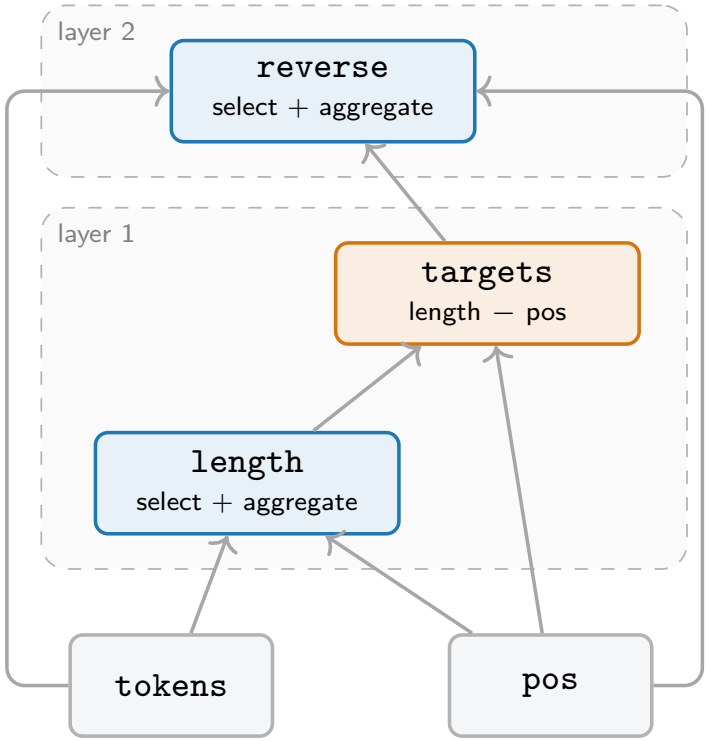
2. Each node is a transformer part

- select + aggregate → **attention head**
- element-wise → **MLP**
- variables → **residual** stream

3. Graph depth gives the layers

layer 1: `length`, then `targets`
layer 2: `reverse` (head)

From a program to a transformer, in principle



1. The program is a graph

nodes: operations; edges: what each one reads

2. Each node is a transformer part

- select + aggregate → **attention head**
- element-wise → **MLP**
- variables → **residual** stream

3. Graph depth gives the layers

layer 1: `length`, then `targets`
layer 2: `reverse` (head)

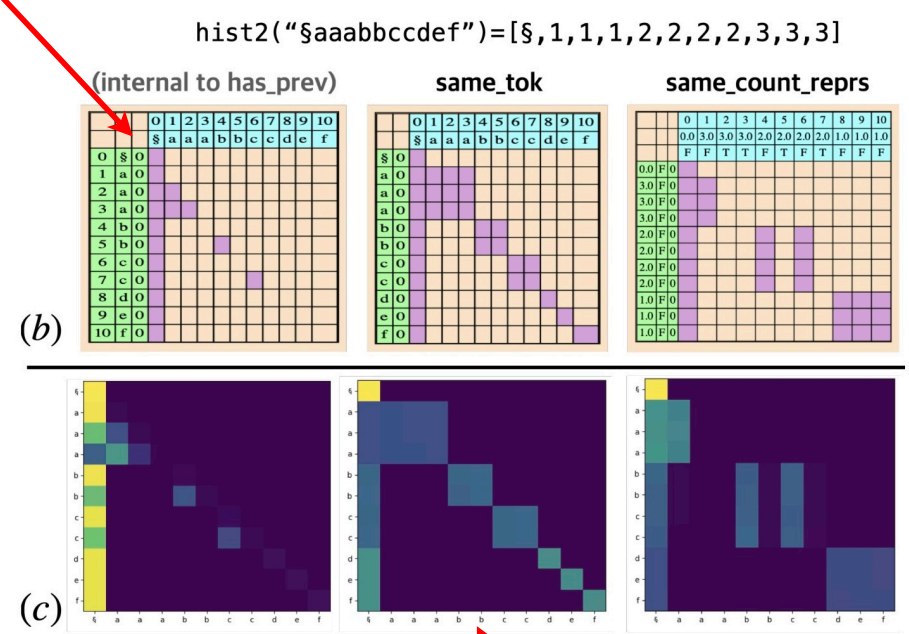
So: *some* transformer with 2 layers, 1 head each, reverses strings.
An existence argument (an upper bound); training may not find it.

Does a transformer implement my algorithm?

Target attention

```
1  same_tok = select(tokens,tokens,==);
2  hist = selector_width(
3              same_tok,
4              assume_bos = True);
5
6  first = not has_prev(tokens);
7  same_count = select(hist,hist,==);
8  same_count_reprs = same_count and
9      select(first,True,==);
10
11 hist2 = selector_width(
12             same_count_reprs,
13             assume_bos = True);
```

(a)



Rasp program

Language	Layers	Heads	Test Acc.	Attn. Matches?
Reverse	2	1	99.99%	✓*
Hist BOS	1	1	100%	✓
Hist no BOS	1	2	99.97%	✓*
Double Hist	2	2	99.58%	✓*
Sort	2	1	99.96%	✗
Most Freq	3	2	95.99%	✗
Dyck-1 PTF	2	1	99.67%	✓*
Dyck-2 PTF [8]	3	1	99.85%	✗

Real transformer attention patterns

Try it yourself: programs to write in Rasp

	task	example
1.	**length**, without an end symbol	`[3,1,4,1,5]` → `[5,5,5,5,5]`
2.	**histogram**: count of each token	`[3,1,4,1,5]` → `[1,2,1,2,1]`
3.	**shift** right by one	`[3,1,4,1,5]` → `[0,3,1,4,1]`
4.	**reverse**, without an end symbol	`[3,1,4,1,5]` → `[5,1,4,1,3]`
5.	**sort** distinct tokens	`[3,1,4,5,2]` → `[1,2,3,4,5]`
6.	**sort** with repeats (harder)	`[3,1,4,1,5]` → `[1,1,3,4,5]`

Use only `indices`, `select`, `aggr`, `sel_width` and element-wise operations. Then ask: how many layers and heads does your program need?

Recent efforts

Tracr: Compiled Transformers as a Laboratory for Interpretability

David Lindner[†] Google DeepMind
János Kramár Google DeepMind
Sebastian Farquhar Google DeepMind
Matthew Rahtz Google DeepMind
Thomas McGrath Google DeepMind
Vladimir Mikulik[†] Google DeepMind

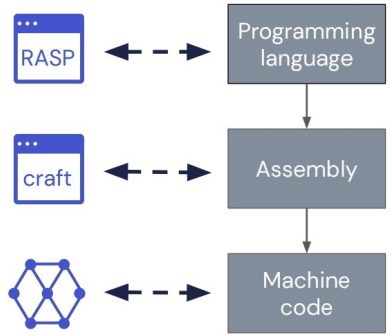


Figure 3: `Tracr` translates RASP to `Craft` and then to model weights, analogous to how programming languages are first translated to assembly then to machine code.

Recent efforts

Learning Transformer Programs

Dan Friedman **Alexander Wettig** **Danqi Chen**
Department of Computer Science & Princeton Language and Intelligence
Princeton University
{dfriedman,awettig,danqic}@cs.princeton.edu

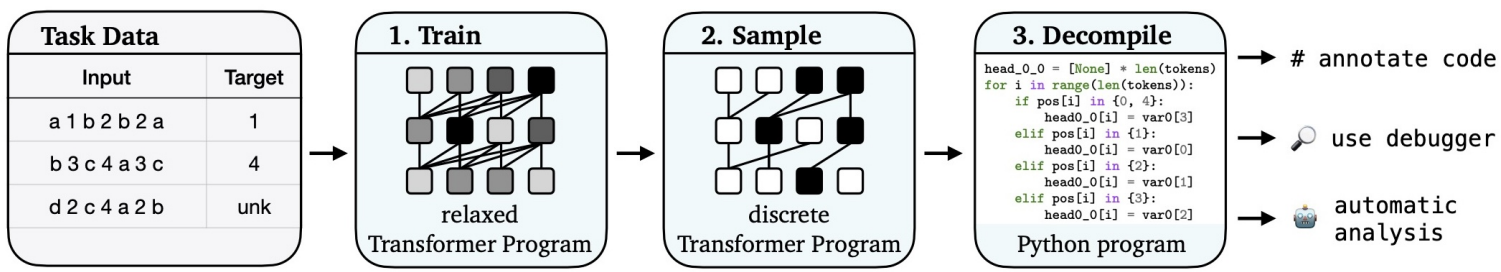
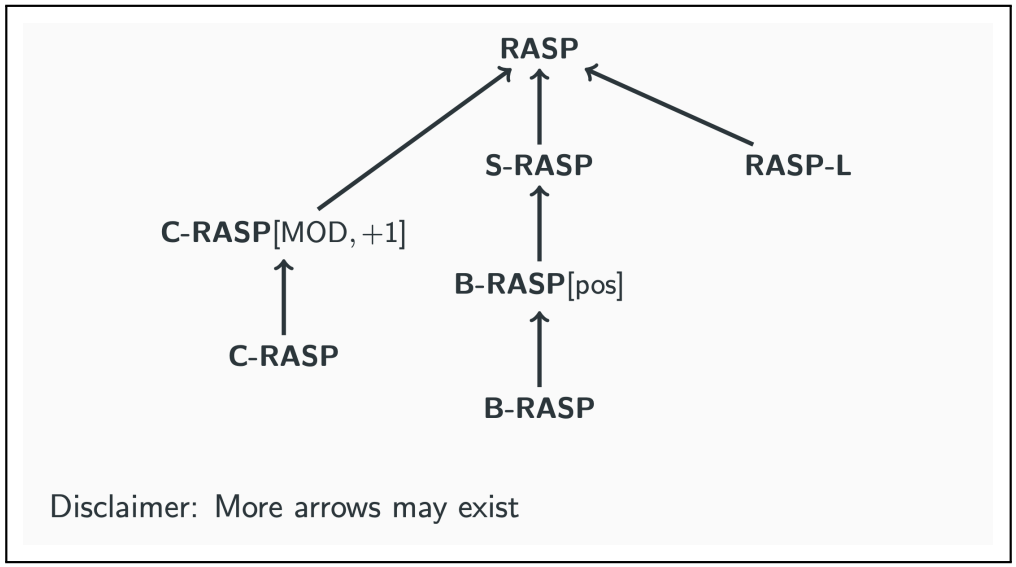


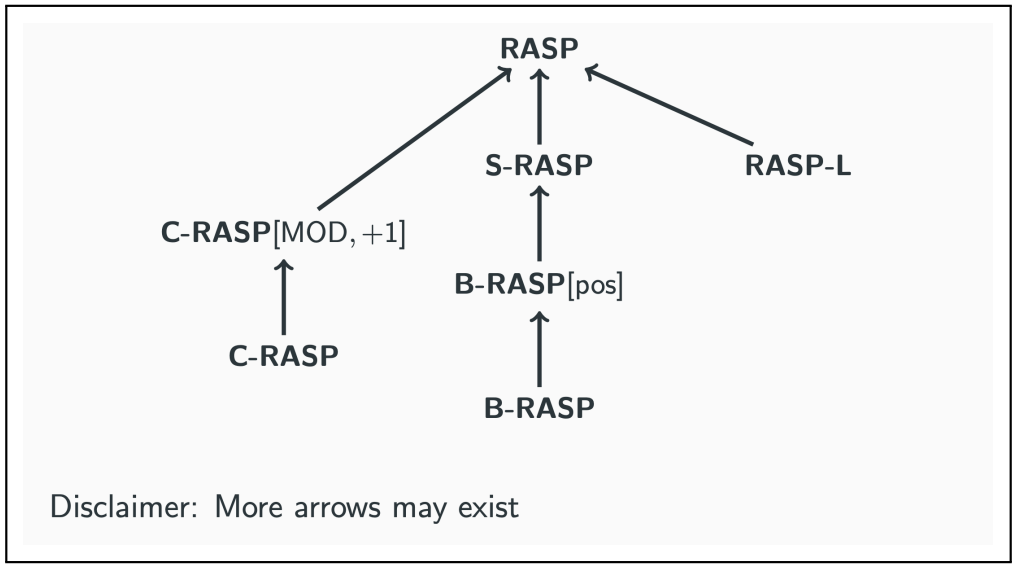
Figure 1: We design a modified Transformer that can be trained on data and then automatically discretized and converted into a human-readable program. The program is functionally identifical to the Transformer, but easier to understand—for example, using an off-the-shelf Python debugger.

Different variants of Rasp



Taken from Andy Yang https://pentagonalize.github.io/files/CLAY_slides.pdf

Different variants of Rasp



Taken from Andy Yang https://pentagonalize.github.io/files/CLAY_slides.pdf

Why variants? Each matches a precise kind of transformer, so claims become **theorems**: what transformers can and cannot express, with provable bounds.

e.g., B-RASP = masked hard-attention transformers = star-free languages (Yang et al.2024)

e.g., C-RASP adds counting; compiles into softmax transformers (Yang and Chiang2024)

Different variants of Rasp

Some of these papers are example projects

Why variants? Each matches a precise kind of transformer, so claims become **theorems**: what transformers can and cannot express, with provable bounds.

e.g., B-RASP = masked hard-attention transformers = star-free languages (Yang et al.2024)

e.g., C-RASP adds counting; compiles into softmax transformers (Yang and Chiang2024)

Conclusion: what you should know

Language models

- ✓ chain rule: $\prod_j p(w_j \mid w_{<j})$
- ✓ discrete vs. continuous
- ✓ pre-training: the next word

Conclusion: what you should know

Language models

- ✓ chain rule: $\prod_j p(w_j \mid w_{<j})$
- ✓ discrete vs. continuous
- ✓ pre-training: the next word

Transformers

- ✓ attention: contextual $\mathbf{y}_j$
- ✓ keys, queries, values; heads
- ✓ residual stream; causal mask

Conclusion: what you should know

Language models

- ✓ chain rule: $\prod_j p(w_j \mid w_{<j})$
- ✓ discrete vs. continuous
- ✓ pre-training: the next word

Transformers

- ✓ attention: contextual $\mathbf{y}_j$
- ✓ keys, queries, values; heads
- ✓ residual stream; causal mask

Rasp: the idea

- ✓ attention $\mapsto$ `select`, `aggr`
- ✓ MLP $\mapsto$ element-wise ops
- ✓ algorithms as programs

Conclusion: what you should know

Language models

- ✓ chain rule: $\prod_j p(w_j \mid w_{<j})$
- ✓ discrete vs. continuous
- ✓ pre-training: the next word

Transformers

- ✓ attention: contextual $\mathbf{y}_j$
- ✓ keys, queries, values; heads
- ✓ residual stream; causal mask

Rasp: the idea

- ✓ attention $\mapsto$ `select`, `aggr`
- ✓ MLP $\mapsto$ element-wise ops
- ✓ algorithms as programs

Rasp: why it matters

- ✓ program depth $\rightarrow$ layers
- ✓ existence, not learnability
- ✓ variants: provable claims

Conclusion: what you should know

Language models

- ✓ chain rule: $\prod_j p(w_j \mid w_{<j})$
- ✓ discrete vs. continuous
- ✓ pre-training: the next word

Rasp: the idea

- ✓ attention $\mapsto$ `select`, `aggr`
- ✓ MLP $\mapsto$ element-wise ops
- ✓ algorithms as programs

Transformers

- ✓ attention: contextual $\mathbf{y}_j$
- ✓ keys, queries, values; heads
- ✓ residual stream; causal mask

Rasp: why it matters

- ✓ program depth $\rightarrow$ layers
- ✓ existence, not learnability
- ✓ variants: provable claims

Next time: fine-tuning with constraints (semantic loss, WMC).

References I

Beurer-Kellner, L., Fischer, M., and Vechev, M. (2023). Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

De Raedt, L. and Kimmig, A. (2015). Probabilistic (logic) programming concepts. *Machine Learning*, 100:5–47.

De Raedt, L., Kimmig, A., and Toivonen, H. (2007). Problog: A probabilistic prolog and its application in link discovery. In *IJCAI 2007, Proceedings of the 20th international joint conference on artificial intelligence*, pages 2462–2467. IJCAI-INT JOINT CONF ARTIF INTELL.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

Dries, A., Kimmig, A., Meert, W., Renkens, J., Van den Broeck, G., Vlasselaer, J., and De Raedt, L. (2015). Problog2: Probabilistic logic programming. In *Joint european conference on machine learning and knowledge discovery in databases*, pages 312–315. Springer.

References II

Fierens, D., Van den Broeck, G., Renkens, J., Shterionov, D., Gutmann, B., Thon, I., Janssens, G., and De Raedt, L. (2015). Inference and learning in probabilistic logic programs using weighted boolean formulas. *Theory and Practice of Logic Programming*, 15(3):358–401.

Huang, X., Bakalova, A., Bhattamishra, S., Merrill, W., and Hahn, M. (2026). Discovering interpretable algorithms by decompiling transformers to rasp. *arXiv preprint arXiv:2602.08857*.

Mandel, L., Baudart, G., Vaziri, M., and Hirzel, M. (2026). Ppdl: Llm-based flows as probabilistic programs. In *Forty-third International Conference on Machine Learning*.

Manhaeve, R., Dumancic, S., Kimmig, A., Demeester, T., and De Raedt, L. (2018). Deepproblog: Neural probabilistic logic programming. *Advances in neural information processing systems*, 31.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008.

Weiss, G., Goldberg, Y., and Yahav, E. (2021). Thinking like transformers. In *International Conference on Machine Learning*, pages 11080–11090. PMLR.

References III

Wu, C., Yang, S., and Murali, A. (2026). Imprompt: A language framework for prompt programming. *arXiv preprint arXiv:2607.22683*.

Yang, A. and Chiang, D. (2024). Counting like transformers: Compiling temporal counting logic into softmax transformers. *arXiv preprint arXiv:2404.04393*.

Yang, A., Chiang, D., and Angluin, D. (2024). Masked hard-attention transformers recognize exactly the star-free languages. *Advances in Neural Information Processing Systems*, 37:10202–10235.